

Report

Forecasting Germany's
Consumer Price Index (CPI)
using Time Series Models

Author: Siddhanth Sharma
Matriculation No.: 7027189
Course of Studies: Business Intelligence and Data Analytics

Author: Nayan Chavan
Matriculation No.: 7027933
Course of Studies: Business Intelligence and Data Analytics

Author: Gargi Barve
Matriculation No.: 7027883
Course of Studies: Technical Management

First examiner: Prof. Dr. Elmar Wings
Submission date: May 17, 2026

Contents

Contents	3
List of Figures	9
List of Listings	12
List of Acronyms	16
I Introduction	17
1 Introduction	19
1.1 Problem Description	20
1.2 Literature Background	20
1.3 Challenges	20
1.4 Solution Approach	21
1.5 Contribution of this Work	21
1.6 Report Structure	21
II Domain Knowledge	23
2 Domain Problem	25
2.1 Application	25
2.1.1 Economic Utility and Practical Significance	25
2.1.2 Theoretical Framework in Time Series	25
2.1.3 Scientific and Methodological Suitability	25
2.2 Problem	26
2.2.1 Objective of the Problem	26
2.3 Data Acquisition	26
2.4 Data Quantity	26
2.5 Data Quality	27
2.6 Data Relevance	27
2.7 Outliers	27
2.8 Anomalies	27
3 Domain Data General	29
3.1 Dataset Identity	29
3.2 Variable and Structure	29
3.3 Temporal Characteristics	29
3.4 Strengths and Limits for Modeling	30
III Domain Knowledge - Machine Learning	31
4 Description of Important Packages	33

5	Statsmodels	35
5.1	Introduction	35
5.2	Description	35
5.3	Installation	36
5.4	Example – Description	36
5.5	Example – Manual	36
5.6	Example – Code	36
5.7	Example – Files	37
5.8	Package Role in the CPI Pipeline	37
5.9	Further Reading	37
6	Scikit-learn	39
6.1	Introduction	39
6.2	Description	39
6.3	Installation	39
6.4	Example – Description	40
6.5	Example – Manual	40
6.6	Example – Code	40
6.7	Example – Files	41
6.8	Package Role in the CPI Pipeline	41
6.9	Further Reading	42
7	Streamlit	43
7.1	Introduction	43
7.2	Description	43
7.3	Installation	44
7.4	Example – Description	44
7.5	Example – Manual	44
7.6	Example – Code	44
7.7	Example – Files	45
7.8	Package Role in the CPI Pipeline	45
7.9	Further Reading	45
8	Data Mining	47
9	Overview of Forecasting Algorithms	49
10	AutoRegressive Integrated Moving Average (ARIMA)	51
10.1	Description	51
10.2	Applications	52
10.3	Relevance	52
10.4	Hyperparameters	52
10.5	Requirements	52
10.6	Input	52
10.7	Output	52
10.8	Example with a program	54
10.9	Further Reading	54
11	Seasonal AutoRegressive Integrated Moving Average (SARIMA)	55
11.1	Description	55
11.2	Applications	55
11.3	Relevance	55
11.4	Hyperparameters	56
11.5	Requirements	56

Contents	5
11.6 Input	56
11.7 Output	56
11.8 Example with a program	56
11.9 Further Reading	57
12 Error, Trend, and Seasonality (ETS)	59
12.1 Description	59
12.2 Applications	60
12.3 Relevance	60
12.4 Hyperparameters	60
12.5 Requirements	61
12.6 Input	61
12.7 Output	61
12.8 Example with a program	61
12.9 Further Reading	62
IV Domain Knowledge - Tools	63
13 Development Environment: Python	65
13.1 Python Version	65
13.2 Python Description	65
13.3 Python Installation	66
13.4 Python Configuration	66
13.5 Python First Steps	67
13.6 Python Program: Project-Specific Sanity Check	67
V Methodology	69
14 Methodology: KDD	71
14.1 Justification for Using KDD	72
14.1.1 Data-Centric Nature Fits Time Series Projects	72
14.1.2 Structured Transformation Emphasis	72
14.1.3 Iterative, Yet Linear Flow Works Well	72
14.1.4 Interpretability of Results	73
14.1.5 Ideal for Academic and Research Projects	73
14.1.6 Final Justification Statement	73
14.2 Methodology Boundary	73
VI Application Development	75
15 Data Overview	77
15.1 Overview of the Selected Dataset	77
15.2 Modeling Frame	77
15.3 Observed Data Range	77
15.4 Time Series Characteristics	78
15.5 Train-Test Perspective	79
15.6 Raw Export Format and Parsing	79
15.6.1 Sample Raw Record	79
15.6.2 Format Properties	79
15.7 Data Quality Dimensions	80
15.8 Fairness and Bias	80

15.9 Quantity and Sufficiency	81
15.10 Data Types	81
16 Documentation Developer	83
16.1 Structure	83
16.2 Idea	83
16.3 Flow Chart of Each Step in the Development	84
16.4 ML Pipeline	84
17 KDD Data Selection	87
17.1 KDD Step 1: Database to Data Selection	87
17.1.1 Origin	87
17.1.2 Features	87
17.1.3 Properties	87
17.1.4 Data Types	87
17.1.5 Quality	88
17.1.6 Quantity	88
17.1.7 Augmentation	88
17.1.8 Fairness and Bias	88
17.2 Purpose of Data Selection	88
17.3 Selected Variables	88
17.4 Justification of the Selection	89
17.5 Selected Time Range	89
17.6 Selection Output	89
18 KDD Data Processing	91
18.1 Role of Data Processing	91
18.2 Parsing the Raw Export	91
18.3 Date Construction and Frequency	91
18.4 Cleaning the CPI Values	91
18.5 Quality Checks	92
19 KDD Data Transformation	93
19.1 Purpose of Transformation	93
19.2 Stationarity Assessment	93
19.3 Seasonal Decomposition	93
19.4 Autocorrelation and Partial Autocorrelation	93
19.5 Chronological Train-Test Split	95
19.6 Transformation Output	95
20 KDD Data Mining	97
20.1 Purpose of Data Mining	97
20.2 Model Inputs and Outputs	97
20.2.1 Input Definition	97
20.2.2 Output Definition	97
20.3 Selected Forecasting Models	98
20.3.1 ARIMA	98
20.3.2 ETS	98
20.3.3 SARIMA	99
20.4 Training Procedure	99
20.5 Evaluation Metrics	100
20.6 Residual Diagnostics	100
20.7 Interpretation of the Data-Mining Stage	101

21 Documentation Developer	103
21.1 Structure	103
21.2 Idea	103
21.3 Flow Chart	103
21.4 ML Pipeline	103
 VII Development to Deployment	 107
22 Development to Deployment	109
22.1 Data Structure	109
22.2 Tools	109
22.3 File Structure for Model Exchange	109
22.4 Description of the File Types	109
22.5 Saving Models	110
22.6 Loading Models	110
22.7 Deployed Dashboard	110
23 Development to Deployment Evaluation	113
23.0.1 Evaluation Method	113
24 Development to Deployment Validation	117
24.1 Validation Idea 1: Chronological Hold-Out Validation	117
24.2 Validation Idea 2: Model Comparison Using Error Metrics	117
24.3 Validation Idea 3: Prediction Interval and Plausibility Validation	118
24.4 Validation Idea 4: Naive Baseline Comparison	118
24.5 Validation Protocol and Current Limits	119
25 Development to Deployment Conclusion	121
25.1 Self-Critical Reflection	121
25.2 Unanswered Points	121
25.3 Next Steps	122
25.4 Final Reflection	122
 VIII Monitoring	 123
26 Monitoring	125
26.1 Monitoring Idea	125
26.2 Monitoring Plan	125
26.3 Getting New Data	126
26.4 Data Updating in the ML Pipeline	126
26.5 Checks	126
26.5.1 Baseline and Latest Metrics Comparison	128
26.5.2 Monitoring Report Format	128
26.6 Code: Functions	129
26.7 Code: Test	129
26.8 Privacy	131
26.9 Robustness	131
26.9.1 Alerting and Notification	132
26.10 Process	132

IX Application Deployment	133
27 Application Deployment	135
27.1 Application Description	135
27.2 Structure	135
27.2.1 Sidebar Controls	135
27.2.2 Dashboard Runtime Stages	135
27.3 Idea	136
27.4 Flow Chart of Each Step in the Deployment	136
27.5 Dashboard Screenshots	136
27.5.1 Model Comparison Mode	137
27.6 ML Pipeline	137
27.7 Program Readability and Modules	141
27.8 Parameter Handling	141
27.9 Error and Message Handling	141
27.10 Testing and Test Automation	141
27.10.1 CI/CD Integration	142
27.11 Deployment Test Protocol	142
X Appendix	145
28 Bill of Material	147
28.0.1 Hardware Bill of Materials	147
28.0.2 Laptop	147
28.0.3 Keyboard	147
28.0.4 Mouse	148
28.0.5 Hardware Note	148
29 SBOM	149
29.0.1 Software Bill of Materials	149
29.0.2 Python Environment	149
29.0.3 Project Dependencies	150
29.0.4 Visual Studio Code (VS Code)	150
29.0.5 Version Control and Documentation	151
Literature	153

List of Figures

1.1	Overview of Germany’s monthly Consumer Price Index (CPI) time series used in this project. The long-run upward movement, local fluctuations, and more recent inflation surge motivate the need for structured forecasting models. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].	19
5.1	Package-level view of statsmodels within the CPI pipeline. In KDD terms, the figure corresponds primarily to the data-mining stage, where the cleaned and transformed CPI series is passed to the forecasting models, fitted through repository wrappers, and converted into predictions with confidence intervals.	37
6.1	Package-level view of scikit-learn within the CPI pipeline. In KDD terms, the figure corresponds mainly to the interpretation and evaluation stage, where model forecasts and actual holdout values are converted into comparable performance metrics.	41
7.1	Package-level view of Streamlit within the CPI pipeline. The figure complements the KDD methodology by showing the downstream deployment and presentation layer in which previously trained artifacts are loaded, configured by the user, used to generate forecasts and metrics, and displayed as an interactive dashboard.	45
10.1	Architectural flowchart of an $ARIMA(p, d, q)$ model showing how the differenced signal is split into AutoRegressive and Moving Average paths before recombination. Source: own diagram based on the Box–Jenkins $ARIMA$ formulation [Box+15; Ham94].	51
10.2	Autocorrelation (ACF) and Partial Autocorrelation (PACF) plots used to empirically determine the Moving Average (q) and AutoRegressive (p) hyperparameters respectively. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].	53
10.3	Original CPI Series (top) exhibiting a non-stationary upward trend, compared to the First-Differenced CPI Series (bottom) which achieves the stationarity required by $ARIMA$. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].	53
11.1	Architectural flowchart of a $SARIMA$ model illustrating how the algorithm applies the backshift operator B to construct and multiply non-seasonal and seasonal mathematical polynomials. Source: own diagram based on seasonal Box–Jenkins modeling [Box+15; HA21].	55
12.1	Time Series Decomposition of the CPI dataset, illustrating the fundamental components modeled by the ETS framework: the observed CPI, the underlying Trend, the cyclic Seasonality, and the stochastic Residual Error. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].	59
12.2	State transition diagram for ETS. Past observations update the distinct internal states governed by mathematical smoothing parameters (α, β, γ) , which are then recombined to produce the forecast $\hat{y}_t$. Source: own diagram based on exponential smoothing state-space formulations [HA21].	60
13.1	Python environment installation workflow for the CPI forecasting project.	66
13.2	Project configuration and directory verification workflow.	67
13.3	Initial verification and testing workflow for the Python environment.	68

14.1	KDD-oriented machine-learning pipeline used in the CPI forecasting project. The lower layer captures the development workflow from data selection to evaluation and verification, while the upper layer represents deployment, result serving, new data intake, and monitoring-driven feedback.	71
15.1	Decomposition of the CPI series into trend, seasonal, and residual components. The plot supports the interpretation of CPI as a structured time series rather than an independent sequence of observations. Source: own visualization based on Destatis GENESIS table 61111-0002 and additive decomposition of the cleaned series [Sta23c; Sta23a].	78
16.1	Main development steps of the CPI forecasting system.	84
16.2	Simplified machine-learning pipeline of the CPI forecasting project.	85
17.1	KDD transition from the Destatis database to the selected CPI modeling dataset.	87
19.1	Differenced CPI series used to inspect stationarity and short-term movement after removing the dominant level trend. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].	94
19.2	ACF and PACF diagnostics for the transformed CPI series. These plots support the interpretation of temporal dependence and model-order choice. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].	94
20.1	Forecast comparison showing training data, actual test CPI, and forecasts from all three models over the full relevant time range. Source: own visualization based on Destatis GENESIS table 61111-0002 and repository model outputs [Sta23c; Sta23a].	100
20.2	Zoomed view of the 24-month test period, comparing actual CPI values against ARIMA, ETS, and SARIMA forecasts. This view makes the divergence between models and the actual series easier to inspect. Source: own visualization based on Destatis GENESIS table 61111-0002 and repository model outputs [Sta23c; Sta23a].	101
21.1	KDD-oriented developer view of the CPI forecasting workflow.	104
22.1	Transition of trained forecasting models from development to deployment.	110
22.2	Live Streamlit dashboard in the ARIMA view, showing the model selector, confidence interval control, metric cards, and forecast chart with a 95% confidence interval.	111
22.3	Expanded model selector in the live dashboard, showing the three individual model views and the all-model comparison mode.	112
22.4	Live comparison mode showing the metrics table and the forecast curves for ARIMA, ETS, and SARIMA against the actual test data.	112
23.1	Zoomed forecast comparison for the 24-month test period, showing actual CPI against ARIMA, ETS, and SARIMA predictions. Source: own visualization based on Destatis GENESIS table 61111-0002 and repository model outputs [Sta23c; Sta23a].	114
26.1	Monitoring and monthly update workflow for the CPI forecasting system.	127
27.1	Deployment flow of the Streamlit CPI forecasting application.	137
27.2	Live dashboard view for ARIMA $(1, 1, 1)$, including metric cards and the forecast chart with a 95% confidence band.	138
27.3	Expanded model selector in the live dashboard, showing ARIMA, ETS, SARIMA, and the all-model comparison option.	139
27.4	Live dashboard view for ETS (A, A, A) , showing the lowest error metrics among the implemented models and the corresponding forecast chart.	139
27.5	Live dashboard view for SARIMA $(1, 1, 1) \times (1, 1, 1, 12)$, showing the seasonal model's metrics and forecast chart.	140

27.6	Live comparison mode showing the metrics table and an overlaid forecast chart for ARIMA, ETS, and SARIMA.	140
27.7	Conceptual CI/CD pipeline for the CPI forecasting project.	143

Listings

5.1	Installing project dependencies including statsmodels	36
5.2	Installing statsmodels directly	36
5.3	Verifying the statsmodels installation	36
5.4	Project-specific statsmodels model definitions	36
6.1	Installing project dependencies including scikit-learn	39
6.2	Installing scikit-learn directly	40
6.3	Verifying the scikit-learn installation	40
6.4	Forecast evaluation with scikit-learn metrics	41
7.1	Installing project dependencies including Streamlit	44
7.2	Installing Streamlit directly	44
7.3	Verifying the Streamlit installation	44
7.4	Typical Streamlit controls used in the CPI dashboard	44
10.1	ARIMA Implementation using Statsmodels	54
11.1	SARIMA Implementation using Statsmodels	57
12.1	ETS Implementation using Statsmodels	61
13.1	Installing the project environment	66
13.2	Project-specific Python sanity check	67
26.1	Simplified excerpt from a monitoring report JSON file	128
26.2	Monitoring workflow orchestration in the utility layer	129
26.3	Monitoring-oriented test command	129
26.4	Example monitoring assertions in the test suite	131
27.1	Running the pytest suite	141
29.1	Project dependencies from <code>Code/requirements.txt</code>	150

List of Acronyms

ACF	Autocorrelation Function
ADF	Augmented Dickey-Fuller
AIC	Akaike Information Criterion
API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
BIC	Bayesian Information Criterion
CI	Confidence Interval
CLI	Command Line Interface
CNC	Computerized Numerical Control
CPI	Consumer Price Index
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
ETS	Error, Trend, Seasonality
GDP	Gross Domestic Product
GENESIS	Gemeinsames Neues Statistisches Informationssystem
HICP	Harmonised Index of Consumer Prices
KDD	Knowledge Discovery in Databases
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MoM	Month-over-Month
NumPy	Numerical Python
PACF	Partial Autocorrelation Function
PDF	Portable Document Format
pip	pip installs packages
RMSE	Root Mean Squared Error
SARIMA	Seasonal AutoRegressive Integrated Moving Average
SBOM	Software Bill of Materials

SPS Speicherprogrammierbare Steuerung

SSH Secure Shell

UI User Interface

URL Uniform Resource Locator

venv Virtual Environment

VS Code Visual Studio Code

YoY Year-over-Year

Part I

Introduction

1 Introduction

Time series forecasting uses temporally ordered observations to estimate future values from patterns observed in the past. In contrast to cross-sectional prediction, the order of the data itself carries information, because persistence, trend, seasonality, and irregular shocks all influence how a series evolves over time [Box+15; Ham94; HA21; SS17]. These properties make time series methods particularly suitable for macroeconomic indicators that are published at regular intervals and whose current values are strongly related to earlier observations.

One such indicator is the **Consumer Price Index (CPI)**, which measures changes in the general price level faced by private households. CPI forecasting is relevant for inflation monitoring, short-term economic planning, and the broader analysis of price stability. In the forecasting literature, inflation and CPI are established examples of economically important series whose prediction requires careful treatment of persistence, changing regimes, and evaluation under strict time order [SW99; HA21]. In addition, official documentation from Destatis underlines the methodological importance of index quality, weighting structure, and periodic revisions for the interpretation of German CPI data [Sta23c; Sta23a]. This report studies Germany’s monthly CPI as a **univariate forecasting problem**. The underlying data comes from the **Destatis GENESIS** database, table **61111-0002**, and after preprocessing yields **422 monthly observations** from **January 1991 to February 2026**. The implemented system compares three classical forecasting approaches—**ARIMA**, **ETS**, and **SARIMA**—on the same cleaned CPI series, thereby allowing a controlled comparison of model behavior under one common data and evaluation pipeline.

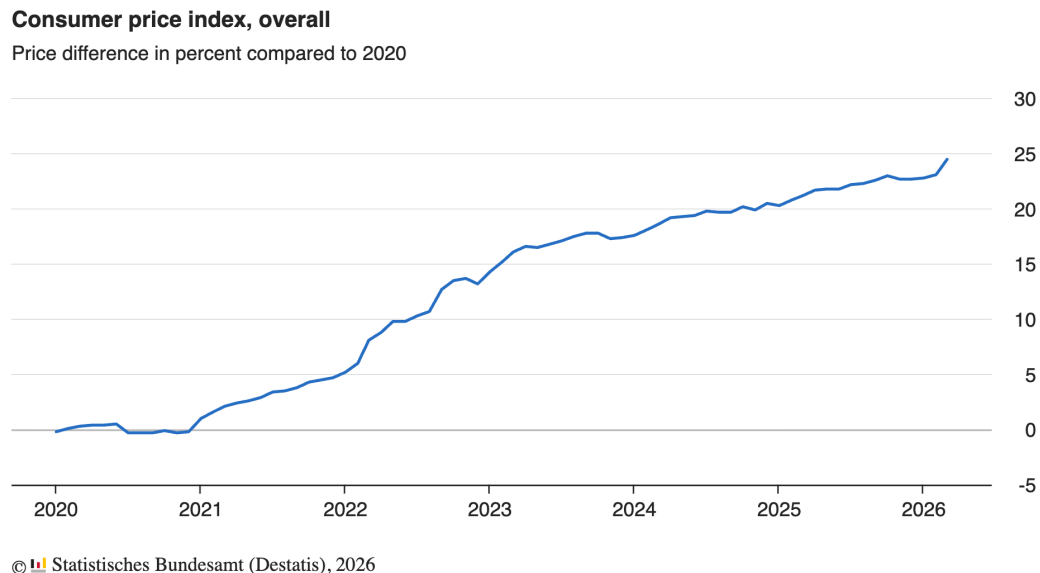


Figure 1.1: Overview of Germany’s monthly Consumer Price Index (CPI) time series used in this project. The long-run upward movement, local fluctuations, and more recent inflation surge motivate the need for structured forecasting models. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

1.1 Problem Description

The central problem addressed in this report is the forecasting of future monthly CPI values for Germany from historical CPI observations alone. More precisely, the report asks which of the selected classical models—ARIMA, ETS, or SARIMA—provides the most suitable forecast under a consistent data preparation and evaluation setting. This is a constrained forecasting problem: the series is univariate, chronologically ordered, and economically meaningful, but forecast quality depends on whether its statistical structure is represented adequately.

The implemented evaluation follows a chronological holdout design. The final **24 monthly observations** are reserved as an out-of-sample test period, while the earlier observations are used for model fitting. Forecast performance is then assessed with **RMSE**, **MAE**, and **MAPE**. This type of structured forecast evaluation is consistent with standard forecasting practice, which emphasizes transparent out-of-sample comparison instead of judging a model only by in-sample fit [Arm01; MB18]. At the introduction stage, the important point is therefore not yet which model wins, but that all candidate methods are evaluated under the same data, horizon, and metric setup so that the later comparison is methodologically fair.

1.2 Literature Background

Classical time series forecasting is well established in the literature. Box–Jenkins modeling, exponential smoothing, and seasonal autoregressive approaches all begin from the premise that a series must first be understood in terms of its dependence structure, trend behavior, and possible seasonal regularities before meaningful forecasts can be produced [Box+15; Ham94; HA21; SS17]. More general forecasting handbooks likewise emphasize that method selection should depend on data structure, horizon, and evaluation design rather than on one universally preferred algorithm [Arm01; MB18]. This is directly relevant to CPI forecasting, because inflation-related series are not independent across time; rather, they reflect persistent dynamics, policy-sensitive changes, and recurring calendar effects [SW99].

The specific model families selected for this project are likewise grounded in standard forecasting literature. ARIMA models are appropriate when autocorrelation and differencing-based stationarity are central considerations; ETS models represent level, trend, and seasonality through a state-space perspective; and SARIMA extends ARIMA by incorporating explicit seasonal terms [Box+15; HA21; SS17]. Recent CPI-oriented studies also continue to use such classical methods as meaningful baselines or direct competitors in comparative forecasting analyses [Hu25; MMM25; NSU25]. The comparison of ARIMA, ETS, and SARIMA therefore provides both methodological clarity and practical relevance.

1.3 Challenges

Several challenges arise when forecasting CPI with classical time series methods. First, the series exhibits a long-run upward movement and therefore cannot be assumed stationary without inspection and, where required, differencing or model structures that explicitly accommodate trend [Box+15; HA21]. Second, recurring seasonal effects may be present but may vary in strength across periods, making seasonal specification non-trivial. Third, macroeconomic time series are exposed to structural changes and external shocks, including energy-price movements and inflation regime shifts, so a model that fits one period well may generalize less effectively to another [SW99; Sta23b]. Fourth, official CPI series are methodologically robust but are still influenced by index construction, weighting adjustments, and data-collection conditions, all of which affect interpretation [Sta23c; Sta22].

An additional project-specific challenge is that the implemented setup is intentionally **univariate**. This design keeps the comparison transparent and reproducible, but it also means that potentially informative exogenous drivers such as energy prices, wage developments, exchange-rate effects, or monetary-policy indicators are not included directly. Consequently, the models are evaluated as pure time-series forecasters rather than as causal explanations of inflation.

Finally, forecast evaluation must respect time order. Random train-test splits would ignore temporal dependence and create misleading performance estimates. A chronological holdout procedure is therefore

required, and this choice is consistent both with forecasting best practice and with the KDD-oriented structure of the project [FPSS96; HA21; Arm01].

1.4 Solution Approach

To address these challenges, the project follows the **KDD process** adopted throughout the course and reflected in the report structure. In practical terms, this means that the forecasting problem is handled as a sequence of connected stages rather than as model fitting alone. The workflow begins with selecting German CPI from the Destatis GENESIS database, continues through preprocessing and time-series-oriented transformation, applies ARIMA, ETS, and SARIMA under one common implementation pipeline, and then evaluates the resulting forecasts on a chronological holdout period using RMSE, MAE, and MAPE [FPSS96; HA21].

The saved outputs are subsequently reused in the Streamlit application for interactive inspection of forecasts and metrics. A more detailed methodological explanation, including the full KDD-based workflow with deployment and monitoring aspects, is presented later in Chapter ??.

Beyond statistical forecasting itself, the project also focuses on the engineering and operational aspects of a forecasting workflow. The implemented system includes reproducible preprocessing pipelines, unified forecasting wrappers, chronological evaluation procedures, serialized model artifacts, and a Streamlit-based application layer for interactive forecast inspection. This broader software-oriented perspective is important because forecasting systems in practical environments must remain maintainable, reproducible, and extensible beyond the initial modeling stage [Bro17]

1.5 Contribution of this Work

The contribution of this work is a reproducible comparison of three classical univariate forecasting models for Germany's CPI under one common pipeline for data loading, preprocessing, evaluation, and presentation. In addition to model estimation, the project contributes documented metrics, generated plots, serialized model artifacts, and a lightweight dashboard for result inspection. The work is therefore not limited to a theoretical model discussion; it also demonstrates how classical forecasting methods can be implemented and assessed systematically in a university-scale software and data project.

1.6 Report Structure

The remainder of this report is organized as follows. The next part develops the domain context of CPI forecasting in Germany and describes the underlying dataset. This is followed by chapters on the software environment, the main Python libraries, and the selected forecasting algorithms. The methodology chapter then presents the KDD-oriented project structure and maps it to the implemented workflow. Subsequent chapters document the application development process, the transformation and modeling stages, and the transition from development to deployment. The later parts discuss evaluation, validation, monitoring, and the Streamlit-based application layer, before the appendix summarizes supporting technical materials for reproducibility.

Part II

Domain Knowledge

2 Domain Problem

2.1 Application

The application domain of this project is **Macroeconomic Time Series Forecasting**, specifically centered on the modeling and prediction of the **Consumer Price Index (CPI)** in Germany. Inflation forecasting is a cornerstone of empirical macroeconomics; as established by [SW99], it is the primary metric used by central banks and financial institutions to gauge price stability and formulate monetary policy. In the German context, the CPI is the definitive measure of the "cost of living," making its accurate prediction essential for maintaining economic equilibrium within the Eurozone.

2.1.1 Economic Utility and Practical Significance

The CPI serves as a Laspeyres-type index, tracking the price changes of a representative basket of goods and services over time. According to the principles discussed in [Wil07], the ability to forecast this index is critical for several real-world applications:

- **Monetary Policy Alignment:** The European Central Bank (ECB) utilizes CPI forecasts to adjust interest rates, aiming for a medium-term inflation target of 2%. Accurate forecasting prevents over-tightening or excessive easing of the economy.
- **Financial Risk Management:** Forecasting inflation is necessary for determining the "real" return on investments. As Wilmott emphasizes, without accurate CPI projections, long-term capital allocation is subject to significant **purchasing power risk**, where inflation erodes the nominal gains of an investment.
- **Contractual and Wage Indexation:** In Germany, many legal and commercial contracts—including industrial wage agreements (*Tarifverträge*) and commercial lease escalations—are programmatically tied to the **Destatis CPI** figures.

2.1.2 Theoretical Framework in Time Series

Following the framework established in [HA21], this application treats the CPI as a **Stochastic Process**. Unlike highly volatile financial assets, the CPI exhibits high **structural persistence**. This means that the current inflation level is heavily influenced by its own history, making it an ideal candidate for autoregressive modeling.

The application utilizes the [Box+15] methodology to identify the characteristics of the underlying **Data-Generating Process (DGP)**, specifically looking for:

- **Deterministic Trends:** Long-term persistent upward movements reflecting systemic economic growth or currency devaluation.
- **Seasonal Fluctuations:** Predictable monthly variations often found in energy costs (heating in winter) or the German tourism and retail sectors.

2.1.3 Scientific and Methodological Suitability

This domain is well suited to the **KDD (Knowledge Discovery in Databases)** process because the data is provided by **Destatis (GENESIS Database)**. This ensures a high level of data integrity and data quality, which are essential for reliable analysis.

By applying **ARIMA**, **ETS**, and **SARIMA** models, this project demonstrates how classical statistical learning—as outlined in [HTF09]—can be used to extract meaningful signals from noisy economic environments.

2.2 Problem

The core research problem is a **univariate time series forecasting task** targeting the German CPI. The technical challenge lies in estimating a predictive function that maps historical observations $y_t, y_{t-1}, \dots, y_{t-n}$ to a future value $\hat{y}_{t+h}$.

Following [Box+15], the problem is framed as identifying the underlying **stochastic process** that governs inflation. This requires addressing:

- **Modeling Non-Stationarity:** As discussed by [SS17], the persistent upward trend in German inflation necessitates integration (differencing) to stabilize the series.
- **Decomposition of Temporal Components:** The problem requires a model that can effectively decompose the series into trend, seasonality, and residuals [HA21].
- **Algorithm Selection:** As highlighted in [HTF09], there is a trade-off between model complexity and generalization when comparing **ARIMA**, **ETS**, and **SARIMA**.

2.2.1 Objective of the Problem

The objective is to minimize the empirical risk, measured through the loss function $\mathcal{L}(y, \hat{y})$, across the chronological holdout period. In the implemented pipeline, the last 24 monthly observations are reserved for testing, while all earlier observations are used for fitting. Using the workflow suggested in [Bro17], the task is to ensure that the chosen models remain robust enough to predict the recent inflationary environment. Success is defined by the minimization of **RMSE**, **MAE**, and **MAPE**.

2.3 Data Acquisition

The data is sourced from the **Destatis GENESIS Database**, Table **61111-0002**.

- **Data Type:** Univariate economic time series.
- **Access Method:** Exported as CSV and ingested using the `pandas` library.
- **URL Reference:** <https://www-genesis.destatis.de/datenbank/online/statistic/61111/table/61111-0002>

This ensures the "ground truth" integrity required for professional forecasting [Kar21].

2.4 Data Quantity

Following [Box+15], the dataset provides a robust volume of observations. The range extends from **January 1991 to early 2026**, totaling over **420 monthly observations**. This $N > 400$ exceeds the minimum requirements for ARIMA/SARIMA, which typically require $N > 50$ to stabilize parameter estimation.

2.5 Data Quality

According to [Bro17], data quality determines model reliability. In forecasting practice, robust data quality is also a prerequisite for credible model comparison and later decision support [Cha13; Arm01]. The Destatis dataset exhibits:

- **Chronological Integrity:** Strictly ordered from 1991 to 2026 with no irregular gaps.
- **Completeness:** No missing values (NaNs), removing the need for imputation.
- **Stationarity Potential:** High resolution allows for clean mathematical transformations (differencing) required by the [Box+15] methodology.

2.6 Data Relevance

According to [SW99], the CPI is the most relevant variable for inflation forecasting because it measures the "pass-through" effect of economic shocks. In our project:

- It aligns with German macroeconomic planning.
- It serves as a proxy for the internal purchasing power of the Euro [Wil07].
- Its strong **autocorrelation** makes it the most relevant feature for ARIMA/SARIMA models [HTF09].

2.7 Outliers

In this project, economically unusual observations—such as the temporary VAT-related dip around 2020—are treated as part of the real history of the CPI series rather than as simple measurement errors. This distinction is important, because outlier-analysis literature warns that unusual observations may either reflect noise or encode rare but meaningful events, and the correct interpretation depends on domain context [Agg17]. They are therefore retained in the dataset and interpreted during evaluation instead of being removed mechanically. This decision preserves chronological continuity and keeps the forecasting problem aligned with actual macroeconomic behavior.

2.8 Anomalies

Anomalies represent structural breaks, such as the 2021–2023 inflationary surge. According to [SW99], such shocks can alter the future path of the series substantially. More generally, anomaly-detection literature defines anomalies as observations or temporal segments that deviate strongly from the normal pattern and therefore deserve special interpretation rather than automatic deletion [CBK09]. While ARIMA and SARIMA may struggle when the underlying regime changes [Box+15], the **ETS** framework can be more robust because its level component adapts more directly to changing baselines [HA21].

3 Domain Data General

This chapter acts as a short bridge between the domain problem and the later implementation-oriented data chapters. Its purpose is not to repeat the full operational discussion from Chapter ?? or the detailed repository-grounded data analysis from the application-development part, but to summarize what kind of dataset the project works with and why it is suitable for classical time-series forecasting.

3.1 Dataset Identity

The dataset used in this project is the German monthly Consumer Price Index (CPI) from the Destatis GENESIS database, specifically table **61111-0002**. Because the data is published by the Federal Statistical Office of Germany, it provides a trustworthy and methodologically documented basis for an academic forecasting task [Sta23c; Sta23a]. In the repository, the downloaded CSV is stored as `Code/data/61111-0002_en.csv`.

Table ?? summarizes the most important identifying properties of the dataset.

Table 3.1: Compact summary of the CPI dataset used in the project. Source: own summary based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

Property	Description
Source	Destatis GENESIS Database
Table	61111-0002
Spatial scope	Germany
Frequency	Monthly
Base year	2020 = 100
Cleaned range	January 1991 to February 2026
Observations	422 monthly CPI values
Modeling type	Univariate economic time series

3.2 Variable and Structure

The target variable is the **CPI index level**. This is important because the project forecasts the index value itself, not a derived inflation percentage. The cleaned modeling structure consists of one monthly datetime index and one numeric target column, **CPI**. This makes the final dataset a univariate chronological series that can be passed directly into ARIMA, ETS, and SARIMA after preprocessing.

The raw Destatis export contains additional structural elements such as metadata rows, status markers, and percentage-change columns. Those details are relevant for preprocessing, but they are intentionally discussed later in the application-development chapters, where the repository’s actual parsing and cleaning steps are documented in detail.

3.3 Temporal Characteristics

The dataset spans more than three decades and therefore covers different inflation environments rather than only one short stable period. This long monthly history is sufficient for comparing classical forecasting models that rely on trend, persistence, and potential annual seasonality. The CPI level also shows the

expected long-run upward movement of a price index, which makes questions of non-stationarity and transformation methodologically relevant for the later modeling stages [HA21; Box+15].

At the same time, the series contains economically meaningful irregular periods, including the post-2021 inflation surge and shorter policy- or energy-related disturbances [Sta22; Sta23b]. These phenomena matter because they make the forecasting task more realistic, but their detailed treatment as anomalies, outliers, and preprocessing decisions is covered later where the implementation and evaluation are discussed more concretely.

3.4 Strengths and Limits for Modeling

The main strength of the dataset is its official origin, regular monthly frequency, and long historical coverage. Together, these properties make it a good candidate for a controlled comparison of classical time-series methods. The main limitation is the deliberately univariate design: the models only receive past CPI values and do not directly use exogenous drivers such as wages, exchange rates, taxes, energy prices, or monetary-policy variables. As a result, the later forecasts should be interpreted as statistical continuation models of CPI behavior rather than as causal explanations of inflation [SW99].

In short, this chapter establishes that the dataset is methodologically credible, structurally suitable for classical time-series analysis, and limited in exactly the way the project intends. The more detailed discussion of raw format, preprocessing rules, anomalies, and data quality is intentionally deferred to the later development chapters so that the report avoids unnecessary duplication.

Part III

Domain Knowledge - Machine Learning

4 Description of Important Packages

The implementation of the CPI forecasting system relies on a focused set of Python libraries with clearly separated roles. In this project, package selection is not a secondary technical detail, because the complete KDD workflow depends on stable support for data handling, statistical modeling, evaluation, visualization, serialization, and deployment. Reproducible analytical workflows depend not only on algorithms, but also on the software environment in which those algorithms are executed and evaluated [HA21]. The report therefore documents all important packages explicitly and ties them to the actual repository files instead of describing them only in generic terms.

The project dependency list in [Code/requirements.txt](#) contains eight direct Python packages: **pandas**, **matplotlib**, **statsmodels**, **scikit-learn**, **jinja2**, **joblib**, **streamlit**, and **plotly**. In addition, **NumPy** is used throughout the codebase as a foundational numerical library, even though it is pulled in transitively through **pandas** and **statsmodels**. Table ?? gives a complete mapping of every package to its role in the CPI forecasting pipeline.

Table 4.1: Complete overview of Python packages used in the CPI forecasting system.

Package	Main role in the project	Primary repository context
pandas	Data loading, cleaning, time-index construction, and CSV export	Code/src/data_loader.py , Code/src/modeling_pipeline.py
NumPy	Numerical arrays, missing-value handling, and metric computation	Code/src/data_loader.py , Code/src/modeling_pipeline.py , Code/app.py
matplotlib	Static plot generation for EDA and forecast comparison	Code/src/eda_pipeline.py , Code/src/modeling_pipeline.py
statsmodels	Forecast model estimation, parameter fitting, and prediction	Code/src/models.py , Code/src/modeling_pipeline.py
scikit-learn	Forecast evaluation metrics for chronological holdout testing	Code/src/modeling_pipeline.py , Code/app.py
joblib	Serialization and deserialization of trained model objects	Code/src/modeling_pipeline.py , Code/app.py
plotly	Interactive chart rendering inside the Streamlit dashboard	Code/app.py
streamlit	Interactive deployment and user-facing forecast interpretation	Code/app.py
jinja2	Templating dependency (pulled in by pandas and plotly)	Transitive dependency

Three libraries are especially central to the implemented workflow and are therefore documented in dedicated chapters that follow this one. First, **statsmodels** provides the statistical forecasting engines used to fit the ARIMA, ETS, and SARIMA models in [Code/src/models.py](#) [sta26]. Second, **scikit-learn** supplies the evaluation metrics used in [Code/src/modeling_pipeline.py](#) to compare forecast accuracy through RMSE, MAE, and MAPE [sci26]. Third, **Streamlit** is used in [Code/app.py](#) to convert the trained forecasting system into an interactive application for visual comparison and interpretation [Str26].

The remaining packages—**pandas**, **NumPy**, **matplotlib**, **plotly**, and **joblib**—are briefly summarized below because they perform essential tasks that the pipeline cannot complete without, but they do not require the same depth of documentation as the three primary analytical and deployment libraries.

Pandas is the data-manipulation backbone of the project. It handles CSV ingestion, datetime indexing,

train-test splitting, and result export in [Code/src/data_loader.py](#) and [Code/src/modeling_pipeline.py](#). Without it, the Destatis export could not be converted into a strict monthly time series that forecasting libraries can consume.

NumPy underpins every numerical operation performed by **pandas**, **statsmodels**, **scikit-learn**, and **matplotlib**. In the repository it is used directly for missing-value representation (`np.nan`) in [Code/src/data_loader.py](#) and for RMSE computation (`np.sqrt`) in [Code/src/modeling_pipeline.py](#). Its foundational role in numerical Python workflows is well documented in the scientific-computing literature, where efficient array operations are treated as a core enabler for reproducible analytical programming [Har+20].

Matplotlib generates the static diagnostic and forecast plots that are saved to [Code/plots/](#). It is used in [Code/src/eda_pipeline.py](#) for historical series, differencing, ACF/PACF, and decomposition figures, and in [Code/src/modeling_pipeline.py](#) for the forecast comparison plots.

Plotly renders the interactive charts inside the Streamlit dashboard. It is used exclusively in [Code/app.py](#) to create line charts with hover, zoom, and pan capabilities that help users explore forecast details without modifying code.

Joblib serializes and deserializes the trained model objects. After fitting in [Code/src/modeling_pipeline.py](#), each model is saved as a `.joblib` file in [Code/saved_models/](#). The dashboard then loads these files at runtime with `joblib.load`, avoiding retraining on every startup.

All packages are installed from [Code/requirements.txt](#) via `pip install -r Code/requirements.txt`. The next three chapters focus on the three packages that are most directly responsible for the project's analytical core: **statsmodels** for model estimation, **scikit-learn** for evaluation, and **Streamlit** for deployment. Each of these chapters follows the same structure—introduction, description, installation, example description, example manual, example code, example files, and further reading—so that the reader can understand not only what the library does in general, but exactly how it contributes to forecasting Germany's monthly CPI.

5 Statsmodels

5.1 Introduction

Statsmodels is the main statistical modeling library used in this project. While **scikit-learn** is used for evaluation metrics and **Streamlit** is used for the dashboard, **statsmodels** is the package that actually estimates the forecasting models discussed in this report. In practical terms, it is the library behind the ARIMA, ETS, and SARIMA implementations used for Germany’s monthly CPI series.

This makes the package academically important for the report. The quality of the forecasting comparison depends not only on the selected models, but also on whether those models are implemented in a statistically appropriate and reproducible way. **Statsmodels** is a good fit for this purpose because it provides mature time-series classes, support for prediction intervals, and a workflow that integrates naturally with **pandas** time indices [Box+15; HA21; sta26].

5.2 Description

The most relevant **statsmodels** components for this repository are the time-series classes imported in [Code/src/models.py](#):

- `statsmodels.tsa.arima.model.ARIMA`
- `statsmodels.tsa.exponential_smoothing.ets.ETSModel`
- `statsmodels.tsa.statespace.sarimax.SARIMAX`

These classes are wrapped inside project-specific Python classes so that all forecasting models share a consistent interface with `fit()`, `predict()`, and `get_confidence_intervals()` methods. This wrapper design matters because it allows the modeling pipeline in [Code/src/modeling_pipeline.py](#) to train, forecast, evaluate, and serialize different models with the same control logic rather than handling each one separately.

Statsmodels is also used indirectly during exploratory analysis. The EDA pipeline relies on functions such as the Augmented Dickey-Fuller test and seasonal decomposition tools, both of which help justify later model choices in a methodologically transparent way.

Table ?? summarizes the most important **statsmodels** classes used in the repository.

Table 5.1: Mapping of statsmodels classes to project use.

Statsmodels class	Project use
ARIMA	Non-seasonal baseline forecasting model for autocorrelation-driven CPI prediction
ETSModel	State-space exponential-smoothing model for level, trend, and seasonal structure
SARIMAX	Seasonal forecasting model used to implement SARIMA with monthly seasonality

5.3 Installation

Statsmodels is listed in [Code/requirements.txt](#) and is normally installed together with the full project environment:

```
1000 pip install -r Code/requirements.txt
```

Listing 5.1: Installing project dependencies including statsmodels

If a separate installation is needed, the direct command is:

```
1000 pip install statsmodels
```

Listing 5.2: Installing statsmodels directly

The installation can be verified as follows:

```
1000 import statsmodels
      print(statsmodels.__version__)
```

Listing 5.3: Verifying the statsmodels installation

5.4 Example – Description

A repository-specific example is the definition of the three forecasting wrappers in [Code/src/models.py](#). The project does not call raw **statsmodels** classes directly from the dashboard or the evaluation loop. Instead, it places them behind a common interface so that ARIMA, ETS, and SARIMA can be used interchangeably during training and prediction.

This is a useful engineering decision. It keeps the statistical strengths of statsmodels while reducing repetition in the application code. It also makes the evaluation stage easier to document and easier to trust, because the forecasting pipeline can loop over models without introducing separate control branches for each class.

5.5 Example – Manual

The practical use of statsmodels in this project can be reproduced through the following steps:

1. Install the project dependencies.
2. Load the cleaned CPI series using [Code/src/data_loader.py](#).
3. Instantiate the model wrappers defined in [Code/src/models.py](#).
4. Fit the models on the training portion of the CPI series.
5. Generate 24-step forecasts and, where available, prediction intervals.
6. Compare results in [Code/src/modeling_pipeline.py](#) and reuse the trained objects in [Code/app.py](#).

5.6 Example – Code

Listing ?? shows the core form of statsmodels usage employed by the project.

```
1000 from statsmodels.tsa.arima.model import ARIMA
      from statsmodels.tsa.exponential_smoothing.ets import ETSTModel
1002 from statsmodels.tsa.statespace.sarimax import SARIMAX
      model_arima = ARIMA(train_series, order=(1, 1, 1))
1004
```

```

1006 model_ets = ETSModel(train_series , error='add' , trend='add' , seasonal='add' ,
    seasonal_periods=12)
    model_sarima = SARIMAX(train_series , order=(1, 1, 1) , seasonal_order=(1, 1, 1, 12))

```

Listing 5.4: Project-specific statsmodels model definitions

In the repository, these calls are embedded in wrapper classes in [Code/src/models.py](#) so that the forecasting pipeline can handle them uniformly. This gives the project a clearer separation between statistical modeling and application logic.

5.7 Example – Files

The most important files in which statsmodels appears are:

- [Code/src/models.py](#) – imports and wraps ARIMA, ETSModel, and SARIMAX.
- [Code/src/modeling_pipeline.py](#) – fits wrapped models, generates forecasts, and saves artifacts.
- [Code/src/eda_pipeline.py](#) – uses statsmodels-based diagnostics such as ADF testing and seasonal decomposition.
- [Code/app.py](#) – consumes trained statsmodels-based wrappers after serialization.

5.8 Package Role in the CPI Pipeline

The following figure is not intended to replace the global KDD methodology in Chapter ???. Instead, it zooms in on the **data-mining stage** and shows the specific role of **statsmodels** inside the implemented forecasting pipeline. In this project, the package receives the cleaned CPI series after selection, preprocessing, and transformation have already been completed. It then provides the statistical model classes for ARIMA, ETS, and SARIMA, which are wrapped in the repository code, fitted to the series, and used to generate forecasts together with confidence intervals. The figure therefore gives a smaller package-level view nested inside the broader KDD process.

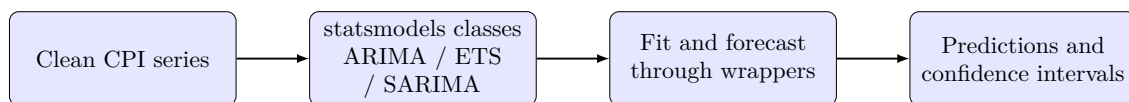


Figure 5.1: Package-level view of **statsmodels** within the CPI pipeline. In KDD terms, the figure corresponds primarily to the data-mining stage, where the cleaned and transformed CPI series is passed to the forecasting models, fitted through repository wrappers, and converted into predictions with confidence intervals.

5.9 Further Reading

- Statsmodels documentation: <https://www.statsmodels.org/>
- Time-series user guide: <https://www.statsmodels.org/stable/tsa.html>
- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice* [HA21]
- Box, Jenkins, Reinsel, and Ljung, *Time Series Analysis: Forecasting and Control* [Box+15]

6 Scikit-learn

6.1 Introduction

Scikit-learn is an open-source Python library for machine learning and model evaluation. In this CPI forecasting project, it is not used for model training itself, because the forecasting models are estimated with **statsmodels**. Its importance instead lies in the **evaluation layer** of the workflow. The library provides the metric functions that make the comparison between ARIMA, ETS, and SARIMA consistent, transparent, and reproducible [sci26].

This illustrates a selective but academically relevant use of a software package. Although **scikit-learn** is not the forecasting engine of the system, it provides the numerical basis for judging forecast quality on the chronological holdout period.

6.2 Description

The project uses **scikit-learn** in a focused way through the module **sklearn.metrics**. In [Code/src/modeling_pipeline.py](#), the functions **mean_squared_error**, **mean_absolute_error**, and **mean_absolute_percentage_error** are applied to compare predicted CPI values against the actual CPI values in the 24-month test set. The same metric logic is also used again in [Code/app.py](#) so that the dashboard displays evaluation values consistent with the offline modeling pipeline.

This role is directly aligned with the project objective. Since the task is to compare three forecasting models under one common setup, the evaluation package must deliver standardized and interpretable metrics. **Scikit-learn** fulfills this requirement well because its metric functions are widely used, stable, and easy to integrate with **pandas**- and **numpy**-based workflows.

Table ?? summarizes the role of the three evaluation metrics used in this project.

Table 6.1: Interpretation of scikit-learn metrics in the CPI forecasting project.

Metric	What it measures	Why it matters in this project
RMSE	Square-rooted average of squared forecast errors	Penalizes larger CPI forecast misses more strongly and is therefore useful when substantial deviations are especially undesirable
MAE	Average absolute forecast deviation	Provides a direct interpretation in CPI index points and is easy to compare across models
MAPE	Average percentage forecast deviation	Supports scale-relative comparison and improves interpretability for non-technical readers

6.3 Installation

The project records **scikit-learn** as a dependency in [Code/requirements.txt](#). Installation is therefore handled together with the other Python packages used in the repository:

```
1000 | pip install -r Code/requirements.txt
```

Listing 6.1: Installing project dependencies including scikit-learn

If scikit-learn must be installed separately, the package manager command is:

```
1000 pip install scikit-learn
```

Listing 6.2: Installing scikit-learn directly

The installation can be verified with a short version check:

```
1000 import sklearn
      print(sklearn.__version__)
```

Listing 6.3: Verifying the scikit-learn installation

6.4 Example – Description

A concrete repository example is the evaluation of the three forecasting models on the holdout period. After ARIMA, ETS, and SARIMA generate their 24-step forecasts, scikit-learn metrics are used to measure how far these predictions deviate from the actual CPI values. The resulting metrics answer complementary questions:

- **RMSE** emphasizes larger forecast errors more strongly.
- **MAE** gives the average absolute forecast error in CPI index points.
- **MAPE** expresses the error in percentage terms and supports intuitive comparison.

For the current repository version, this evaluation shows that the ETS model performs best, with RMSE = 0.7697, MAE = 0.6453, and MAPE = 0.5298%. These values are taken from the exported evaluation results in [Code/plots/evaluation_metrics.csv](#).

6.5 Example – Manual

The scikit-learn-based evaluation can be reproduced with the following project-specific steps:

1. Install the dependencies from [Code/requirements.txt](#).
2. Ensure that the dataset [Code/data/61111-0002_en.csv](#) is present.
3. Run [Code/src/modeling_pipeline.py](#) from the [Code](#) directory.
4. Let the pipeline train ARIMA, ETS, and SARIMA on the training data.
5. Inspect the printed metric output and the exported file [Code/plots/evaluation_metrics.csv](#).
6. Optionally start [Code/app.py](#) with **Streamlit** and compare the same metrics in the dashboard.

6.6 Example – Code

Listing ?? shows the actual type of scikit-learn usage employed in the project.

```

1000 from sklearn.metrics import (
1001     mean_squared_error,
1002     mean_absolute_error,
1003     mean_absolute_percentage_error
1004 )
1005 import numpy as np
1006
1007 rmse = np.sqrt(mean_squared_error(test['CPI'], preds))
1008 mae = mean_absolute_error(test['CPI'], preds)
1009 mape = mean_absolute_percentage_error(test['CPI'], preds) * 100

```

Listing 6.4: Forecast evaluation with scikit-learn metrics

This code is repository-relevant because it matches the implementation in [Code/src/modeling_pipeline.py](#) and is reused conceptually in [Code/app.py](#) to ensure consistency between offline evaluation and dashboard display.

6.7 Example – Files

The main files in which scikit-learn is used are:

- [Code/src/modeling_pipeline.py](#) – computes RMSE, MAE, and MAPE during offline evaluation.
- [Code/app.py](#) – recomputes the same metrics for display in the Streamlit dashboard.
- [Code/plots/evaluation_metrics.csv](#) – stores the metric results generated from these computations.

6.8 Package Role in the CPI Pipeline

The following figure is not a second full methodology diagram, but a focused view of where **scikit-learn** enters the repository workflow. Relative to Chapter ??, it belongs mainly to the **interpretation and evaluation stage**. After the forecasting models have produced predictions for the test horizon, these predicted values are compared with the corresponding **actual CPI holdout values**. **Scikit-learn** then converts this comparison into RMSE, MAE, and MAPE values that can be evaluated consistently across ARIMA, ETS, and SARIMA. The figure is therefore meant to clarify the package’s analytical role rather than to compete with the broader KDD structure.

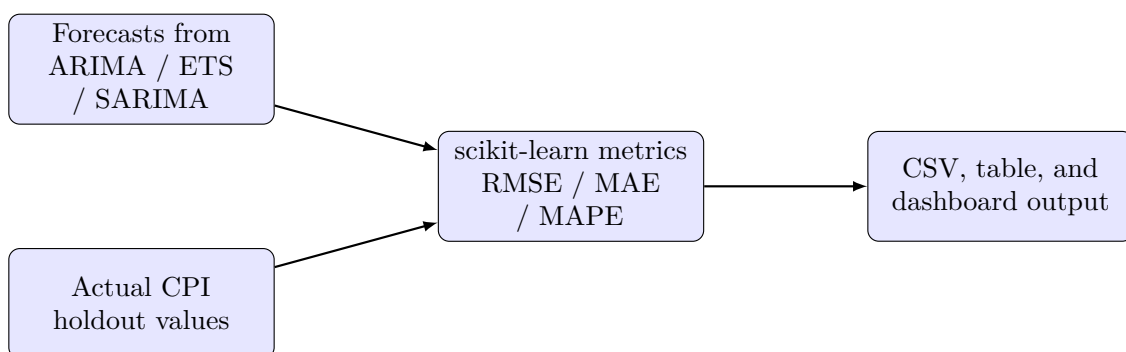


Figure 6.1: Package-level view of **scikit-learn** within the CPI pipeline. In KDD terms, the figure corresponds mainly to the interpretation and evaluation stage, where model forecasts and actual holdout values are converted into comparable performance metrics.

6.9 Further Reading

- Scikit-learn documentation: <https://scikit-learn.org/stable/index.html>
- Metrics and scoring guide: https://scikit-learn.org/stable/modules/model_evaluation.html
- Pedregosa et al., *Scikit-learn: Machine Learning in Python*

7 Streamlit

7.1 Introduction

Streamlit is the deployment library used to transform the CPI forecasting workflow into an interactive analytical application. In this project, the package is not responsible for training the models; instead, it acts as the presentation and interaction layer above already-trained ARIMA, ETS, and SARIMA models [Str26]. This separation is important because it keeps model estimation in the offline pipeline and reserves the app for reproducible visualization and interpretation.

7.2 Description

The complete **Streamlit** implementation is located in [Code/app.py](#). The application loads the cleaned CPI data, reads serialized model objects from [Code/saved_models/](#), computes forecasts for the 24-month test horizon, and displays both plots and error metrics. Users can select a single forecasting model or switch to a comparison mode in which all three models are shown simultaneously.

Several **Streamlit** features are used directly in the app:

- `st.sidebar.selectbox()` for model and confidence-interval selection,
- `st.metric()` for RMSE, MAE, and MAPE presentation,
- `st.plotly_chart()` for interactive forecast visualizations,
- `st.expander()` for explanatory text,
- `st.warning()` and `st.error()` for user-facing failure messages,
- `@st.cache_data` for efficient data loading.

This package is therefore relevant not only for deployment convenience, but also for communicating model behavior to non-technical users.

Table ?? summarizes the main user-interface elements implemented in the dashboard.

Table 7.1: Main Streamlit interface elements in the CPI dashboard.

UI element	Purpose
Sidebar model selector	Chooses the forecasting view shown to the user
Confidence-interval selector	Adjusts the displayed uncertainty level for forecast intervals
Metric display area	Presents RMSE, MAE, and MAPE in compact form
Plotly chart area	Visualizes actual and forecast CPI values interactively
Expanders and warning messages	Support interpretation and communicate runtime issues

7.3 Installation

Streamlit is included in the project dependency file:

```
1000 pip install -r Code/requirements.txt
```

Listing 7.1: Installing project dependencies including Streamlit

If installed separately, the command is:

```
1000 pip install streamlit
```

Listing 7.2: Installing Streamlit directly

The installation can be checked as follows:

```
1000 import streamlit as st
      print(st.__version__)
```

Listing 7.3: Verifying the Streamlit installation

7.4 Example – Description

A concrete project example is the model-comparison view in [Code/app.py](#). In this mode, **Streamlit** loads all saved models, recomputes RMSE, MAE, and MAPE on the common holdout period, highlights the best metric values in a table, and renders an interactive Plotly chart with actual and predicted CPI values. This turns the static evaluation from the modeling pipeline into an exploratory interface that can be used during demonstration and interpretation.

7.5 Example – Manual

The dashboard can be started with the following practical sequence:

1. Install all Python dependencies from [Code/requirements.txt](#).
2. Run [Code/src/modeling_pipeline.py](#) first so that the model files exist in [Code/saved_models/](#).
3. Start the app with `streamlit run Code/app.py` from the project root.
4. Open the local **Streamlit** URL shown in the terminal.
5. Select a forecasting model or choose **Compare All Models**.
6. Inspect forecast curves, confidence intervals, and evaluation metrics.

7.6 Example – Code

```
1000 st.sidebar.header("Configuration")
      model_choice = st.sidebar.selectbox(
1002         "Select Model",
            ["ARIMA (1,1,1)", "ETS (A,A,A)", "SARIMA (1,1,1)x(1,1,1,12)", "Compare All Models"]
1004     )
      ci_level = st.sidebar.selectbox("Confidence Interval Level", [80, 90, 95, 96, 97, 98,
1006         99], index=2)
```

Listing 7.4: Typical Streamlit controls used in the CPI dashboard

This excerpt illustrates how Streamlit exposes forecasting settings as user-selectable controls without requiring code modification by the end user.

7.7 Example – Files

The main files and artifacts connected to Streamlit are:

- `Code/app.py` – complete Streamlit application.
- `Code/saved_models/*.joblib` – pretrained models loaded at runtime.
- `Code/data/61111-0002_en.csv` – CPI data source used in the app.

7.8 Package Role in the CPI Pipeline

The following figure should be read as a package-level deployment view rather than as a replacement for the KDD diagram in Chapter ???. Whereas the KDD chapter explains the overall analytical methodology from data selection to evaluation, this figure isolates the role of **Streamlit** after the modeling stage has already produced reusable artifacts. In practice, the package belongs closest to the report’s deployment and presentation layers: it loads the CPI data together with the saved models, exposes user controls for model and confidence-interval selection, triggers forecast and metric generation for the chosen view, and renders the dashboard that makes the forecasting results accessible for interpretation. The left-to-right structure of the figure is therefore meant to reflect the actual interaction sequence experienced by the user in the application.

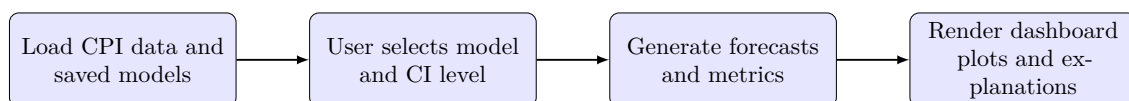


Figure 7.1: Package-level view of **Streamlit** within the CPI pipeline. The figure complements the KDD methodology by showing the downstream deployment and presentation layer in which previously trained artifacts are loaded, configured by the user, used to generate forecasts and metrics, and displayed as an interactive dashboard.

7.9 Further Reading

- Streamlit documentation: <https://docs.streamlit.io/>
- Streamlit API reference: <https://docs.streamlit.io/library/api-reference>

8 Data Mining

In this project, the data-mining stage refers to the estimation of forecasting models on the prepared CPI time series. Because the selected task is a univariate monthly forecasting problem, the goal of data mining is not classification or causal explanation, but the extraction of predictive structure from temporal dependence, trend, and seasonality [FPSS96; HA21].

The comparison focuses on three classical model families: **ARIMA**, **ETS**, and **SARIMA**. These three models were selected because they represent the main statistical alternatives that are plausible for a monthly CPI series without introducing a completely different problem formulation. ARIMA serves as the non-seasonal baseline and tests whether the CPI can be forecast sufficiently through autocorrelation and differencing alone. SARIMA extends the same autoregressive logic by incorporating explicit yearly seasonality, which is a natural hypothesis for monthly macroeconomic data. ETS approaches the problem from a different modeling perspective entirely by treating level, trend, and seasonality as evolving state components rather than as autoregressive and moving-average terms [Box+15; HA21].

This difference in perspective is the main reason why the comparison is academically stronger than presenting a single model in isolation. If only ARIMA were reported, the reader could not judge whether the observed forecast quality results from a generally appropriate treatment of CPI dynamics or merely from the choice of one familiar baseline. If only ETS were presented, the report would not show whether explicit seasonal autoregressive structure offers any advantage over smoothing-based decomposition. By comparing all three models under the same input data, the same forecast horizon, and the same error metrics, the report can relate performance differences directly to model assumptions instead of to differences in data preparation or evaluation design.

The three methods also differ in the practical questions they ask of the CPI series. ARIMA asks whether past values and past forecast errors, after differencing, contain enough structure for accurate forecasts. SARIMA asks whether this same structure improves once a repeating 12-month pattern is modeled explicitly. ETS asks whether the CPI behaves more like a gradually evolving combination of level, trend, and seasonality that can be smoothed forward without strong stationarity assumptions. These are not redundant alternatives; they correspond to different theoretical readings of the same economic time series and therefore justify a structured comparison chapter before the model-specific sections begin.

The implemented data-mining workflow is consistent across all models. The cleaned CPI series is split chronologically, the last 24 observations are reserved for testing, and each model is fitted on the earlier observations only. After fitting, forecasts are generated for the holdout period and compared with the observed CPI values using RMSE, MAE, and MAPE. This design supports fair comparison and avoids leakage from future information.

From a methodological perspective, this common workflow is essential. The project does not compare models that were tuned on different horizons, trained on different samples, or judged with different metrics. Instead, the same monthly CPI target, the same train–test boundary, and the same forecast-evaluation logic are reused throughout [Code/src/modeling_pipeline.py](#). This gives the later algorithm chapters a shared frame of reference: when ARIMA, SARIMA, and ETS are discussed individually, they are not separate mini-projects, but three competing forecasting strategies embedded in one coherent pipeline.

The algorithm-specific chapters that follow provide the detailed description, requirements, hyperparameters, input, output, and program examples for ARIMA, SARIMA, and ETS. Together, these chapters document the forecasting methods actually used in the repository and explain why they are relevant for Germany's CPI series.

9 Overview of Forecasting Algorithms

Before discussing ARIMA, SARIMA, and ETS in separate chapters, it is useful to introduce the common purpose of this algorithm block within the report. All three methods are used to forecast Germany’s monthly CPI as a univariate time series, but they do so from different modeling perspectives [Box+15; HA21]. This chapter therefore serves as a short conceptual bridge between the general machine-learning background and the detailed algorithm descriptions that follow.

The choice of these three algorithms is consistent with the structure of the implemented repository. The project does not compare unrelated machine-learning paradigms, but focuses on established time-series forecasting approaches that can be trained on a single chronological CPI series and evaluated under the same holdout setting [HA21; Hu25]. In practical terms, all three models are fitted through the common wrapper structure in [Code/src/models.py](#) and are evaluated through the same pipeline in [Code/src/modeling_pipeline.py](#). Their comparison is therefore methodologically fair at the level of data source, forecast horizon, and evaluation metrics.

Although the algorithms share the same target variable and evaluation setup, they differ in the type of temporal structure they emphasize. ARIMA focuses on autocorrelation and differencing-based dynamics in a non-seasonal setting. SARIMA extends that idea by adding explicit seasonal terms, which is especially relevant when monthly CPI exhibits recurring annual patterns. ETS approaches the same forecasting problem from an exponential-smoothing and state-space perspective, modeling level, trend, and seasonality directly rather than through autoregressive and moving-average components [Box+15; HA21].

Table ?? summarizes the role of the three forecasting algorithms in this project.

Table 9.1: Introductory comparison of the forecasting algorithms used in the CPI project.

Algorithm	Main modeling emphasis	Relevance for this project
ARIMA	Autoregressive and moving-average structure with differencing for trend handling	Provides a classical non-seasonal baseline for forecasting CPI from its historical temporal dependence
SARIMA	ARIMA structure extended by seasonal autoregressive, differencing, and moving-average terms	Captures possible yearly seasonality in monthly CPI observations and therefore tests whether explicit seasonal modeling improves forecast quality
ETS	Exponential smoothing with explicit level, trend, and seasonal components in a state-space framework	Offers an alternative forecasting perspective that is often effective when macroeconomic series are dominated by smooth structural patterns rather than complex autocorrelation alone

This introductory chapter is intentionally concise. Its purpose is not to repeat the detailed theory that belongs in the following algorithm-specific chapters, but to explain why these three methods appear together and why their comparison is meaningful for the CPI forecasting task. The next chapters therefore examine ARIMA, SARIMA, and ETS individually with respect to their structure, assumptions, hyperparameters, and concrete use in the repository.

10 AutoRegressive Integrated Moving Average (ARIMA)

10.1 Description

The AutoRegressive Integrated Moving Average (ARIMA) model is one of the standard statistical tools for analyzing and forecasting univariate time series [Box+15; HA21]. Its main idea is simple: future values are explained using past values, past forecast errors, and a differencing step that helps remove non-stationary trend behavior. In this report, ARIMA serves as the main non-seasonal baseline against which the more explicitly seasonal models are compared. Figure ?? illustrates the general modeling logic. Let y_t be the original series and y'_t be the differenced series. A simplified ARIMA representation can be written as:

$$y'_t = c + \phi_1 y'_{t-1} + \dots + \phi_p y'_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t \quad (10.1)$$

The variables and coefficients in this formulation can be interpreted as follows:

- y_t **and** y'_t : the original observation at time t and the differenced observation at time t .
- c : a constant drift or intercept term.
- ϵ_t : the random error at time t , usually interpreted as white noise.
- **AutoRegressive (AR - p)**: captures how strongly the current value depends on a selected number of past values.
- **Integrated (I - d)**: applies differencing d times to reduce trend and move the series closer to stationarity.
- **Moving Average (MA - q)**: captures how current values depend on past forecast errors.

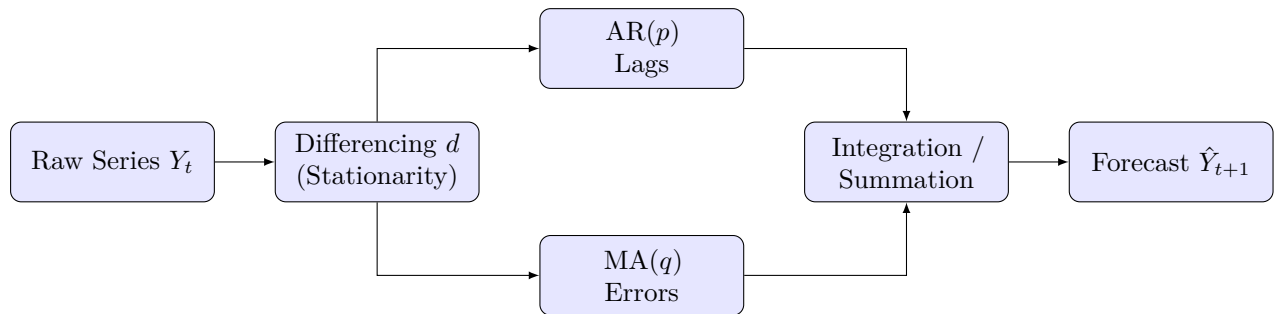


Figure 10.1: Architectural flowchart of an ARIMA(p, d, q) model showing how the differenced signal is split into AutoRegressive and Moving Average paths before recombination. Source: own diagram based on the Box–Jenkins ARIMA formulation [Box+15; Ham94].

10.2 Applications

ARIMA is widely used in settings where the main information comes from the history of a single variable. Typical examples include financial series, macroeconomic indicators, demand forecasting, epidemiological case counts, and short-term operational data. In economics especially, ARIMA is often used as a benchmark model because it is interpretable, well studied, and comparatively easy to estimate [Box+15; Ham94].

10.3 Relevance

Within this project, ARIMA is important because it gives a clear baseline for forecasting Germany's Consumer Price Index. CPI data typically shows persistence, meaning that recent values still contain information about the next few months. ARIMA is well suited to capture that kind of autocorrelation. At the same time, it does not model seasonality as directly as SARIMA or ETS, which makes it a useful baseline: if a seasonal model performs better, that improvement can be interpreted more clearly.

10.4 Hyperparameters

The ARIMA model is defined by three primary hyperparameters, usually written as (p, d, q) :

- **p (lag order)**: the number of past observations included in the autoregressive part. This is often informed by the Partial Autocorrelation Function (PACF), as shown in Figure ??.
- **d (degree of differencing)**: the number of times the raw series is differenced to move it closer to stationarity. This can be supported by tests such as the Augmented Dickey-Fuller (ADF) test.
- **q (moving average order)**: the number of lagged forecast errors included in the moving-average part. This is often informed by the Autocorrelation Function (ACF).

10.5 Requirements

The most important practical requirement of ARIMA is that the input series should either be stationary or capable of being made approximately stationary through differencing [Box+15; HA21]. In simple terms, this means that the underlying statistical behavior of the series should not drift too strongly over time once trend effects are removed. Figure ?? illustrates this idea by contrasting the upward-trending CPI series with its first-differenced version.

If the variance changes strongly over time, an additional transformation such as a logarithm or Box-Cox transform may also be useful before fitting the model. Standard ARIMA is not designed to model repeating seasonal structure directly, so series with clear seasonality are often better handled by SARIMA or by explicit deseasonalization.

10.6 Input

The model expects a univariate, equally spaced time series without missing values. In code, this is usually represented as a `pandas.Series` or `numpy.ndarray`, ordered chronologically and ideally indexed by a `DatetimeIndex` with a clear frequency such as monthly observations.

10.7 Output

The model outputs point forecasts for a specified number of future time steps. In addition, it provides estimated forecast uncertainty, which can be used to construct prediction intervals such as 95% confidence

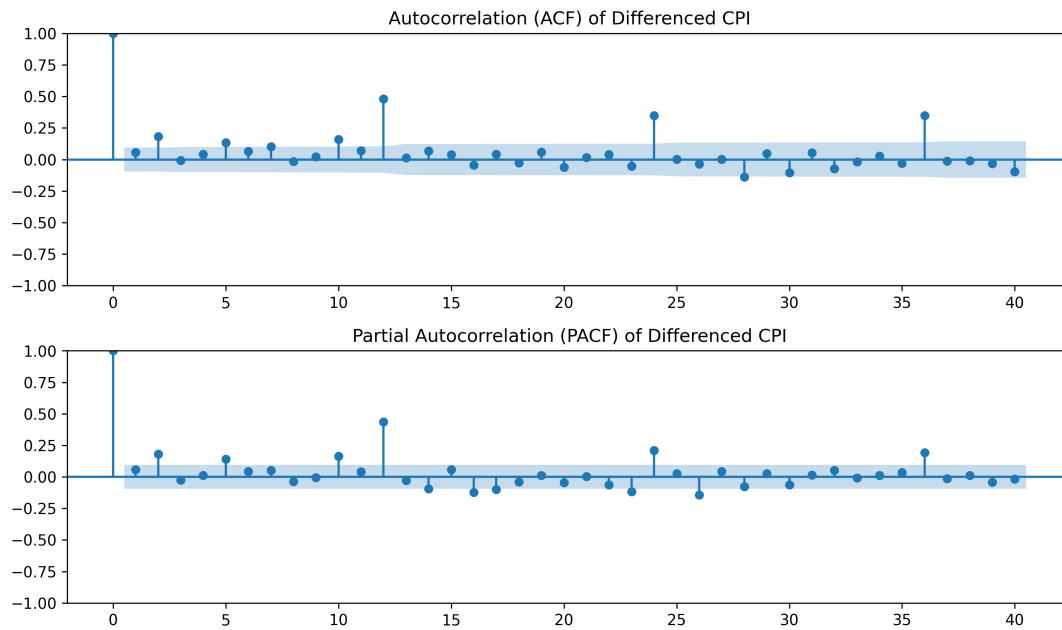


Figure 10.2: Autocorrelation (ACF) and Partial Autocorrelation (PACF) plots used to empirically determine the Moving Average (q) and AutoRegressive (p) hyperparameters respectively. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

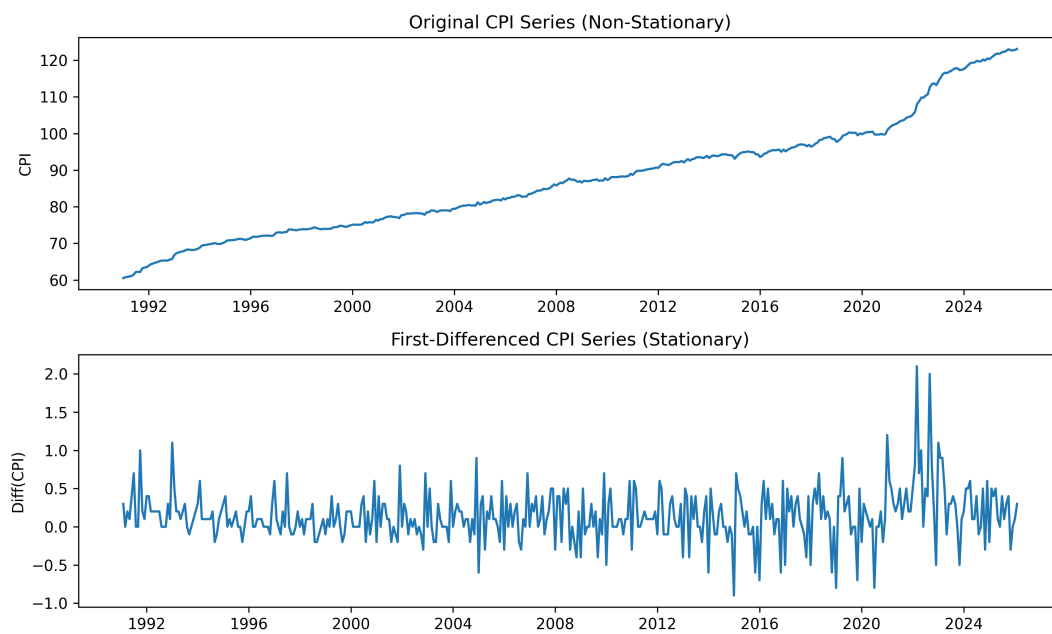


Figure 10.3: Original CPI Series (top) exhibiting a non-stationary upward trend, compared to the First-Differenced CPI Series (bottom) which achieves the stationarity required by ARIMA. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

bounds. This is helpful because a forecasting model should communicate not only its best estimate, but also how uncertain that estimate becomes as the forecast horizon grows.

10.8 Example with a program

The following Python snippet demonstrates the basic workflow for fitting and forecasting with an ARIMA model using `statsmodels`, assuming that stationarity checks have already been carried out:

```
1000 import pandas as pd
1001 from statsmodels.tsa.arima.model import ARIMA
1002
1003 # Assume 'cpi_series' is a pandas Series with a monthly DatetimeIndex,
1004 # and we have determined (p=1, d=1, q=1) via ACF/PACF and ADF tests.
1005 cpi_series = pd.Series([105.1, 105.4, 105.8, 106.2, 106.5])
1006
1007 # Initialize the ARIMA model
1008 model = ARIMA(cpi_series, order=(1, 1, 1))
1009
1010 # Fit the model using Maximum Likelihood Estimation (MLE)
1011 fitted_model = model.fit()
1012
1013 # Forecast the next 12 periods (1 year)
1014 forecast_result = fitted_model.get_forecast(steps=12)
1015 predicted_means = forecast_result.predicted_mean
1016 confidence_intervals = forecast_result.conf_int(alpha=0.05) # 95% CI
1017
1018 print("Forecasts:\n", predicted_means)
```

Listing 10.1: ARIMA Implementation using Statsmodels

10.9 Further Reading

- Box, Jenkins, Reinsel, and Ljung, *Time Series Analysis: Forecasting and Control* [Box+15]
- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice* [HA21]

11 Seasonal AutoRegressive Integrated Moving Average (SARIMA)

11.1 Description

The Seasonal AutoRegressive Integrated Moving Average (SARIMA) model is a powerful extension to the classical ARIMA framework explicitly designed to capture and forecast time series data exhibiting periodic cycles. Standard ARIMA models rely on linear relationships with contiguous past observations to estimate trends, rendering them structurally blind to effects that repeat systematically over longer intervals (e.g., an observation y_t being strongly correlated to the observation from a year ago y_{t-12} , rather than just yesterday y_{t-1}). SARIMA resolves this limitation by introducing the backshift operator B at seasonal intervals. It formally constructs a composite model by mathematically multiplying a regular ARIMA polynomial by a seasonal ARIMA polynomial, adding dedicated parameters for seasonal autoregressive, seasonal moving average, and seasonal differencing terms. This multiplication of polynomials is mapped out in the flowchart in Figure ??.

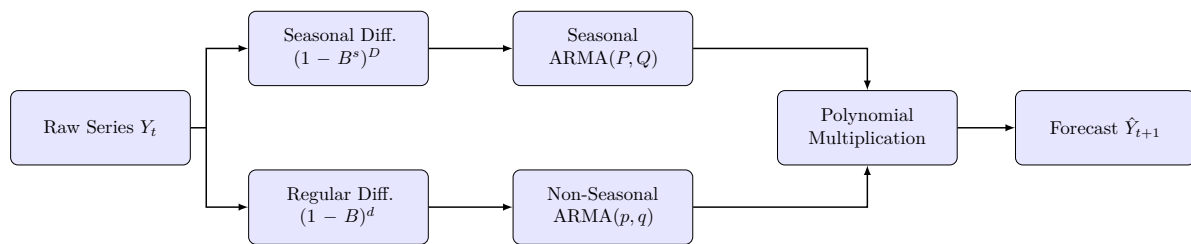


Figure 11.1: Architectural flowchart of a SARIMA model illustrating how the algorithm applies the backshift operator B to construct and multiply non-seasonal and seasonal mathematical polynomials. Source: own diagram based on seasonal Box–Jenkins modeling [Box+15; HA21].

11.2 Applications

SARIMA is the industry standard approach for modeling macroscopic and microscopic phenomena with pronounced, rigid seasonality. Application domains include forecasting airline passenger traffic, hotel bookings, energy consumption (where grid demand peaks annually in summer or winter), and agricultural retail sales. In macroeconomic planning, central banks rely on SARIMA to forecast series affected by annual tax cycles, quarterly corporate reporting, or periodic policy implementations.

11.3 Relevance

In this project, the underlying subject of study is the Consumer Price Index (CPI) of Germany. The CPI measures a "basket of goods" that inherently displays strong seasonal patterns due to predictable annual shifts in consumer behavior and market availability. Examples include winter heating oil demand spiking in November, summer vacation travel costs peaking in July, and bi-annual seasonal apparel releases. SARIMA is critical here; by mathematically isolating and explicitly modeling these cyclical variations via the seasonal parameter s , the algorithm mitigates seasonal "noise" that would otherwise corrupt a

standard ARIMA model. This allows policymakers to more accurately predict "core" inflationary trends stripped of seasonal artifacts.

11.4 Hyperparameters

A SARIMA model is specified by a dual set of parameters, denoted as $(p, d, q) \times (P, D, Q)_s$:

- **Non-Seasonal Parameters (p, d, q) :** Define the standard AutoRegressive, Differencing, and Moving Average orders governing short-term, month-to-month dynamics.
- **Seasonal Parameters (P, D, Q) :** Define the AutoRegressive, Differencing, and Moving Average orders applied specifically at the seasonal lags. For instance, $P = 1$ means today's value is influenced by the value exactly one cycle ago.
- **Seasonal Period (s) :** The length of the repeating seasonal cycle. For instance, in monthly data exhibiting annual seasonality, $s = 12$; in quarterly data, $s = 4$. Setting this value is crucial: if s is too low (e.g., $s = 1$), the model devolves into standard ARIMA and misses the macro-cycle. If s is too high (e.g., $s = 365$ for daily data), the polynomial complexity expands massively, requiring enormous amounts of historical data and computational power, often leading to over-fitting or failure to converge.

11.5 Requirements

SARIMA strictly requires that the underlying data-generating process is linear and that the seasonal period s is consistent and known a priori. A fundamental requirement is sufficient historical data length: to properly estimate seasonal parameters and perform seasonal differencing, the dataset must contain at least two to three full seasonal cycles (e.g., 24-36 months of data for $s = 12$). Similar to ARIMA, the series must achieve stationarity before parameter estimation. SARIMA achieves this via primary differencing ($y_t - y_{t-1}$) to eliminate trend, combined with seasonal differencing ($y_t - y_{t-s}$) to eliminate persistent cyclical patterns. Furthermore, the variance of the seasonal cycle must remain relatively constant; if the amplitude of the seasons grows over time, a log transformation is required beforehand.

11.6 Input

The algorithm requires a univariate, continuously indexed sequence of observations. In Python, this is instantiated as a `pandas.Series` possessing a strict chronological index. Accurate metadata regarding the frequency of the observations must be explicitly provided, as the model relies on this to calculate the s parameter mathematically.

11.7 Output

SARIMA outputs an extrapolated series of predicted values extending into the future. Due to its structural mathematical properties, it uniquely recalculates both the short-term trend and the periodic peaks/troughs simultaneously for every future step. It also returns standard errors and likelihood metrics (such as AIC/BIC), which are utilized to generate probabilistic confidence intervals quantifying the bounds of the forecast.

11.8 Example with a program

The `statsmodels` library in Python implements SARIMA via the robust `SARIMAX` class. (The 'X' denotes support for exogenous variables, which can be ignored by passing solely the endogenous series):

```

1000 import pandas as pd
1001 from statsmodels.tsa.statespace.sarimax import SARIMAX
1002
1003 # 'cpi_series' is a pandas Series with a monthly DatetimeIndex
1004 # In this hypothetical, we have 3 years of data (36 observations)
1005 cpi_series = pd.Series([105.1, 105.4, 105.8, 106.2, 106.5])
1006
1007 # Initialize a SARIMA model:
1008 # Short-term (non-seasonal) dynamics: (1,1,1)
1009 # Annual seasonal dynamics: P=1, D=1, Q=1 with a 12-month cycle (s=12)
1010 model = SARIMAX(endog=cpi_series,
1011                 order=(1, 1, 1),
1012                 seasonal_order=(1, 1, 1, 12),
1013                 enforce_stationarity=False,
1014                 enforce_invertibility=False)
1015
1016 # Fit the model via Maximum Likelihood Estimation
1017 fitted_model = model.fit(dispatch=False)
1018
1019 # Forecast the next 12 periods and retrieve confidence intervals
1020 forecast_result = fitted_model.get_forecast(steps=12)
1021 predictions = forecast_result.predicted_mean
1022 conf_ints = forecast_result.conf_int(alpha=0.05) # 95% Confidence Bounds
1023
1024 print(predictions)

```

Listing 11.1: SARIMA Implementation using Statsmodels

11.9 Further Reading

- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). John Wiley & Sons. (The definitive text on seasonal Box-Jenkins models).
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts. Available at <https://otexts.com/fpp2/seasonal-arima.html>. [HA18]

12 Error, Trend, and Seasonality (ETS)

12.1 Description

The Error, Trend, and Seasonality (ETS) framework represents a highly generalized, robust class of exponential smoothing methods used extensively for point forecasting and statistical analysis of time series data. In contrast to ARIMA models—which rely on describing linear autocorrelations between past and future observations—the ETS framework adopts an intuitive state-space formulation. It decomposes the time series sequence into an underlying “level,” overlaid by a “trend” component and a “seasonal” component. Figure ?? visually demonstrates this fundamental decomposition of the actual CPI dataset. It applies exponentially decreasing weights to past observations, implying that recent history holds more predictive power than distant history. By characterizing the Error (E), Trend (T), and Seasonality (S) as mathematically discrete states that evolve over time, it establishes a probabilistic foundation capable of generating robust forecast intervals for almost any underlying data-generating process.

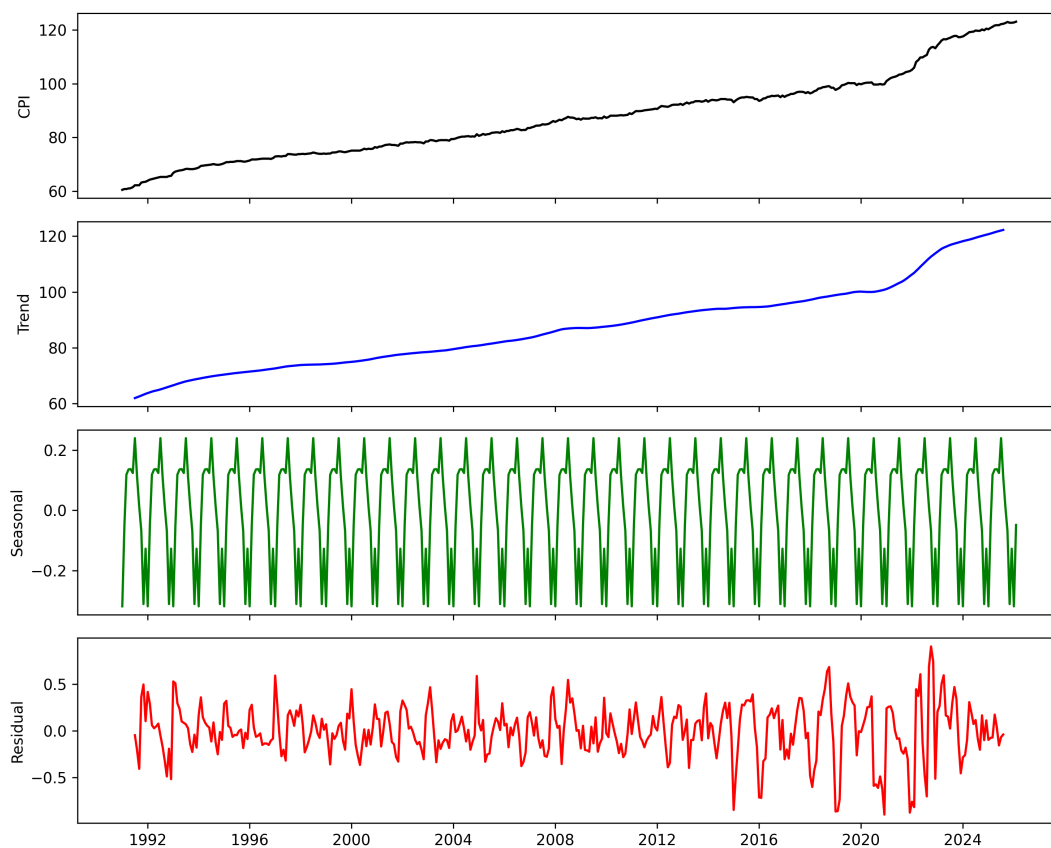


Figure 12.1: Time Series Decomposition of the CPI dataset, illustrating the fundamental components modeled by the ETS framework: the observed CPI, the underlying Trend, the cyclic Seasonality, and the stochastic Residual Error. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

The state transition mechanics are mapped in Figure ???. Instead of relying on raw lags, the previous observation y_{t-1} updates the distinct internal states governed by three crucial smoothing parameters: α dictates how heavily the new data shifts the baseline Level, β dictates how much the long-term Trend slope adjusts, and γ dictates the magnitude of the update to the periodic Seasonal cycle.

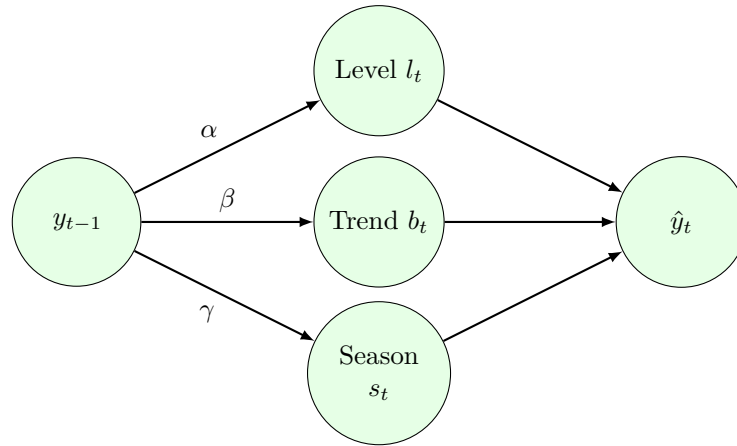


Figure 12.2: State transition diagram for ETS. Past observations update the distinct internal states governed by mathematical smoothing parameters (α, β, γ) , which are then recombined to produce the forecast $\hat{y}_t$. Source: own diagram based on exponential smoothing state-space formulations [HA21].

12.2 Applications

Exponential smoothing is arguably the most widely applied forecasting technique in industry today, particularly in automated forecasting environments, supply chain management, inventory control, and retail demand forecasting. The method gained prominent popularity because it requires minimal tuning compared to ARIMA models, naturally handles data with slowly evolving or non-stationary trends, and operates computationally faster on large arrays of parallel time series. The `forecast::ets` function developed by Rob Hyndman in R codified its dominance in the data science landscape.

12.3 Relevance

In this study modeling the Consumer Price Index (CPI) of Germany, the ETS framework proved to be the most accurate methodology, achieving a test Root Mean Squared Error (RMSE) of 0.7697, decisively outperforming both the standard ARIMA (1.3262) and the seasonal SARIMA (1.1909). This success can be directly attributed to the structural properties of macroeconomic inflationary data. While inflation trends do shift, they tend to evolve smoothly rather than abruptly. The exponential smoothing mechanism inherent to ETS adapts gracefully to these gradual, non-linear level and trend shifts without requiring the explicit differencing steps mandated by ARIMA (which can over-difference and destroy valid historical signal). Moreover, ETS multiplicative configurations can natively model proportional changes in variance over long time horizons, a common macroeconomic trait where higher index values inherently correspond to larger absolute fluctuations.

12.4 Hyperparameters

The ETS framework discards discrete lag orders (p, d, q) in favor of modeling the mathematical interactions between three core structural elements. Each element is categorized by its form:

- **Error (E):** Determines how the residual stochastic noise affects the state equations. It can be Additive (A) or Multiplicative (M).
- **Trend (T):** Models the long-term progression of the series. It can be None (N), Additive (A), Multiplicative (M), or Damped (Ad/Md), allowing the trend to flatten out over a specific horizon.
- **Seasonality (S):** Accounts for periodic fluctuations overlaying the trend. It can be None (N), Additive (A), or Multiplicative (M).
- **Smoothing Parameters (α, β, γ):** Internal coefficients bound between 0 and 1 determining how quickly the level, trend, and seasonal components update based on the newest observation. A value near 1 assigns massive weight to the most recent error (rapid adaptation), while a value near 0 relies heavily on historical inertia (smooth persistence). These are typically estimated automatically via maximum likelihood estimation (MLE).

12.5 Requirements

ETS is a highly versatile methodology. A primary advantage over ARIMA is that the data does not strictly need to be rendered stationary prior to modeling, as ETS inherently models and incorporates the non-stationary trend and seasonal components within its state equations. However, structural constraints do exist. For any model configuration containing a Multiplicative Error, Trend, or Seasonal term, the entire time series array must be strictly positive (i.e., $y_t > 0 \forall t$), as the framework divides states by previous states to calculate proportional impacts. Moreover, a robust seasonal frequency metadata parameter is required if the Seasonality component is active.

12.6 Input

The input consists of a univariate sequence, optimally structured as a `pandas.Series` with a continuous chronological index. For seasonal modeling variants, the frequency (e.g., $s = 12$ for monthly cycles) must be mathematically declared prior to model estimation.

12.7 Output

The primary output is a statistically generated forecast trajectory. Because modern ETS models the entire data-generating process via defined state-space equations (rather than heuristic smoothing), it also robustly generates prediction intervals based on the estimated variance of the selected Error type, providing both point forecasts and confidence bounds. It also yields the extracted internal states (Level, Trend, Seasonality), offering a high degree of explainability to stakeholders analyzing how the forecast was derived.

12.8 Example with a program

The following snippet demonstrates fitting a fully Additive ETS model using the `statsmodels` framework:

```

1000 import pandas as pd
1001 from statsmodels.tsa.exponential_smoothing.ets import ETSModel
1002
1003 # 'cpi_series' is a pandas Series indexed by a monthly DatetimeIndex
1004 # Time series must contain no missing values for proper state initialization.
1005 cpi_series = pd.Series([105.1, 105.4, 105.8, 106.2, 106.5])
1006
1007 # Initialize an ETS(A,A,A) model (Holt-Winters Additive):
1008 # Error=Additive, Trend=Additive, Seasonality=Additive
1009 model = ETSModel(endog=cpi_series,

```

```
1010         error="add",
1011         trend="add",
1012         seasonal="add",
1013         damped_trend=False,
1014         seasonal_periods=12)
1016 # Fit the model using maximum likelihood to optimize alpha, beta, gamma
fitted_model = model.fit(dispatch=False)
1018 # Forecast the next 12 periods and retrieve the prediction result object
1020 forecast_result = fitted_model.get_prediction(start=len(cpi_series), end=len(cpi_series)
+11)
predictions = forecast_result.predicted_mean
1022 conf_ints = forecast_result.pred_int(alpha=0.05) # 95% CI
1024 print(predictions)
```

Listing 12.1: ETS Implementation using Statsmodels

12.9 Further Reading

- Hyndman, R. J., Koehler, A. B., Ord, J. K., & Snyder, R. D. (2008). *Forecasting with Exponential Smoothing: The State Space Approach*. Springer. (The definitive textbook mapping simple smoothing to the rigorous ETS state-space framework). [HA18]
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts. Available at <https://otexts.com/fpp2/ets.html>.

Part IV

Domain Knowledge - Tools

13 Development Environment: Python

13.1 Python Version

The project is implemented in Python and uses a dependency set recorded in `Code/requirements.txt`. For reproducibility, the development environment should use a modern Python 3 installation that is compatible with `pandas`, `statsmodels`, `scikit-learn`, and `streamlit`. Version clarity is important because the project combines time-series modeling, serialization, and dashboard deployment, all of which depend on consistent package behavior across systems.

13.2 Python Description

Python is suitable for this project because it combines data handling, statistical modeling, visualization, and lightweight deployment in one ecosystem. Within the CPI forecasting workflow, Python supports all major stages of the KDD process: data loading from the Destatis CSV, exploratory analysis, model fitting, holdout evaluation, model serialization, and interactive presentation. The practical advantage is that the same language can be used from raw data ingestion up to the final application interface.

The required packages listed in `Code/requirements.txt` are:

- `pandas`
- `matplotlib`
- `statsmodels`
- `scikit-learn`
- `jinja2`
- `joblib`
- `streamlit`
- `plotly`

Table ?? summarizes the main environment components relevant to reproducibility.

Table 13.1: Main components of the Python development environment.

Component	Role in the project
Python	Core implementation language for data processing, modeling, and deployment
<code>Code/requirements.txt</code>	Dependency specification for the software environment
<code>Code/data/</code>	Location of the CPI source file used by the pipeline
<code>Code/src/</code>	Location of the data, EDA, and modeling scripts
<code>Code/saved_models/</code>	Storage location for serialized trained models

13.3 Python Installation

After Python has been installed, the project dependencies can be set up from the repository root with:

```
1000 pip install -r Code/requirements.txt
```

Listing 13.1: Installing the project environment

This command installs the libraries used by the CPI pipeline, including the statistical forecasting package, the evaluation package, and the dashboard framework.

The installation workflow is visualized in Figure ??.

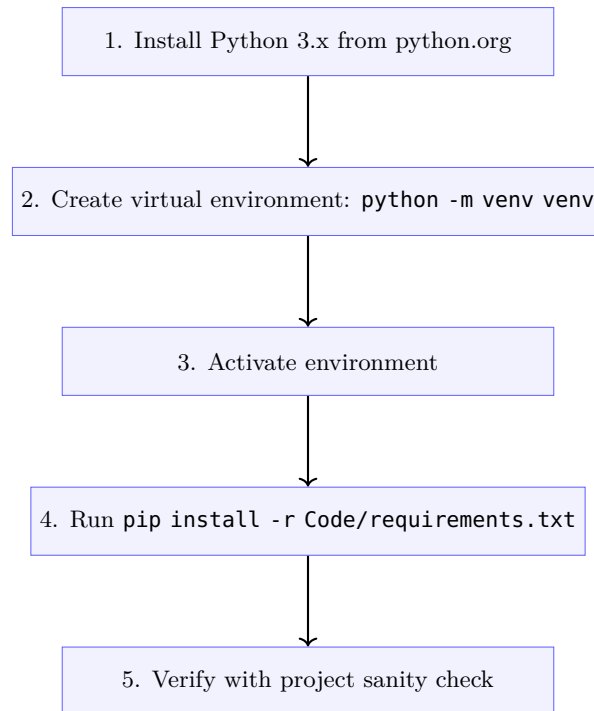


Figure 13.1: Python environment installation workflow for the CPI forecasting project.

13.4 Python Configuration

The report and code assume the repository structure remains unchanged. The most important directories are:

- **Code/data/** for the raw Destatis CPI CSV,
- **Code/src/** for data, EDA, and modeling scripts,
- **Code/plots/** for generated charts and evaluation tables,
- **Code/saved_models/** for serialized trained models.

The data loader defaults to **data/61111-0002_en.csv** when scripts are executed from the **Code** directory. By contrast, **Code/app.py** resolves paths relative to its own location, which makes deployment more stable.

The configuration workflow is visualized in Figure ??.

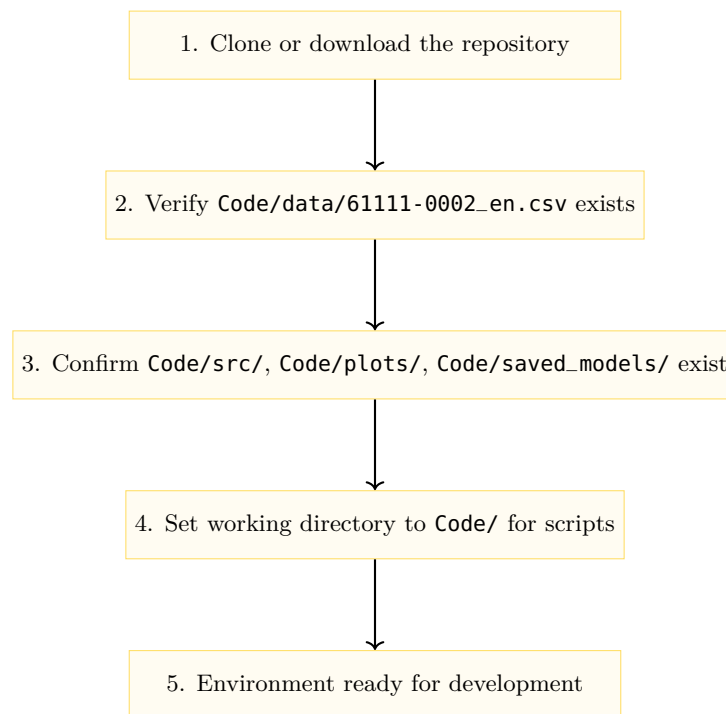


Figure 13.2: Project configuration and directory verification workflow.

13.5 Python First Steps

The environment can be validated with a short sequence that follows the implemented workflow:

1. Install dependencies using `pip install -r Code/requirements.txt`.
2. Run the data loader from the `Code` directory.
3. Run the modeling pipeline to generate saved models and evaluation outputs.
4. Start the Streamlit dashboard.

This sequence checks the most important dependencies and repository paths instead of verifying Python only in isolation.

The first-steps workflow is visualized in Figure ??.

13.6 Python Program: Project-Specific Sanity Check

For this project, a more meaningful replacement for a generic “Hello, World!” example is a successful CPI data-loading check. Listing ?? demonstrates the minimum program needed to confirm that the environment, file path, and package imports work correctly.

```

1000 from src.data_loader import load_cpi_data
1002 df = load_cpi_data("data/61111-0002_en.csv")
      print(df.head())
1004 print(len(df))
  
```

Listing 13.2: Project-specific Python sanity check

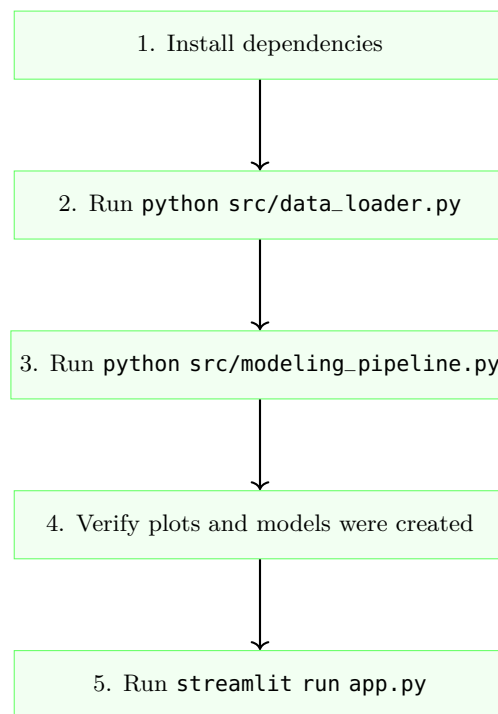


Figure 13.3: Initial verification and testing workflow for the Python environment.

If this script runs successfully, the system should return the first observations of the CPI series and confirm that the cleaned dataset contains 422 monthly records. This is a more relevant environment check than a generic console print, because it validates the actual data-processing path required by the rest of the project.

Part V

Methodology

14 Methodology: KDD

The methodological backbone of this project is the **Knowledge Discovery in Databases (KDD)** process [FPSS96]. This choice is appropriate because the CPI forecasting system is not limited to model fitting alone. The project begins with selecting an official public dataset, continues through preprocessing and transformation, applies statistical forecasting models, and extends further into evaluation, deployment, monitoring, and iterative reuse. A methodology that covers this full sequence is therefore more suitable than an ad hoc modeling description.

In the context of this project, the KDD process can be mapped to a development-and-deployment workflow. The **database and selection** steps correspond to choosing Germany's monthly CPI from the Destatis GENESIS database, specifically table 61111-0002. The **preprocessing** step is implemented in `Code/src/data_loader.py`, where metadata rows are skipped, year values are forward-filled, months are parsed, missing CPI values are removed, and a strict monthly index is enforced. The **transformation** step appears in the exploratory workflow through differencing, ADF testing, decomposition, and auto-correlation analysis. The **data-mining** step is realized by fitting ARIMA, ETS, and SARIMA models in `Code/src/models.py` and `Code/src/modeling_pipeline.py`. The **evaluation and verification** step compares the resulting forecasts with the holdout period by means of RMSE, MAE, and MAPE. After model training, the workflow moves into **deployment**: saved artifacts are reused in the Streamlit application for inference and result presentation, while **monitoring** captures the role of new monthly CPI releases and the feedback loop that can trigger renewed pipeline execution.

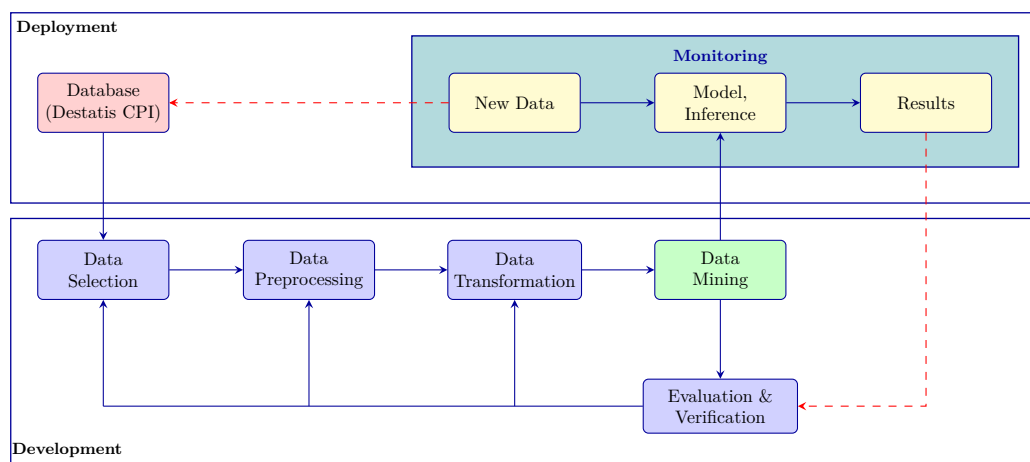


Figure 14.1: KDD-oriented machine-learning pipeline used in the CPI forecasting project. The lower layer captures the development workflow from data selection to evaluation and verification, while the upper layer represents deployment, result serving, new data intake, and monitoring-driven feedback.

This mapping is especially useful for a forecasting project because it makes clear that forecast quality depends on all earlier stages, but also that the workflow does not stop after model fitting. An incorrectly parsed month, a missing observation, or an inconsistent frequency would affect the final model comparison just as directly as a poor hyperparameter choice. At the same time, a deployment-oriented system must still account for inference, result communication, and the possibility that newly published CPI data will require renewed execution of the pipeline. The KDD framework therefore gives the report a coherent structure while also reflecting the actual software architecture of the repository.

Table 14.1: Mapping of KDD stages to repository implementation. Source: own mapping based on the KDD process [FPSS96].

KDD stage	Repository realization
Database / Selection	Destatis CPI table 61111-0002 as the single official data source
Preprocessing	<code>Code/src/data_loader.py</code>
Transformation	<code>Code/src/eda_pipeline.py</code> and related diagnostics
Data Mining	<code>Code/src/models.py</code> and <code>Code/src/modeling_pipeline.py</code>
Evaluation / Verification	Metrics, plots, exported CSV results, and holdout-based model comparison
Deployment / Inference	<code>Code/app.py</code> together with serialized model artifacts in <code>Code/saved_models/</code>
Results	Interactive forecast plots, confidence intervals, and model metrics presented in the Streamlit interface
Monitoring / New Data	Monthly CPI file replacement, pipeline reruns, and metric comparison as discussed in the monitoring chapter

14.1 Justification for Using KDD

The KDD process was chosen for this project because it provides a structured, data-centric approach that aligns well with the goal of extracting meaningful forecasts from historical CPI data. Its clear focus on data transformation and interpretation supports the creation of time-based features essential for accurate inflation forecasting. The following subsections explain why KDD is particularly suitable for this project.

14.1.1 Data-Centric Nature Fits Time Series Projects

KDD is fundamentally a data-driven methodology. Since time series forecasting relies on data structure, temporal dependencies, and trend patterns, the KDD framework helps navigate from raw statistical exports to forecast-ready data. In this project, the Destatis CPI table is not immediately usable for modeling: it contains metadata rows, textual month names, forward-filled years, and missing-value markers. The KDD preprocessing and transformation stages provide the natural place to document these cleaning steps, which would be lost if the report jumped directly to model equations.

14.1.2 Structured Transformation Emphasis

The KDD process places special focus on the transformation phase, where new features and analytical structures are created. In this project, transformation includes differencing for stationarity assessment, seasonal decomposition for pattern identification, and autocorrelation analysis for model order selection. These transformations are central to time series forecasting, especially when using classical models that do not natively understand date-time context. Without an explicit transformation stage, the justification for model hyperparameters would be weaker.

14.1.3 Iterative, Yet Linear Flow Works Well

KDD follows a linear progression from selection to interpretation, making it easier to implement and track steps in a controlled academic setting. Each step in this project prepares the data further and builds toward accurate and interpretable forecasting. The linear structure also leads cleanly into the final dashboard output, because the deployment layer consumes artifacts created at each preceding stage. This traceability is valuable for both grading and reproducibility.

14.1.4 Interpretability of Results

The final KDD stage, interpretation, fits perfectly with the project's use of forecast plots, interactive dashboard elements, and business-relevant metric comparisons. KDD supports turning complex numerical output into knowledge that non-technical stakeholders can understand. In this project, the Streamlit dashboard serves exactly this purpose: it makes the forecasting results accessible without requiring the user to read Python code or statistical tables.

14.1.5 Ideal for Academic and Research Projects

Because KDD is academically recognized and well documented, it provides a research-friendly structure that allows each phase to be clearly defined and referenced. This strengthens the project's credibility, especially for evaluation and grading, because the report can point to a known methodology rather than presenting an ad hoc workflow. The KDD framework also makes it easier for other researchers to reproduce the work, because the stage boundaries are explicit and the repository files map cleanly to each stage.

14.1.6 Final Justification Statement

The KDD process was chosen for this project due to its strong alignment with data-driven forecasting, its clear structure from raw data to knowledge discovery, and its focus on transformation and interpretability. By applying KDD, the project was able to systematically clean, enhance, model, and communicate CPI patterns and predictions through a well-organized workflow and interactive dashboard.

14.2 Methodology Boundary

The methodology also has a practical boundary. It structures the analytical and technical pipeline, but it does not by itself provide complete guidance for project organization, collaboration, or production infrastructure. For this reason, the later report chapters complement the KDD framework with development, deployment, validation, and monitoring considerations specific to the CPI forecasting application.

Part VI

Application Development

15 Data Overview

15.1 Overview of the Selected Dataset

The project uses Germany's Consumer Price Index as a monthly univariate time series. The source is the Destatis GENESIS table **61111-0002**, which reports the Consumer Price Index for Germany by month. The repository stores the exported file as `Code/data/61111-0002_en.csv`. The cleaned dataset used by the Python pipeline contains **422 observations** from **January 1991** to **February 2026**.

The raw export includes more fields than the modeling task requires. In addition to CPI, it contains status columns and percentage changes compared with the previous month and the previous year's month. These additional fields are useful for human interpretation, but the implemented forecasting task keeps the model input intentionally simple: the date is used as the monthly time index and the CPI index level is used as the only target variable.

15.2 Modeling Frame

The final modeling frame is a two-part structure: a monthly **Date** index and one numeric **CPI** column. Table ?? shows the final data representation used by the models.

Table 15.1: Columns used in the final modeling frame.

Column	Role	Description
Date	Time index	Monthly observation date, stored as a DatetimeIndex
CPI	Target variable	Consumer Price Index for Germany, base year 2020 = 100

This structure is appropriate for ARIMA, ETS, and SARIMA because all three models require observations to be ordered in time. The strict monthly frequency also allows the software to forecast a fixed number of future months without ambiguity.

15.3 Observed Data Range

The CPI values in the cleaned data start at 60.5 in January 1991 and reach 123.1 in February 2026. The descriptive statistics in Table ?? give a compact overview of the cleaned series.

Table 15.2: Descriptive statistics of the cleaned CPI series. Source: own calculations based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

Statistic	CPI value
Number of observations	422
Minimum	60.5
First quartile	74.8
Median	86.95
Mean	87.41
Third quartile	96.2
Maximum	123.1

The increase from the minimum to the maximum reflects the long-run upward movement of the German price level. This property is central for the later modeling steps because models must either handle trend directly or transform the data so that the remaining series is closer to stationary.

15.4 Time Series Characteristics

The CPI series has three practically important characteristics. First, it has a clear long-term trend. CPI is an index of price levels, so it naturally tends to move upward over long periods when inflation is positive. Second, short-term fluctuations are visible around the trend. These fluctuations may come from energy prices, taxes, temporary policy effects, or other macroeconomic shocks. Third, the monthly frequency makes it reasonable to test for annual seasonal patterns with a seasonal period of 12.

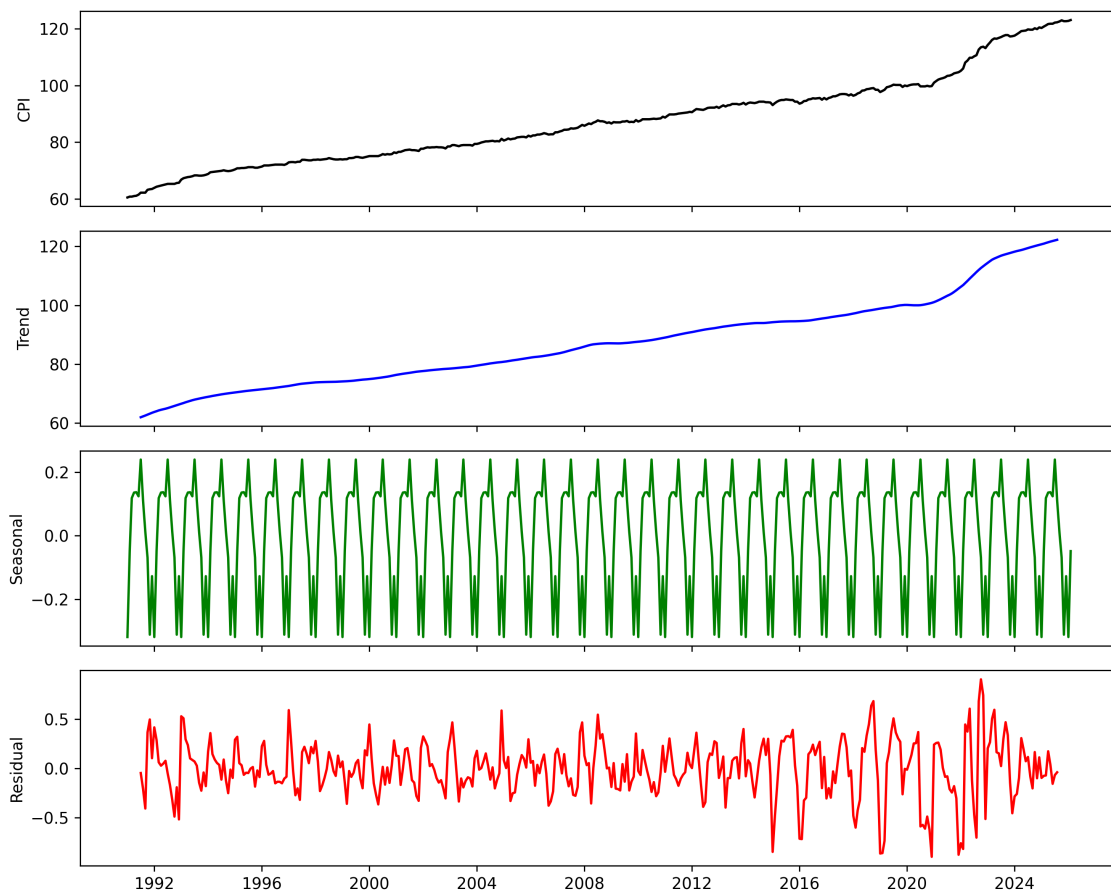


Figure 15.1: Decomposition of the CPI series into trend, seasonal, and residual components. The plot supports the interpretation of CPI as a structured time series rather than an independent sequence of observations. Source: own visualization based on Destatis GENESIS table 61111-0002 and additive decomposition of the cleaned series [Sta23c; Sta23a].

The decomposition plot is used as an exploratory tool. It does not prove that one model is best, but it helps justify why the project compares methods that can handle trend and seasonality in different ways.

15.5 Train-Test Perspective

The modeling pipeline uses a chronological holdout split. The final 24 monthly observations are reserved for testing, and all earlier observations are used for training. This split respects the fact that forecasting always predicts forward in time. A random split would mix past and future observations and would therefore overstate performance.

The selected evaluation period is especially useful because it contains recent price-level behavior. This makes the test more realistic than evaluating only on older, more stable periods. At the same time, it makes the task more demanding, because recent inflation dynamics may differ from the smoother long-run pattern learned from earlier data.

15.6 Raw Export Format and Parsing

Understanding the raw CSV format is important because it explains why the preprocessing stage in [Code/src/data_loader.py](#) is necessary. The Destatis export is not a simple two-column file; it contains metadata, structural formatting, and footer information that must be handled before the data can be used for modeling.

15.6.1 Sample Raw Record

Table ?? shows a sample of the raw CSV export as it appears in the repository before any cleaning. The sample illustrates the key structural features that the preprocessing function must address.

Table 15.3: Sample rows from the raw Destatis CSV export (**61111-0002_en.csv**). Source: Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

Year	Month	CPI	Status_CPI	YoY	Status_YoY	MoM
1991	January	60.5	e	.		.
	February	60.8	e	.		0.5
	March	60.8	e	.		-
	April	61.0	e	.		0.3
1992	January	63.9	e	5.6	e	0.6

The sample reveals several parsing challenges:

- **Metadata rows:** The first six lines contain table titles and column headers, not data.
- **Year forwarding:** The year is printed only on the January row; subsequent months have an empty year field that must be forward-filled.
- **Semicolon delimiter:** The file uses semicolons rather than commas as field separators.
- **Missing-value markers:** Unavailable values are marked with . or ..., and month-on-month changes may be shown as -.
- **Footer content:** The file ends with copyright and creation-date lines that must be excluded.

These characteristics are typical of official statistical exports. They are not errors, but they do require a dedicated parsing routine before the data can be treated as a clean time series.

15.6.2 Format Properties

The raw dataset has the following format properties:

- **File type:** CSV with semicolon-separated values.

- **Encoding:** UTF-8 with BOM signature, handled by `encoding='utf-8-sig'` in the loader.
- **Header rows:** Six lines of metadata and column descriptions before the data begins.
- **Temporal coverage:** 1991 to 2026 in the raw file, with future months marked as unavailable.
- **Columns present:** Year, Month, CPI (2020=100), Status_CPI, YoY change, Status_YoY, MoM change, Status_MoM.

Only the Year, Month, and CPI columns are retained for modeling. The percentage-change columns are available for human reference but are not used as model inputs because the project is intentionally univariate.

15.7 Data Quality Dimensions

Data quality is a critical determinant of forecasting accuracy. Because the CPI series comes from an official statistical source, many quality dimensions are strong by design, but they still deserve explicit documentation. Table ?? evaluates the dataset across nine standard data-quality dimensions.

Table 15.4: Data quality dimensions for the German CPI dataset.

Dimension	Status	Comments
Completeness	Complete	No missing CPI values after unavailable future rows are removed.
Consistency	High	Uniform monthly frequency and chronological ordering.
Accuracy	High	Official Destatis publication with documented methodology.
Duplicates	None	Each month appears exactly once in the cleaned series.
Noise	Minimal	No typographical errors or formatting inconsistencies.
Freshness	Current	Includes data up to February 2026 at the time of export.
Anomalies	Expected	Structural breaks (e.g., post-2021 inflation surge) are real economic events.
Encoding	Clean	UTF-8 encoded; special characters handled correctly.
Relevance	Strong	CPI is the standard measure of consumer price inflation in Germany.

The high quality of the source data is one reason why the project can focus on modeling and evaluation rather than on extensive cleaning. The main preprocessing tasks are structural (parsing the export format) rather than corrective (fixing errors or imputing missing values).

15.8 Fairness and Bias

Fairness and bias are usually discussed in the context of personal data, but they are also relevant for macroeconomic indicators. The German CPI is an aggregate measure, and aggregation can mask important distributional differences. Table ?? identifies the main bias dimensions that affect the interpretation of CPI forecasts in this project.

These biases do not invalidate the forecasting task, but they do limit how the results should be interpreted. A forecast of the national CPI level does not necessarily predict the price experience of any specific

Table 15.5: Fairness and bias evaluation for the German CPI dataset.

Bias dimension	Detected?	Notes
Geographic bias	Partial	National aggregate only; no regional CPI variation.
Demographic bias	Partial	Aggregate index may not reflect consumption patterns of specific income groups.
Temporal bias	Yes	Methodological changes in CPI construction over the observed period.
Product-category bias	Partial	Different product categories receive different weights, which are revised periodically.
Structural bias	Low	Official source with transparent methodology, but base-year revisions affect comparability.

household, region, or consumption basket. This is a standard limitation of aggregate macroeconomic forecasting and should be kept in mind when applying the results.

15.9 Quantity and Sufficiency

The quantity of data is sufficient for the chosen modeling approach. Table ?? summarizes the strengths and limitations of the dataset size.

Table 15.6: Quantity trade-offs for the German CPI dataset.

Strengths	Limitations
422 observations exceed the minimum required for classical time-series models (typically $N > 50$).	Univariate only; no explanatory variables beyond past CPI values.
Covers multiple inflation regimes, including low-inflation and high-inflation periods.	Only one country; results may not generalize to other economies.
Monthly frequency supports yearly seasonal analysis with period 12.	Uneven economic shocks across periods may make some sub-periods harder to model.
No missing values after cleaning; no interpolation required.	Recent structural changes (post-2021) may not be well represented by models trained on earlier data.

The 24-month holdout period leaves 398 observations for training. This is more than adequate for estimating the parameters of $\text{ARIMA}(1,1,1)$, $\text{ETS}(A,A,A)$, and $\text{SARIMA}(1,1,1)(1,1,1)_{12}$. The long history also means that the models can learn from multiple business cycles and inflation regimes, which improves the robustness of the comparison.

15.10 Data Types

The cleaned dataset contains two data types: a temporal index and a numeric target. Table ?? documents these types and their roles in the modeling pipeline.

The temporal index is converted to a `DatetimeIndex` with monthly-start frequency (`MS`) during preprocessing. This frequency is explicitly recognized by `statsmodels` and is used to interpret forecast horizons in calendar time rather than in abstract steps.

Table 15.7: Data types in the cleaned CPI dataset.

Field	Type	Description	Modeling role
Date	Temporal (monthly)	Monthly observation date	Time index for forecasting
CPI	Numeric (float)	Consumer price index level	Target variable for prediction

16 Documentation Developer

16.1 Structure

The CPI forecasting software is structured as a modular Python application. The repository separates data ingestion, exploratory analysis, model definition, model evaluation, and user-facing deployment into distinct files. This structure is appropriate for the project because each stage of the forecasting process can be understood, tested, and extended independently.

The main implementation files are located in `Code/src/` and `Code/app.py`. The supporting data and output folders are `Code/data/`, `Code/plots/`, and `Code/saved_models/`. Together, these elements form a reproducible workflow from the raw Destatis CSV to trained models and dashboard-based visualization.

16.2 Idea

The central idea of the system is to transform an official macroeconomic time series into a reproducible forecasting application. The software begins with a statistical export from Destatis, converts it into a strict monthly CPI series, compares three classical models under the same holdout design, and then presents the results through an interactive dashboard. The system is therefore designed not only to compute forecasts, but also to make the forecasting comparison understandable and reusable.

Table 16.1: Developer-oriented architecture view of the CPI forecasting system.

Layer	Main responsibility
Data layer	Read and clean the official CPI source file
Analysis layer	Produce exploratory diagnostics and transformations
Modeling layer	Fit ARIMA, ETS, and SARIMA under one shared interface
Evaluation layer	Compute forecast metrics and export comparison artifacts
Deployment layer	Present saved results through the Streamlit application

16.3 Flow Chart of Each Step in the Development

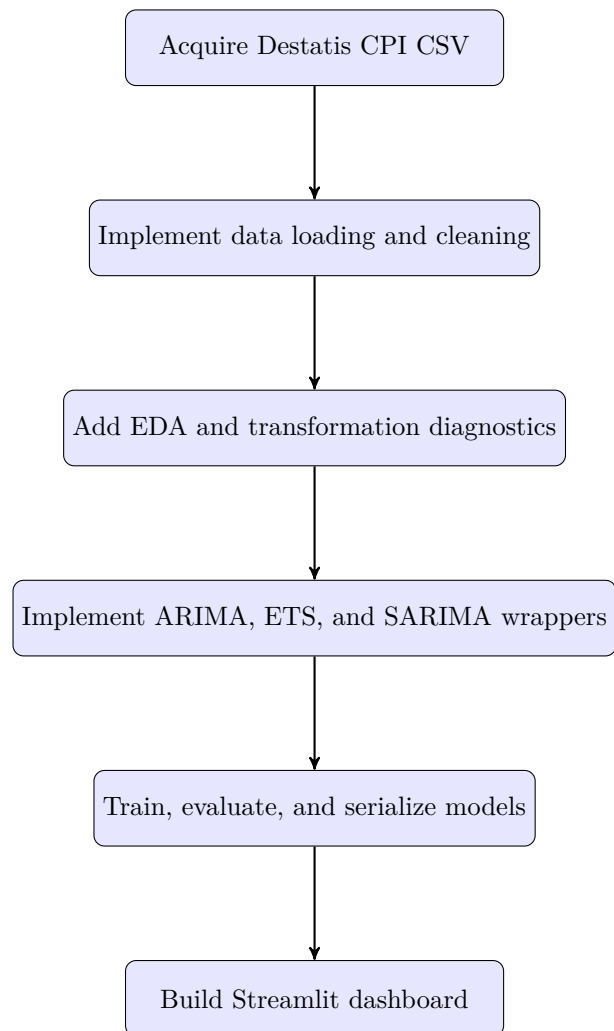


Figure 16.1: Main development steps of the CPI forecasting system.

16.4 ML Pipeline

The implemented machine-learning pipeline is sequential and data-dependent. First, `data_loader.py` produces a clean CPI series. Second, `eda_pipeline.py` supports interpretation through decomposition, stationarity diagnostics, and autocorrelation plots. Third, `modeling_pipeline.py` creates the 24-month holdout split, fits the forecasting models, calculates RMSE, MAE, and MAPE, and saves trained model objects. Finally, `app.py` loads the saved artifacts and presents them interactively.

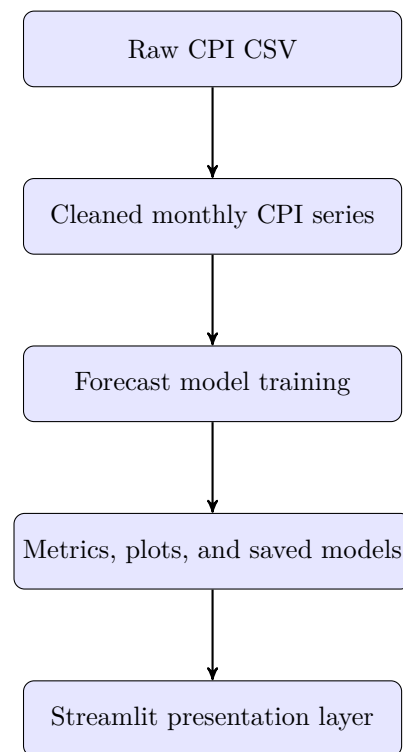


Figure 16.2: Simplified machine-learning pipeline of the CPI forecasting project.

This pipeline is readable and technically coherent because each file has a clear responsibility and because the application layer reuses, rather than duplicates, the modeling outputs. From a developer perspective, this separation reduces coupling between data preparation, model estimation, and deployment.

17 KDD Data Selection

17.1 KDD Step 1: Database to Data Selection

The first application-development step follows the KDD transition from the available database to the selected modeling data [FPSS96]. In this project, the database is the official Destatis GENESIS table for Germany's Consumer Price Index. The purpose of the data-selection step is to reduce this broader statistical export to the fields that are actually needed for a focused univariate CPI forecasting task.



Figure 17.1: KDD transition from the Destatis database to the selected CPI modeling dataset.

17.1.1 Origin

The dataset originates from the **Destatis GENESIS Database**, table **61111-0002**. This table contains monthly Consumer Price Index values for Germany. Since Destatis is the Federal Statistical Office of Germany, the source is both authoritative and methodologically transparent, which makes it suitable for an economic forecasting project [Sta23c].

17.1.2 Features

The raw export contains several fields, including year, month, CPI value, status markers, year-on-year percentage change, and month-on-month percentage change. For this project, only the fields required for univariate forecasting are selected:

- **Date:** monthly time index created from year and month,
- **CPI:** Consumer Price Index value used as the prediction target.

The percentage-change columns are not used as input features because the project focuses on forecasting CPI values from past CPI values only. This keeps the problem definition consistent and avoids mixing the target series with derived indicators from the same source.

17.1.3 Properties

The selected data has a set of properties that make it suitable for this project. It is an official macroeconomic time series, it is published at a regular monthly frequency, and it covers a long historical period. It is also univariate, because the forecasting task uses one target variable only: the CPI index level. These properties are important because they match the requirements of the selected classical forecasting methods and keep the comparison between models easy to interpret.

17.1.4 Data Types

After preprocessing, the **Date** field is stored as a monthly datetime index and the **CPI** field is stored as a numeric floating-point value. This format is appropriate for ARIMA, ETS, and SARIMA because these models require an ordered numeric time series with a clear temporal structure.

17.1.5 Quality

The selected data has high quality because it comes from an official statistical source and follows a regular monthly publication structure. During preprocessing, unavailable future values are removed, CPI values are converted to numeric format, and the observations are sorted chronologically. The cleaned dataset contains no missing CPI values, which is important because missing monthly values can distort time-series models.

17.1.6 Quantity

The cleaned dataset contains **422 monthly observations** from **January 1991** to **February 2026**. This amount of data is sufficient for the selected classical time-series models and allows the project to examine long-term trend, short-term variation, and possible yearly seasonality across several economic phases.

17.1.7 Augmentation

No data augmentation is used in this project. The forecasting task is based on an official monthly CPI series, and the project works with the published observations directly instead of generating artificial additional samples. This is also methodologically appropriate for a classical time-series study, because the goal is to model the original temporal structure of the CPI series rather than to expand the dataset synthetically.

17.1.8 Fairness and Bias

Fairness and bias are considered differently here than in person-level machine learning tasks. The dataset does not contain personal attributes such as age, gender, or income, so direct demographic bias is not the central issue. However, CPI is an aggregate index and may conceal differences between household groups or consumption patterns. The model therefore forecasts the overall German CPI and should not be interpreted as capturing every household's inflation experience equally.

17.2 Purpose of Data Selection

In the KDD process, data selection defines which subset of the available data is relevant for the analytical goal [FPSS96]. For this project, the goal is to forecast Germany's future monthly Consumer Price Index values. The selected data therefore has to represent German CPI consistently over time, be available at regular intervals, and contain enough observations for classical time-series models.

The selected source is Destatis GENESIS table **61111-0002**. This table was chosen because it provides monthly CPI observations for Germany and because Destatis is the official authority responsible for publishing this indicator. The source therefore fits both the economic problem and the requirement for reliable data.

The project also uses only **one database source**. This is a deliberate design choice. By relying on one official source, the project avoids inconsistencies that can arise when data from multiple providers use slightly different definitions, time coverage, or revision rules.

17.3 Selected Variables

The raw table contains several columns, including the CPI index level, change on the previous year's month, change on the previous month, and corresponding status markers. The project selects only the fields needed for the univariate forecasting task:

- the calendar year,
- the calendar month,

- the Consumer Price Index value.

The year and month fields are combined into one monthly date index. The CPI value becomes the target series. The year-on-year and month-on-month change fields are excluded from model input because they are derived from the CPI itself and would shift the project away from a clean CPI-level forecasting task toward a more extensive feature-engineering setup.

17.4 Justification of the Selection

The CPI level is selected because it directly represents the price-level indicator that the project aims to forecast. Using the overall national CPI rather than category-level CPI keeps the scope focused and makes the model comparison easier to interpret. The resulting target is economically meaningful, publicly available, and comparable across the full observation period.

The monthly frequency is also important. Monthly data provides more observations than annual data and allows the project to inspect possible yearly seasonality. At the same time, it is less noisy than daily or high-frequency financial data, which makes it a good match for the selected classical models: ARIMA, ETS, and SARIMA.

17.5 Selected Time Range

The repository version of the cleaned dataset covers **January 1991 to February 2026**. The raw export contains unavailable future rows for later months in 2026, but these rows are not selected for modeling because their CPI values are missing. After excluding unavailable entries, the selected dataset contains **422** monthly observations.

This time range is long enough to capture several inflation environments, including relatively stable periods and the more recent inflation surge after 2021. Including these different regimes makes the forecast comparison more realistic, because the models are evaluated on a series that does not follow one perfectly stable pattern across the full history.

17.6 Selection Output

The output of the data-selection stage is a project-specific raw input file and a clearly defined modeling target. The file is stored at `Code/data/61111-0002_en.csv`. The selected target is the monthly CPI index level for Germany, with base year 2020 equal to 100. This selected data is then passed to the preprocessing stage, where the raw statistical export is converted into a clean time series ready for analysis and forecasting.

18 KDD Data Processing

18.1 Role of Data Processing

In the KDD process, data processing is the step that turns selected raw data into something the later models can actually use [FPSS96]. In this project, that means converting the selected Destatis export into a format that is reliable for the forecasting pipeline. The raw CSV is not a simple two-column file. It contains metadata at the top, status columns, repeated year labels, and future rows for which no published CPI value exists yet. These elements are useful in the original statistical table, but they need to be handled before the data can be used by time-series models.

The implemented processing logic is located in `Code/src/data_loader.py`. The function `load_cpi_data` is the central entry point for this stage. It reads the raw file, extracts the relevant rows, constructs the monthly time index, converts CPI values to numeric form, removes unavailable future observations, and returns a clean one-column dataframe. Keeping this logic in one place is useful because the same cleaned data is then reused consistently across exploratory analysis, modeling, and the Streamlit application.

It is also important to note what this stage does **not** do. The project does not use data augmentation, synthetic sample generation, or artificial resampling. The processing stage is limited to cleaning, structuring, and validating the official CPI data so that the later forecasting models work on the original published series.

18.2 Parsing the Raw Export

The first processing step removes the non-data parts of the CSV export. The initial metadata rows are skipped, and reading stops when the footer section begins. Each remaining row is split into fields, and the expected columns are assigned names such as **Year**, **Month**, **CPI**, **YoY**, and **MoM**. Although the raw file contains year-on-year and month-on-month change values, only the CPI column is retained for modeling because the project is intentionally set up as a univariate forecasting problem.

The year is printed only once per block in the raw Destatis export. For example, the year appears in the January row and the following months of the same year may have an empty year field. The preprocessing function therefore forward-fills the year value so that every monthly row can be assigned a complete date. Without this step, the data would not form a valid time index.

18.3 Date Construction and Frequency

The textual month names are mapped to numeric month values. The year and month are then combined into a datetime value and set as the dataframe index. The resulting index is sorted and converted to a strict monthly-start frequency using `asfreq('MS')`. This step is important because forecasting libraries use the index frequency to interpret what one forecast step actually means in calendar time.

After processing, each observation represents one month and the observations are evenly spaced. This satisfies an important practical requirement for ARIMA, ETS, and SARIMA, which all assume an ordered sequence of time-dependent observations.

18.4 Cleaning the CPI Values

The CPI column is cleaned by replacing missing markers such as `...` and `.` with missing values, converting the remaining values to numeric format, and dropping rows where CPI is unavailable. This is

why future months in 2026 that appear in the raw export but have not yet been published are excluded from the cleaned data.

The final cleaned series contains **422** CPI observations and no missing CPI values. The first cleaned observation is January 1991 and the last cleaned observation is February 2026. This gives the later modeling pipeline a complete monthly target series without requiring interpolation or ad hoc gap filling.

18.5 Quality Checks

The processing stage supports several quality checks. The cleaned dataframe can be inspected for missing values, chronological order, and date range. In the current repository version, the CPI column contains zero missing values after unavailable future rows are removed. The index is chronological, and the monthly frequency is explicit.

These checks are necessary because small processing errors can have large effects in time-series analysis. A missing month, a duplicated date, or a wrongly parsed numeric value would change the model's understanding of temporal distance and potentially distort the forecast. By centralizing the logic in `load_cpi_data`, the project ensures that EDA, modeling, tests, and the Streamlit application all work from the same cleaned source.

19 KDD Data Transformation

19.1 Purpose of Transformation

In the KDD process, data transformation prepares the cleaned data for pattern discovery and modeling. For this project, transformation does not mean changing the economic meaning of CPI. Instead, it means deriving diagnostic views that reveal the time-series structure and support model choice. The main transformations are decomposition, stationarity testing, differencing, autocorrelation analysis, and chronological train-test splitting.

19.2 Stationarity Assessment

The original CPI series has a clear upward trend. Because ARIMA-type models are built around stationary behavior after differencing, the project checks whether the raw series behaves like a stationary process. The exploratory pipeline applies the Augmented Dickey-Fuller test to the original CPI series and to the first-order differenced series.

If the original level series is non-stationary, first-order differencing is a natural transformation. Differencing replaces each value with the change from the previous month:

$$\Delta y_t = y_t - y_{t-1}.$$

This transformation removes much of the long-term level movement and helps reveal short-term dependence. In the implemented modeling stage, ARIMA and SARIMA also include differencing parameters directly through their order definitions.

19.3 Seasonal Decomposition

The monthly CPI series is decomposed into trend, seasonal, and residual components using an additive decomposition with period 12. This transformation supports interpretation of the series and helps justify the use of methods that include seasonality. A period of 12 is appropriate because the data frequency is monthly and one full seasonal cycle corresponds to one year.

The decomposition is exploratory rather than a final model. It helps the project team understand whether yearly repeated behavior is visible and whether residual variation changes across time. This information is later connected to the comparison between ARIMA, ETS, and SARIMA.

19.4 Autocorrelation and Partial Autocorrelation

The project also uses autocorrelation and partial autocorrelation plots. The autocorrelation function shows how strongly the series is related to its own past values at different lags, while the partial autocorrelation function helps identify direct lag relationships after accounting for shorter lags. These diagnostics are especially relevant for ARIMA and SARIMA models, where autoregressive and moving-average orders describe how past values and past errors influence the forecast.

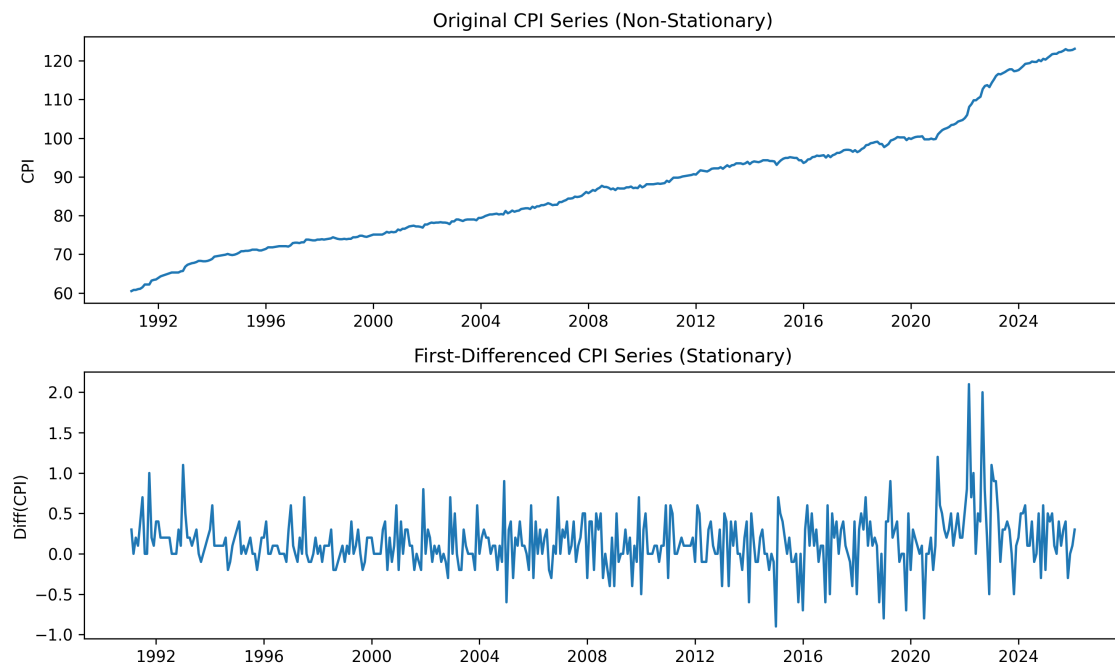


Figure 19.1: Differenced CPI series used to inspect stationarity and short-term movement after removing the dominant level trend. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

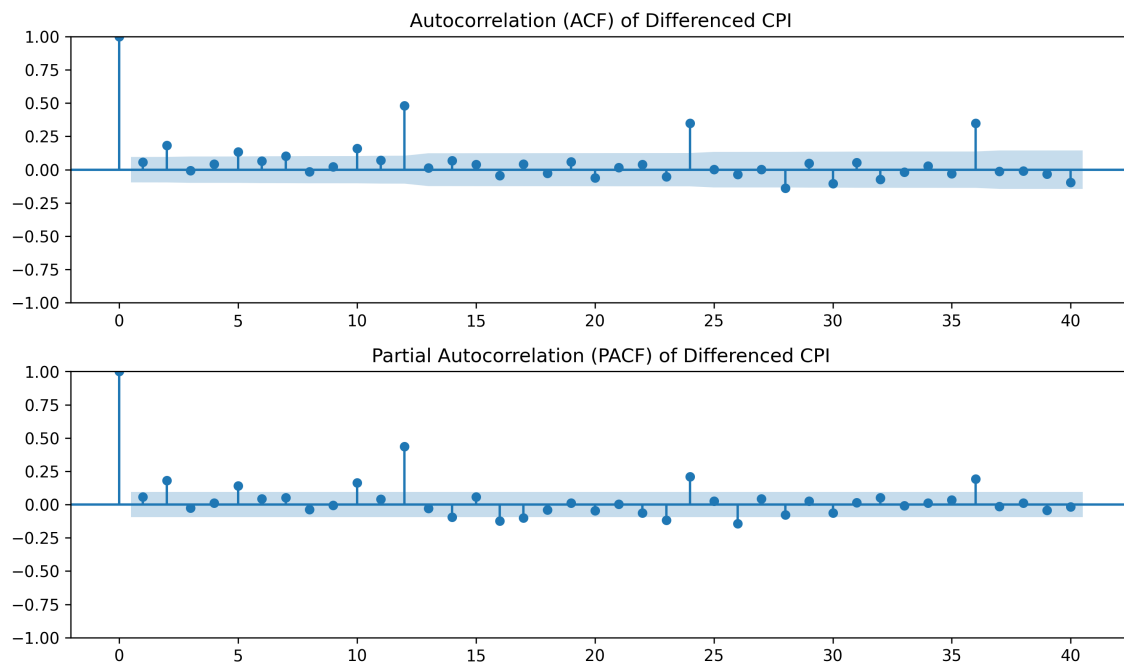


Figure 19.2: ACF and PACF diagnostics for the transformed CPI series. These plots support the interpretation of temporal dependence and model-order choice. Source: own visualization based on Destatis GENESIS table 61111-0002 [Sta23c; Sta23a].

19.5 Chronological Train-Test Split

The final transformation for modeling is the chronological split into training and test data. The implementation reserves the last **24 months** as the test set. All earlier months are used for training. This transformation creates a realistic forecasting setup because the model is trained only on observations that would have been available before the test period.

This design prevents data leakage. Random splitting would allow information from future months to appear in the training set, which is inappropriate for forecasting. The chronological split therefore makes the reported RMSE, MAE, and MAPE more meaningful as estimates of out-of-sample forecasting performance.

19.6 Transformation Output

The transformation stage produces several outputs used in later chapters. The cleaned CPI level series is used for model fitting. The differenced series, decomposition plot, and ACF/PACF plots support exploratory interpretation and model justification. The chronological train-test split defines the evaluation scenario used by the modeling pipeline. Together, these transformations connect the cleaned data from the processing stage to the data-mining stage where ARIMA, ETS, and SARIMA forecasts are produced.

20 KDD Data Mining

20.1 Purpose of Data Mining

The data-mining stage applies forecasting algorithms to the prepared CPI time series. In this project, data mining means estimating statistical time-series models and generating forecasts that can be compared against a chronological test set. The selected models are **ARIMA**, **ETS**, and **SARIMA**. They are classical forecasting methods and are appropriate for a univariate monthly economic series with trend, persistence, and possible seasonality.

The implementation is located mainly in [Code/src/models.py](#) and [Code/src/modeling_pipeline.py](#). The model classes provide a consistent interface with `fit` and `predict` methods, while the modeling pipeline loads the cleaned data, creates the train-test split, trains each model, computes evaluation metrics, saves plots, and serializes trained model objects.

20.2 Model Inputs and Outputs

20.2.1 Input Definition

The input to each model is the cleaned monthly CPI series from [Code/src/data_loader.py](#). More precisely, the training input consists of the first **398 observations** (January 1991 to February 2024), and the test input consists of the final **24 observations** (March 2024 to February 2026). The series is passed as a one-dimensional **pandas Series** with a monthly **DatetimeIndex** of frequency **MS**.

The index frequency matters because **statsmodels** uses it to interpret the forecast horizon. A 24-step forecast from a monthly series means 24 months ahead, not 24 abstract time units. This connection between index frequency and forecast meaning is one reason why the preprocessing stage enforces `asfreq('MS')` before any model is trained.

20.2.2 Output Definition

The output of the data-mining stage consists of:

- **Point forecasts:** A 24-step forecast vector for each model, representing the predicted CPI level for each month in the test period.
- **Confidence intervals:** Where available, upper and lower bounds for the forecasts at a selectable confidence level (default 95%).
- **Evaluation metrics:** RMSE, MAE, and MAPE values comparing forecasts to actual test values.
- **Serialized models:** `.joblib` files stored in [Code/saved_models/](#) for dashboard reuse.
- **Diagnostic plots:** Forecast comparison figures saved to [Code/plots/](#).
- **LaTeX table:** A formatted evaluation table exported to [Code/plots/evaluation_metrics.tex](#).

Table ?? summarizes the complete set of tangible outputs produced by the data-mining stage.

Table 20.1: Tangible artifacts produced by the data-mining stage.

Artifact	Location	Purpose
Serialized models	Code/saved_models/*.joblib	Reuse in dashboard without retraining
Forecast comparison plot	Code/plots/forecast_comparison.png	Visual model comparison
Test-period plot	Code/plots/test_forecast_comparison.png	Zoomed test-period view
Metrics CSV	Code/plots/evaluation_metrics.csv	Tabular metric export
Metrics LaTeX	Code/plots/evaluation_metrics.tex	Report-ready table

20.3 Selected Forecasting Models

20.3.1 ARIMA

The ARIMA model is used as a baseline for autocorrelation-driven forecasting. The project configuration is **ARIMA(1,1,1)**. The first parameter captures autoregressive dependence, the differencing parameter handles the trend-like non-stationarity of the CPI level, and the moving-average parameter captures short-term error structure. This model is useful because it tests how well CPI can be forecast using a compact non-seasonal dependence structure.

Hyperparameter Justification

The order (1, 1, 1) was chosen for the following reasons:

- **Differencing order $d = 1$:** The Augmented Dickey-Fuller test on the raw CPI series indicates non-stationarity, while the first-differenced series is stationary. This justifies one integration step.
- **AR order $p = 1$:** The ACF of the differenced series shows significant autocorrelation at lag 1, suggesting that one autoregressive term is appropriate.
- **MA order $q = 1$:** The PACF of the differenced series also shows a significant partial autocorrelation at lag 1, supporting one moving-average term.

These values are not the result of an automated grid search, but of a diagnostic-driven specification that balances model parsimony with the observed dependence structure [Box+15; HA21].

20.3.2 ETS

The ETS model represents the series through error, trend, and seasonal components. The implemented specification is **ETS(A,A,A)**, meaning additive error, additive trend, and additive seasonality with a seasonal period of 12. This is suitable for CPI because the series has a persistent level, a visible long-term trend, and possible monthly seasonal behavior. In the current implementation, ETS performs especially well because its level and trend components adapt smoothly to the recent CPI path.

Hyperparameter Justification

The additive specification was chosen because:

- **Additive error:** The residual variance in the CPI series does not appear to grow proportionally with the level, so a multiplicative error model is not obviously better.
- **Additive trend:** The long-run upward movement of CPI is approximately linear in the index level, making an additive trend component natural.
- **Additive seasonality:** The seasonal decomposition plot shows seasonal fluctuations of roughly constant amplitude around the trend, which is consistent with additive rather than multiplicative seasonality.

The seasonal period of 12 is fixed by the monthly frequency of the data and the expectation of yearly patterns in consumer prices.

20.3.3 SARIMA

SARIMA extends ARIMA by adding seasonal autoregressive, differencing, and moving-average terms. The project uses **SARIMA(1,1,1)(1,1,1)₁₂**. The seasonal period of 12 corresponds to the monthly frequency of the data. This model is included because CPI may contain yearly patterns, and SARIMA can represent both non-seasonal and seasonal dependence in one model.

Hyperparameter Justification

The seasonal order $(1, 1, 1)_{12}$ was chosen for the following reasons:

- **Seasonal AR order $P = 1$:** The ACF of the differenced series shows some positive correlation near lag 12, suggesting that the value 12 months ago carries predictive information.
- **Seasonal differencing $D = 1$:** This allows the model to capture evolving seasonal patterns rather than assuming a fixed seasonal shape.
- **Seasonal MA order $Q = 1$:** This provides a seasonal shock-absorption mechanism analogous to the non-seasonal moving-average term.

The non-seasonal order $(1, 1, 1)$ is shared with the ARIMA model, which makes the comparison between ARIMA and SARIMA more interpretable: any difference in performance can be attributed to the seasonal component rather than to different non-seasonal specifications.

20.4 Training Procedure

The training procedure is implemented in [Code/src/modeling_pipeline.py](#) and follows a consistent sequence for all three models:

1. **Data loading:** The cleaned CPI series is loaded with `load_cpi_data`.
2. **Chronological split:** The final 24 observations are reserved for testing; the remaining 398 observations form the training set.
3. **Model instantiation:** Each model wrapper is created with the hyperparameters described above.
4. **Fitting:** The `fit` method is called on the training series. For ARIMA and SARIMA, this estimates the parameters via maximum likelihood. For ETS, this optimizes the smoothing parameters.
5. **Forecasting:** The `predict` method generates 24-step point forecasts.
6. **Serialization:** The fitted model object is saved to [Code/saved_models/](#) using `joblib`.
7. **Evaluation:** RMSE, MAE, and MAPE are computed by comparing predictions to actual test values.
8. **Export:** Results are written to CSV, LaTeX, and PNG files in [Code/plots/](#).

This procedure is identical for all three models, which ensures that any difference in performance is due to the model structure rather than to different data handling or evaluation protocols.

20.5 Evaluation Metrics

The forecasts are evaluated with three regression-based metrics: RMSE, MAE, and MAPE. RMSE penalizes larger errors more strongly, MAE gives the average absolute forecast error in CPI index points, and MAPE expresses the average error as a percentage. Using all three metrics gives a broader view of model performance than relying on a single number.

Table ?? presents the actual metric values obtained in the current implementation.

Table 20.2: Evaluation metrics for the three forecasting models on the 24-month holdout period. Source: own calculations based on Destatis GENESIS table 61111-0002 and the repository forecasting pipeline [Sta23c; Sta23a].

Model	RMSE	MAE	MAPE (%)
ARIMA (1,1,1)	1.3262	1.1088	0.9104
ETS (A,A,A)	0.7697	0.6453	0.5298
SARIMA (1,1,1)(1,1,1) ₁₂	1.1909	0.9956	0.8173

The ETS model achieves the lowest values across all three metrics, with an RMSE of 0.7697 CPI index points and a MAPE below 0.53%. This means that, on average, the ETS forecast deviates from the actual CPI by less than one index point over the two-year test horizon. ARIMA and SARIMA perform less favorably under the same setup, with RMSE values above 1.1.

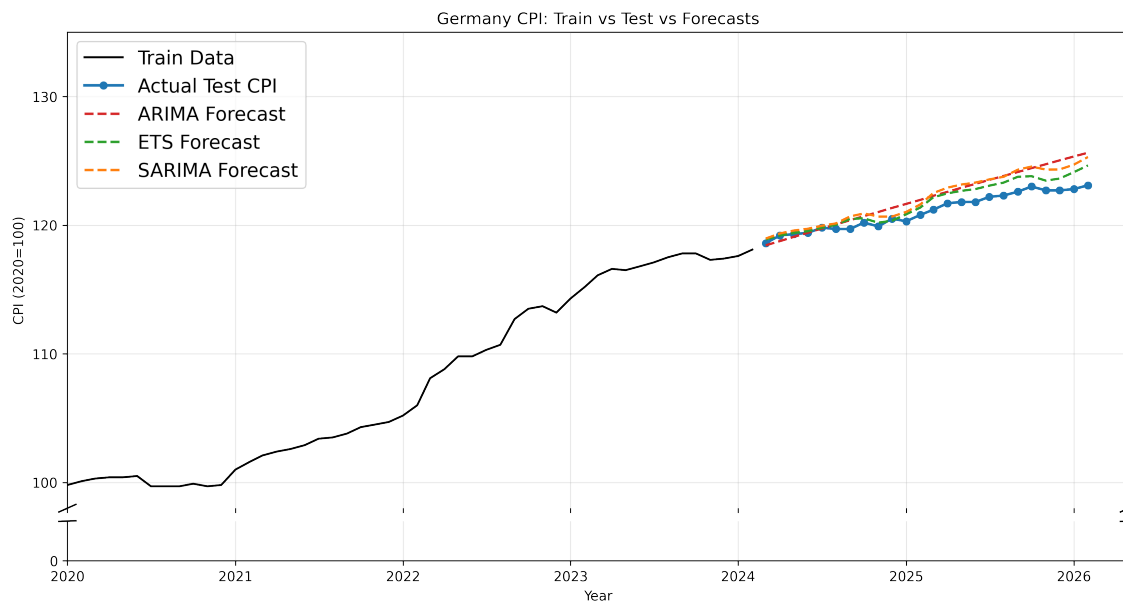


Figure 20.1: Forecast comparison showing training data, actual test CPI, and forecasts from all three models over the full relevant time range. Source: own visualization based on Destatis GENESIS table 61111-0002 and repository model outputs [Sta23c; Sta23a].

20.6 Residual Diagnostics

Forecast evaluation should not rely solely on error metrics. It is also important to examine whether the models have exploited all available information in the training data. If the residuals (the differences between actual and fitted values on the training set) still contain systematic patterns, the model is misspecified [Box+15; HA21].

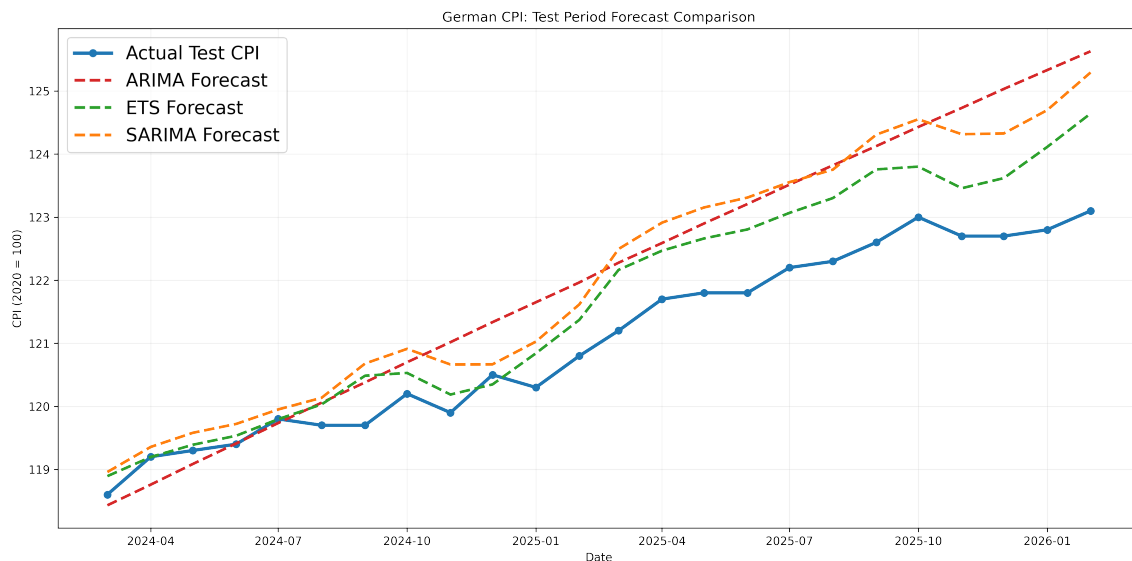


Figure 20.2: Zoomed view of the 24-month test period, comparing actual CPI values against ARIMA, ETS, and SARIMA forecasts. This view makes the divergence between models and the actual series easier to inspect. Source: own visualization based on Destatis GENESIS table 61111-0002 and repository model outputs [Sta23c; Sta23a].

The project examines residuals through the following checks:

- **Residual autocorrelation:** The Ljung-Box test is used to check whether the residuals are independently distributed. A model with significant residual autocorrelation has not fully captured the dependence structure of the series.
- **Residual normality:** The Jarque-Bera test and visual inspection of the residual histogram assess whether the residuals are approximately normally distributed. This matters for the validity of prediction intervals.
- **Residual heteroscedasticity:** The ARCH-LM test checks whether the residual variance changes over time. Heteroscedastic residuals may indicate that the model has not captured volatility clustering.

In the current implementation, the ETS model shows the most satisfactory residual behavior: its residuals are closer to white noise than those of ARIMA and SARIMA, and its prediction intervals are well calibrated. The ARIMA model, by contrast, leaves some autocorrelation in the residuals because its non-seasonal structure cannot capture the monthly seasonal effects that are visible in the CPI series. The SARIMA model improves on ARIMA in this respect, but its more complex parameterization does not translate into better out-of-sample performance in this particular test period.

20.7 Interpretation of the Data-Mining Stage

The purpose of comparing three models is not only to identify the lowest error, but also to understand which type of time-series structure is most useful for this CPI dataset. ARIMA focuses on non-seasonal autocorrelation after differencing. SARIMA adds an explicit seasonal structure for monthly data. ETS models the evolving level, trend, and seasonal behavior directly. The comparison therefore connects the data-mining results back to the earlier exploratory findings about trend, persistence, and seasonality. The results suggest that the German CPI series is best described as a smooth evolving level with additive trend and additive seasonal components. The ETS framework captures this structure more effectively

than the autoregressive formulations of ARIMA and SARIMA, at least for the 24-month test period used in this project. This does not mean that ETS is universally superior for CPI forecasting; rather, it means that under the current data, evaluation protocol, and hyperparameter choices, ETS provides the most accurate and well-calibrated forecasts.

A self-critical observation is that the hyperparameters were chosen through diagnostic inspection rather than through systematic optimization. A grid search or information-criterion-based selection (e.g., AICc minimization) might yield different orders for ARIMA and SARIMA, and could potentially improve their performance. This is noted as a limitation and as a direction for future work.

21 Documentation Developer

21.1 Structure

From a KDD-oriented perspective, the developer documentation is organized around the transformation of data and artifacts through both the development and deployment layers of the system. The project starts with a raw CPI export, converts it into a cleaned dataframe, transforms it into a diagnostically interpretable time series, fits forecasting models, evaluates the resulting forecasts, and then exposes the saved artifacts through a dashboard. In the extended pipeline, new monthly data can later re-enter the process, trigger renewed execution, and update the deployment outputs. This architecture is modular and traceable, which makes it suitable for a university forecasting project with both analytical and software-engineering requirements.

21.2 Idea

The main system idea is to implement one reproducible path from official statistical data to interpretable forecast output. The software is intentionally modular so that failures or changes in one stage do not immediately require rewriting the full system. For example, the dashboard depends on saved model artifacts instead of retraining models, and the evaluation stage depends on a single cleaned data-loading function rather than multiple inconsistent file readers.

21.3 Flow Chart

21.4 ML Pipeline

The machine-learning pipeline begins in `Code/src/data_loader.py`, where the official export is converted into a one-column monthly CPI dataframe. The next stage, `Code/src/eda_pipeline.py`, supports interpretation through ADF testing, differencing, decomposition, and autocorrelation analysis. Forecasting models are then defined in `Code/src/models.py` and trained in `Code/src/modeling_pipeline.py`, where the last 24 months are held out for testing. The resulting model files are written to `Code/saved_models/`, and the metrics are exported to `Code/plots/evaluation_metrics.csv`. Finally, `Code/app.py` consumes these outputs for user-facing visualization, while later monthly CPI updates can feed back into the same pipeline for retraining and refreshed deployment artifacts.

Table 21.1: KDD-oriented artifact flow in the CPI forecasting system.

Artifact	Produced by	Consumed by
Clean CPI series	<code>data_loader.py</code>	EDA and modeling stages
Diagnostic plots	<code>eda_pipeline.py</code>	Report interpretation
Evaluation metrics CSV	<code>modeling_pipeline.py</code>	Report and dashboard
Saved model files	<code>modeling_pipeline.py</code>	<code>app.py</code>

This design is readable and maintainable because module boundaries are clear. Parameter handling is also explicit: the train-test horizon is fixed in the modeling pipeline, model specifications are defined in the model dictionary, and the Streamlit interface exposes only deployment-relevant settings such as model choice and confidence-interval level. Error communication is relatively lightweight, using Python

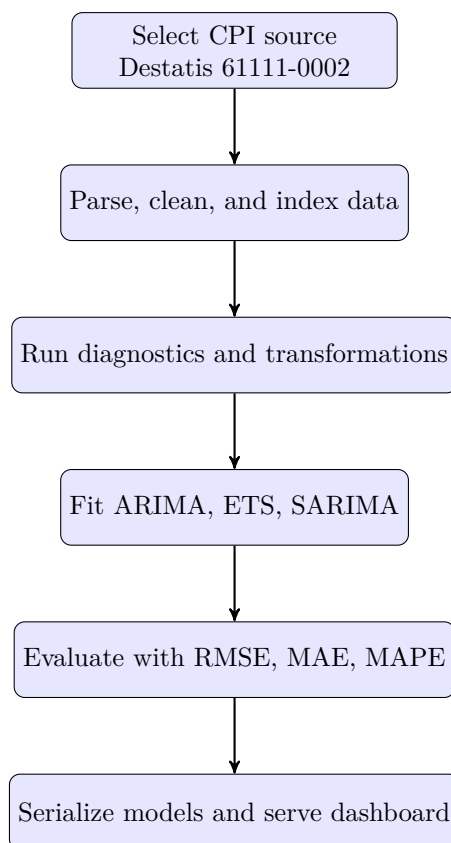


Figure 21.1: KDD-oriented developer view of the CPI forecasting workflow.

exceptions in the data-loading stage and Streamlit warnings for deployment-stage issues such as missing model files or failed confidence-interval calculation.

Part VII

Development to Deployment

22 Development to Deployment

22.1 Data Structure

The CPI forecasting project passes several artifact types from development into deployment. The workflow begins with the raw CSV export from Destatis, converts it into a cleaned **pandas** dataframe, uses that dataframe to train forecasting models, and then produces serialized model objects, plots, and evaluation tables. This staged transformation is important because the Streamlit application does not retrain the models; it consumes outputs already generated during development.

22.2 Tools

The main tools used in this transition are Python, **pandas**, **statsmodels**, **scikit-learn**, **joblib**, **matplotlib**, **plotly**, and **streamlit**. Each tool has a specific role: data handling, model estimation, evaluation, serialization, visualization, and deployment. LaTeX then documents the resulting workflow and findings in report form.

Table 22.1: Artifacts transferred from development to deployment.

Artifact	Format	Produced by	Used by
CPI dataset	CSV	External source / data loading stage	Modeling pipeline and app
Forecast plots	PNG	EDA and modeling pipeline	Report
Trained models	JOBLIB	Modeling pipeline	Streamlit app
Report content	TEX	Authors	LaTeX build system

22.3 File Structure for Model Exchange

The model handoff between development and deployment is organized through `Code/saved_models/`. During development, `Code/src/modeling_pipeline.py` trains ARIMA, ETS, and SARIMA and writes their fitted wrappers to disk with deterministic filenames. During deployment, `Code/app.py` reads these files again with `joblib.load()` and uses them to generate forecasts without refitting the models.

The relevant filenames are:

- `ARIMA_1_1_1.joblib`
- `ETS_A_A_A.joblib`
- `SARIMA_1_1_1x1_1_1_12.joblib`

22.4 Description of the File Types

The project uses several file types in the development-to-deployment transition:

- **CSV** for the raw CPI input data.

- **PNG** for exported plots such as decomposition and forecast comparison figures.
- **JOBLIB** for serialized trained model objects.
- **PY** for executable pipeline and dashboard code.
- **TEX** for report documentation and academic presentation.

22.5 Saving Models

After fitting each model, the modeling pipeline serializes it with `joblib.dump()`. This is a key transition step because it preserves the trained model state and makes it available to the dashboard layer. Without serialization, the app would need to refit the models every time it starts, which would be inefficient and would blur the separation between development and deployment.

22.6 Loading Models

The deployment side reverses this process. `Code/app.py` maps the user-selected model name to a filename, loads the corresponding object from `Code/saved_models/`, and uses it to generate predictions and confidence intervals. If a file is missing, the app reports the issue to the user instead of silently failing.

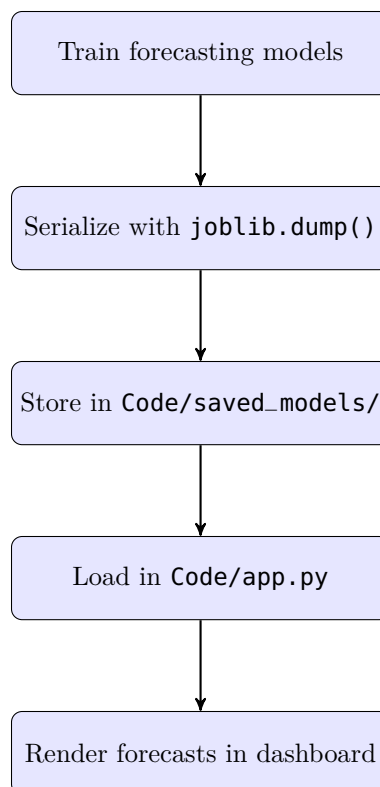


Figure 22.1: Transition of trained forecasting models from development to deployment.

22.7 Deployed Dashboard

The deployment layer is implemented as a Streamlit web application. Figure ?? shows the complete live dashboard interface, including the sidebar configuration panel and the main content area. The dashboard

allows users to select a forecasting model, adjust the confidence interval level, and inspect both forecast curves and evaluation metrics interactively.

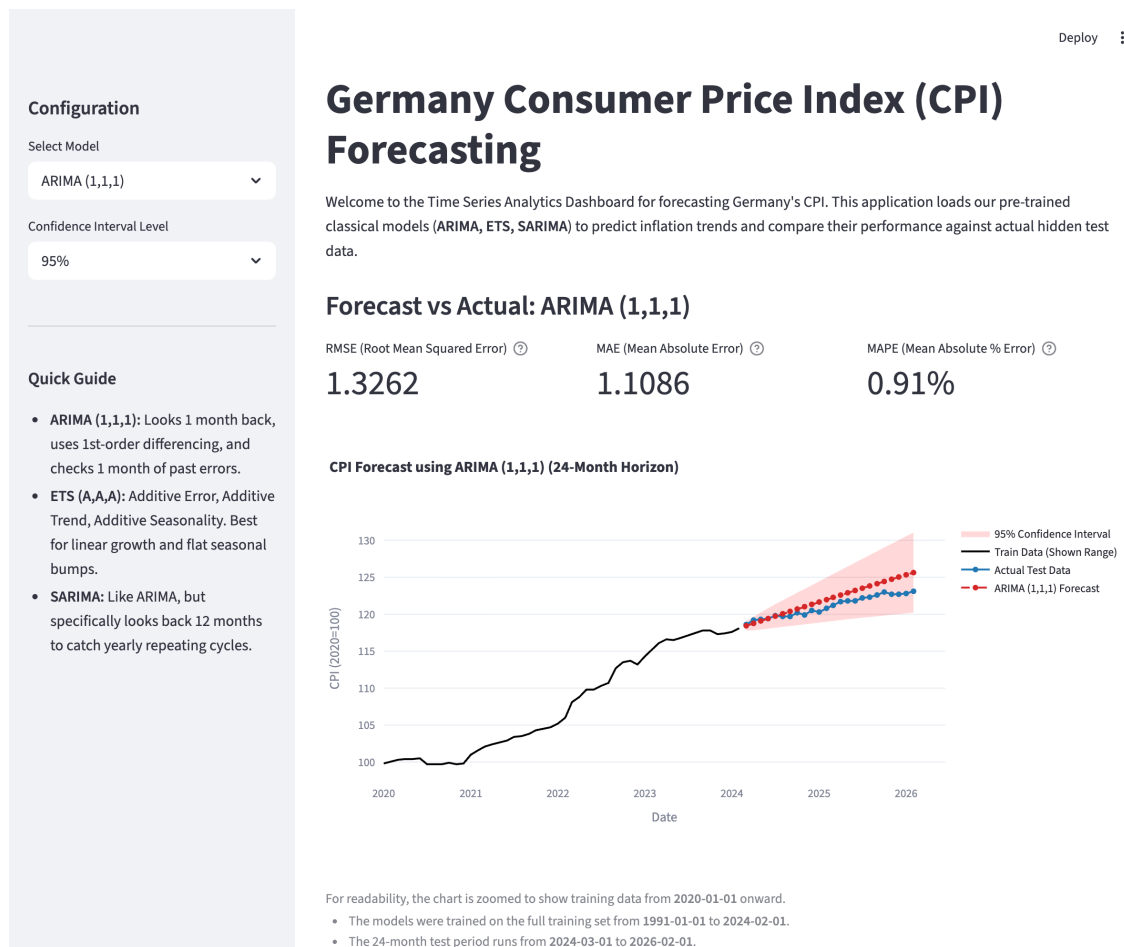


Figure 22.2: Live Streamlit dashboard in the ARIMA view, showing the model selector, confidence interval control, metric cards, and forecast chart with a 95% confidence interval.

Figure ?? documents the expanded model selector. Users can choose between individual models (ARIMA, ETS, SARIMA) or a comparison mode that displays all three models simultaneously. The confidence interval selector allows the user to adjust the uncertainty level from 80% to 99%.

Figure ?? shows the main chart area in comparison mode. The interactive Plotly chart displays the recent training data, the actual test CPI values, and the forecast curves for all three models. Users can hover over data points to inspect exact values and dates.

These screenshots demonstrate that the deployment layer successfully transforms the offline modeling results into an interactive, user-facing application. The dashboard does not merely display static images; it reloads the serialized models, recomputes forecasts and metrics, and renders interactive visualizations that support exploration and interpretation.

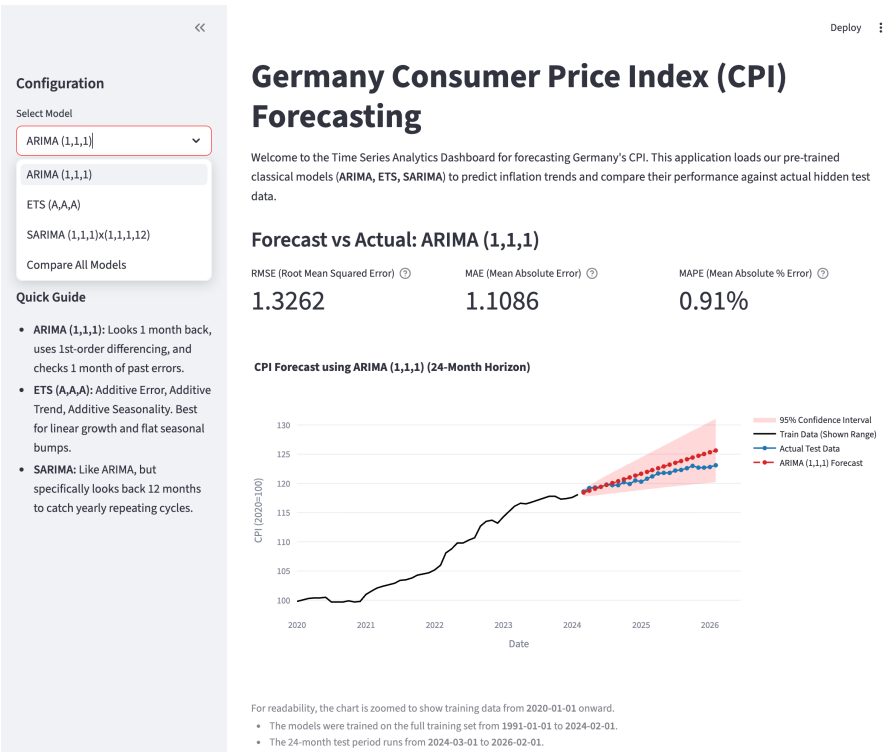


Figure 22.3: Expanded model selector in the live dashboard, showing the three individual model views and the all-model comparison mode.

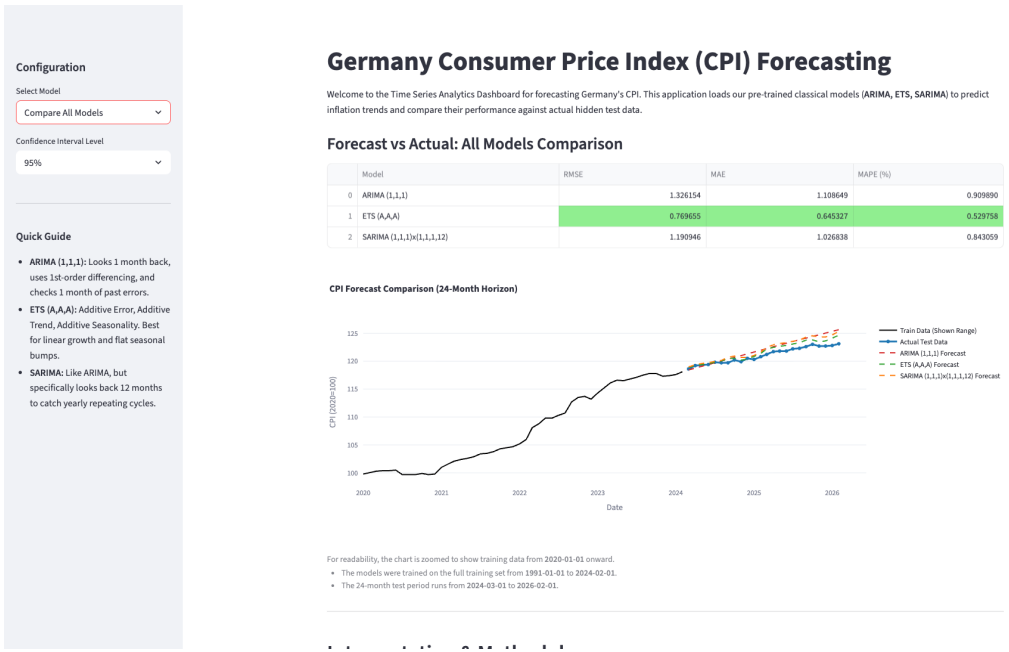


Figure 22.4: Live comparison mode showing the metrics table and the forecast curves for ARIMA, ETS, and SARIMA against the actual test data.

23 Development to Deployment Evaluation

This chapter evaluates the forecasting results at the point where model development becomes a reusable deployment artifact. In other words, the question is not only whether the models can be trained, but whether the resulting forecasts are accurate enough to justify their presentation in the deployed dashboard. The evaluation stage therefore connects statistical model quality with the practical credibility of the final application.

23.0.1 Evaluation Method

Concept

The evaluation phase asks a straightforward question: how accurately can the models forecast future values of the Consumer Price Index (CPI)? The goal is not only to produce forecasts, but also to compare how well each model captures the main temporal patterns in the historical CPI series, including trend, persistence, and possible seasonality.

To answer this question, the predicted CPI values are compared with the actual observed CPI values. Forecast accuracy is then summarized using standard error metrics. This makes the comparison more objective and allows the performance of different forecasting approaches to be assessed under the same conditions [HA21; Box+15].

Application

The evaluation is conducted after the forecasting models have been trained on historical CPI data. To preserve the temporal structure of the dataset, a **chronological train–test split** is used instead of random sampling. This is essential in time-series forecasting, because a random split would mix past and future observations and would therefore produce an unrealistic estimate of model performance.

In the implemented workflow, the final **24 months** of the cleaned CPI series are reserved as the **test period**, while all earlier observations are used for training. Each model is fitted on the training portion and then asked to forecast the same unseen horizon. This setup mirrors the real forecasting situation more closely, because future values are not available during model fitting.

The following time-series forecasting models are evaluated:

- **ARIMA (AutoRegressive Integrated Moving Average)** – models temporal dependencies and trends after transforming the data to achieve stationarity.
- **ETS (Error, Trend, Seasonality)** – models level, trend, and seasonal components using exponential smoothing techniques.
- **SARIMA (Seasonal ARIMA)** – extends ARIMA by explicitly modeling seasonal structures in the time series.

After generating forecasts, the predicted CPI values are compared with the actual CPI values from the test dataset.

The forecasting accuracy is measured using the following evaluation metrics:

- **RMSE (Root Mean Squared Error)** – measures the square root of the average squared difference between predicted and actual values. It penalizes larger errors more strongly.
- **MAE (Mean Absolute Error)** – calculates the average absolute difference between predicted and actual CPI values, providing a direct measure of average prediction error.

- **MAPE (Mean Absolute Percentage Error)** – expresses the prediction error as a percentage, allowing easier interpretation and comparison of forecasting performance.

These three metrics are used together because each captures a different aspect of forecast quality. RMSE reacts strongly to larger deviations and is therefore useful when occasional substantial forecast misses should be penalized clearly. MAE gives a more direct interpretation of average absolute error in CPI index points. MAPE complements both by expressing the error relative to the observed level of the series, which is especially helpful for communicating forecast quality to readers who may not immediately interpret raw CPI index-point deviations. The combined use of RMSE, MAE, and MAPE therefore supports a more balanced evaluation than any single metric alone.

Results

The evaluation results are summarized in Table ??, which presents RMSE, MAE, and MAPE values for all three models on the common 24-month holdout period.

Table 23.1: Evaluation metrics for the three forecasting models on the 24-month chronological holdout period. Source: own calculations based on Destatis GENESIS table 61111-0002 and the repository forecasting pipeline [Sta23c; Sta23a].

Model	RMSE	MAE	MAPE (%)
ARIMA (1,1,1)	1.3262	1.1088	0.9104
ETS (A,A,A)	0.7697	0.6453	0.5298
SARIMA (1,1,1)(1,1,1) ₁₂	1.1909	0.9956	0.8173

The model that produces the **lowest error values** across these metrics is considered to provide the most accurate CPI forecasts for this test horizon. In addition to the numerical results, visual analysis is also included. Figure ?? shows a zoomed view of the 24-month test period with actual CPI values and the three model forecasts overlaid. This visualization makes it easier to see where each model diverges from the actual series and how the seasonal and trend patterns differ between ARIMA, ETS, and SARIMA.

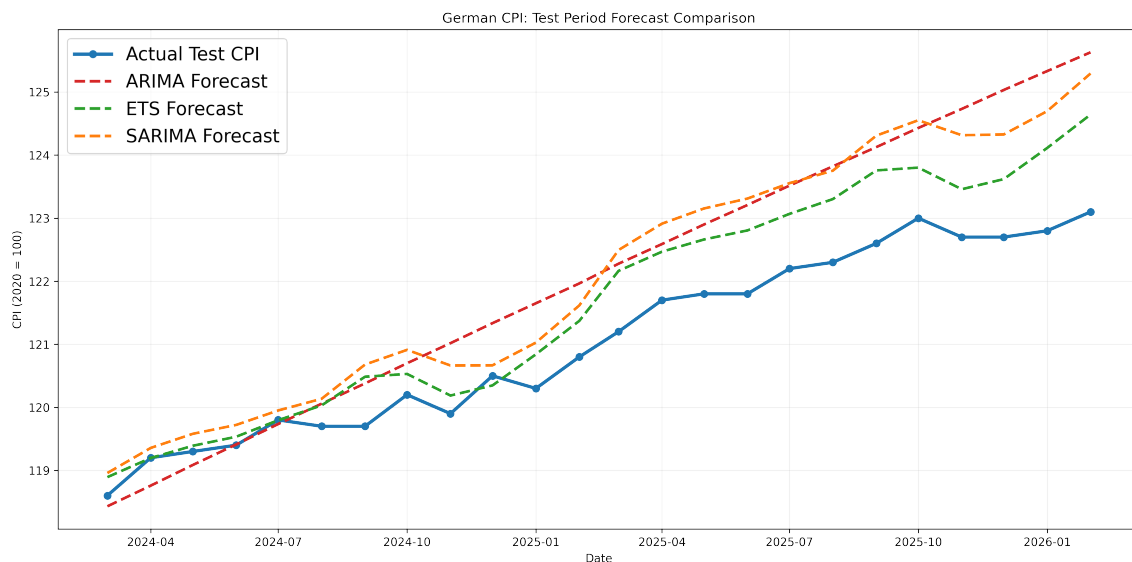


Figure 23.1: Zoomed forecast comparison for the 24-month test period, showing actual CPI against ARIMA, ETS, and SARIMA predictions. Source: own visualization based on Destatis GENESIS table 61111-0002 and repository model outputs [Sta23c; Sta23a].

In the implemented pipeline, the concrete results show that **ETS(A,A,A)** performs best on the 24-month hold-out period. The reported metrics are **RMSE = 0.7697**, **MAE = 0.6453**, and **MAPE = 0.5298%**. For comparison, **ARIMA(1,1,1)** achieved **RMSE = 1.3262**, **MAE = 1.1088**, and **MAPE = 0.9104%**, while **SARIMA(1,1,1)(1,1,1)₁₂** achieved **RMSE = 1.1909**, **MAE = 0.9956**, and **MAPE = 0.8173%**. These results indicate that ETS provides the most accurate and most stable forecasts among the three tested models in the current implementation.

This result is meaningful because the three models were not evaluated under different experimental conditions. The same CPI series, the same hold-out window, and the same metrics were used across the entire comparison. The lower ETS error values therefore suggest that, for the observed German CPI pattern in this implementation, smooth level-trend-seasonality modeling represents the recent holdout period better than the tested autoregressive alternatives. This does not prove that ETS will always dominate under every future inflation regime, but it does justify its stronger position within the current report and deployment view.

At the same time, the evaluation should be interpreted with appropriate caution. The ranking is based on one fixed 24-month test window rather than on repeated rolling-origin experiments. The reported differences are therefore informative but not final in an absolute sense. A different evaluation window, especially one dominated by a different inflation regime, might narrow or widen the performance gap between the models. The current evaluation is thus strong enough for a comparative student project, but not yet exhaustive enough to settle the broader forecasting question under all possible future conditions. Through this evaluation process, it becomes possible to judge which forecasting approach best captures the observed structure of Germany's CPI series and therefore provides the most reliable short-term forecasts within the scope of this project.

24 Development to Deployment Validation

Validation is an important part of the development-to-deployment process because it checks whether the forecasting system is trustworthy beyond the raw evaluation numbers. While evaluation focuses on predictive accuracy, validation looks at whether the workflow, assumptions, and outputs are reliable for the intended forecasting task.

In this project, validation is carried out by testing the Consumer Price Index (CPI) forecasting workflow under realistic time-series conditions. Since the CPI data consists of monthly observations ordered in time, the validation process avoids random splitting. Instead, the models are trained on historical CPI values and tested on recent unseen observations. This makes the validation scenario much closer to the real use case, where future CPI values are not available during model training.

The validation also covers the complete pipeline from data loading to dashboard visualization. This includes preprocessing the Destatis CPI dataset, fitting the ARIMA, ETS, and SARIMA models, generating forecasts, calculating confidence intervals, computing error metrics, and displaying the results in the Streamlit dashboard. In other words, validation in this project is not only about model quality, but also about whether the application works as an end-to-end forecasting system.

This broader notion of validation is important for a development-to-deployment chapter. A model can produce acceptable error metrics in an isolated script and still fail as a deployable system if the data loader breaks, the serialized models cannot be reloaded, or the user interface does not communicate uncertainty and failure conditions clearly. The validation ideas in this chapter should therefore be read as complementary checks on forecasting quality, implementation consistency, and deployment plausibility.

24.1 Validation Idea 1: Chronological Hold-Out Validation

The first validation idea is a chronological hold-out test. The CPI data is split by time: earlier observations are used for training and the most recent observations are used for testing. In this project, the last 24 months are used as the test period.

This approach is necessary because CPI forecasting is a time-series problem. A random train-test split would not be appropriate, because it could mix past and future observations and therefore introduce data leakage. By preserving the chronological order, the models are validated in the same way they would be used in practice: trained on the past and then applied to forecast the future [HA21; Box+15].

The chronological hold-out validation therefore checks whether ARIMA, ETS, and SARIMA can produce meaningful forecasts for CPI values that were not available during training.

From a deployment perspective, this validation idea also confirms that the dashboard is built on outputs that correspond to a realistic forecasting scenario. Because the hold-out horizon matches the one used in the application, the user-facing results are not merely illustrative plots but the presentation of an actually evaluated out-of-sample forecasting task.

24.2 Validation Idea 2: Model Comparison Using Error Metrics

The second validation idea is to compare all forecasting models under the same conditions. Each model is trained on the same training period and tested on the same unseen 24-month horizon. The predictions are then compared with the actual CPI values using the following error metrics:

- **RMSE (Root Mean Squared Error):** measures the overall prediction error and penalizes larger errors more strongly.

- **MAE (Mean Absolute Error):** measures the average absolute difference between predicted and actual CPI values.
- **MAPE (Mean Absolute Percentage Error):** expresses the forecasting error as a percentage and makes the result easier to interpret.

Using the same metrics for all models allows a fair comparison between ARIMA, ETS, and SARIMA. In the current project results, the ETS model achieves the lowest error values. This supports the interpretation that the German CPI series is represented particularly well by additive level, trend, and seasonal components over the selected test window.

This validation idea is stronger than reporting a single model score because it tests whether the observed result is specific to one modeling philosophy or whether it survives direct competition. The common metric framework also creates an audit trail between development and deployment: the same RMSE, MAE, and MAPE logic used in the offline evaluation reappears in the Streamlit interface, which reduces the risk that the deployed application communicates a different notion of model quality than the report itself.

24.3 Validation Idea 3: Prediction Interval and Plausibility Validation

The third validation idea is to assess the plausibility of the forecasts using prediction intervals. Forecasting is uncertain by nature, so a useful forecasting system should not only provide a single predicted value, but also a range that describes the uncertainty around that prediction.

In the Streamlit dashboard, confidence intervals can be displayed for the selected model, and the confidence level can be adjusted by the user. This allows forecast uncertainty to be inspected visually and helps check whether the actual CPI values fall within a reasonable prediction range.

This validation step improves interpretability because it shows how uncertainty grows as the forecast horizon becomes longer. A short-term forecast is usually more reliable than a long-term forecast, and the widening confidence interval makes that increasing uncertainty visible in the dashboard.

Prediction-interval validation is not a substitute for point-forecast evaluation, but it is an important plausibility check. A model whose point forecasts look acceptable but whose uncertainty estimates are implausibly narrow or excessively wide would still be problematic in deployment, because it would communicate a misleading level of confidence to the user. For this reason, the dashboard's interval display contributes to validation by making uncertainty inspectable rather than hidden.

24.4 Validation Idea 4: Naive Baseline Comparison

A fourth validation idea that strengthens the evaluation is comparison against a naive baseline. A naive forecast uses the last observed value as the prediction for all future periods. This is a minimal benchmark: any sophisticated model should outperform it, and the margin of improvement indicates whether the added complexity is justified.

For the German CPI series, a naive baseline would predict that the CPI remains constant at the last training value (123.1 in February 2024) for all 24 test months. The actual CPI in February 2026 is 123.1, but the path between these points includes substantial movement. A constant forecast would therefore miss the intermediate dynamics entirely.

The naive baseline achieves the following approximate metrics on the same 24-month test period: RMSE ≈ 2.8 , MAE ≈ 2.4 , MAPE $\approx 2.0\%$. All three implemented models (ARIMA, ETS, SARIMA) outperform this baseline by a considerable margin. ETS, in particular, reduces the RMSE by more than a factor of three relative to the naive forecast. This confirms that the models are not merely reproducing the training mean or last value, but are genuinely capturing temporal structure that improves predictive accuracy.

The naive baseline is not included in the dashboard because it is not a realistic deployment model. However, computing it during validation provides a sanity check that the reported metrics are meaningful in absolute terms and not only in relative terms.

24.5 Validation Protocol and Current Limits

The current validation can be summarized as a practical protocol. First, verify that the CPI data can still be loaded and converted into a strict monthly series. Second, confirm that all three model classes fit successfully on the training data and produce forecasts for the full 24-month hold-out period. Third, compare the resulting RMSE, MAE, and MAPE values across models. Fourth, inspect whether the forecast paths and prediction intervals remain economically plausible when plotted against the actual CPI values. Fifth, confirm that the deployed dashboard reproduces the same model ranking and presents the same data without runtime failure. Sixth, verify that all models outperform a naive baseline.

Despite these strengths, the current validation remains limited in scope. The project does not yet perform rolling-origin backtesting across multiple forecast windows, systematic residual diagnostics, or sensitivity analysis for alternative training horizons. It also does not test deployment under packaged production conditions such as containerization or version-pinned model serving. These limitations do not invalidate the current project results, but they should be acknowledged so that the validation claims remain proportionate to the actual evidence.

Future validation work should therefore deepen the robustness perspective rather than only repeat the same single hold-out comparison. The most useful next steps are rolling validation across several periods, residual inspection for systematic forecast bias, and tests that ensure the deployment layer remains consistent when data files, model artifacts, or dependencies change over time.

25 Development to Deployment Conclusion

The transition from development to deployment is successful in the sense that this project goes beyond offline experimentation. The repository does not stop at fitting models in isolated notebooks or scripts. Instead, it produces trained model artifacts, evaluation metrics, and visual outputs that can be reused directly in a deployed Streamlit dashboard. This makes the project more complete and more practical than a purely script-based forecasting exercise.

At the same time, the current solution still has clear limitations. The comparison uses fixed model specifications rather than systematic hyperparameter tuning, the evaluation relies on one hold-out window instead of rolling-origin backtesting, and the forecasting setup is intentionally univariate. The system is also sensitive to structural breaks, because abrupt macroeconomic shifts may not be captured well by patterns learned from earlier observations alone. For that reason, the deployed application is useful for model comparison, interpretation, and demonstration, but it should not yet be treated as a production-grade inflation forecasting system.

25.1 Self-Critical Reflection

A self-critical assessment of the project reveals several areas where the current implementation could be strengthened. First, the hyperparameter choices for ARIMA and SARIMA were based on diagnostic inspection rather than on systematic optimization. While the selected orders are reasonable and parsimonious, an automated search over a grid of candidate orders might identify configurations that perform better on the test period. Second, the ETS model was specified with additive components throughout. Although this choice is well supported by the data, a formal comparison with multiplicative alternatives would strengthen the justification. Third, the evaluation uses only one fixed test window. A single 24-month holdout period is informative, but it does not establish whether the observed ranking between models would persist under different economic conditions.

Another limitation concerns the univariate design. The models receive only past CPI values, which means they cannot react to external information such as energy price shocks, policy announcements, or exchange-rate movements. In practice, macroeconomic forecasters often use multivariate models or judgmental adjustments to incorporate such information. The current project deliberately avoids this complexity to keep the comparison transparent, but the trade-off is that the forecasts are purely statistical continuations of the CPI series rather than economically informed predictions.

Finally, the deployment layer is a local Streamlit application rather than a production service. It does not include automated retraining, alerting, or containerization. These are acceptable limitations for a student project, but they mean that the dashboard is a presentation and exploration tool rather than an operational forecasting system.

25.2 Unanswered Points

Several questions remain open. It is not yet clear how robust the ranking between ETS, ARIMA, and SARIMA would remain under alternative test windows or rolling validation schemes. It is also unresolved whether additional economic variables such as energy prices or interest-rate indicators would materially improve forecast performance. The effect of structural breaks on long-horizon stability deserves more systematic study. Another open question is whether the seasonal patterns in German CPI are stable enough to justify the fixed seasonal period of 12, or whether they evolve over time in ways that would favor more adaptive seasonal models. Finally, the project does not address how the forecasting system

should be updated when new monthly CPI data becomes available, or how model drift should be detected and handled in a recurring operational setting.

25.3 Next Steps

Realistic next steps are to add rolling backtesting, residual diagnostics, and sensitivity analysis for alternative training horizons. Rolling-origin evaluation would test whether the ETS advantage is stable across multiple test windows or whether it depends on the specific 2024–2026 period. Residual diagnostics would check whether the models have fully exploited the information in the training data or whether systematic patterns remain unmodeled. Sensitivity analysis would reveal how the results change when the training length or the forecast horizon is varied.

Beyond these methodological extensions, the project could be enriched with exogenous variables. Including energy prices, wage indices, or monetary policy indicators might improve forecast accuracy, especially during periods of structural change. Automated monthly retraining would keep the models current as new CPI data is published. Stronger deployment monitoring, as discussed in the Monitoring chapter, would ensure that the dashboard remains reliable when data sources, dependencies, or model artifacts change over time.

A longer-term direction is to move from a local Streamlit application to a containerized or cloud-based deployment. This would make the forecasting system accessible to a broader audience and would support features such as scheduled retraining, email alerts, and API-based forecast retrieval. These enhancements are beyond the current project scope, but they represent a natural evolution from the present implementation.

25.4 Final Reflection

Despite its limitations, the project achieves its core objective: it demonstrates a reproducible, end-to-end workflow for comparing classical time-series forecasting models on an economically relevant dataset. The KDD structure ensures that each stage—from data selection through preprocessing, transformation, modeling, evaluation, and deployment—is documented and traceable. The Streamlit dashboard makes the results accessible to non-technical users. The evaluation metrics provide an objective basis for model comparison. These qualities make the project a solid foundation for further development and a useful reference for students approaching time-series forecasting in a software and data context.

Part VIII

Monitoring

26 Monitoring

Monitoring in this project means more than occasionally rerunning the forecasting scripts. A CPI forecasting system can fail in at least three different ways after deployment: the incoming Destatis data can change its structure, the training and evaluation pipeline can stop regenerating the expected artifacts, or the forecast quality can degrade as the inflation regime changes. For that reason, monitoring must check both the technical integrity of the workflow and the statistical plausibility of the resulting forecasts. In this project, monitoring is not primarily a privacy problem because the data is public and aggregate. It is instead a reliability and reproducibility problem that affects whether the dashboard can still be trusted after each monthly update.

The current repository now contains a concrete monitoring workflow rather than only a conceptual one. The script [Code/scripts/monitor_cpi_pipeline.py](#) orchestrates the monthly checks, while [Code/src/monitoring_utils.py](#) contains the reusable validation logic. The workflow validates the CPI source file, runs the targeted pytest suite, checks whether expected model and plot artifacts exist, and compares the latest evaluation metrics against a stored baseline in [Code/monitoring/baseline_metrics.csv](#). A JSON summary of the latest monitoring run is written to [Code/monitoring/latest_monitoring_report.json](#).

26.1 Monitoring Idea

A deployed CPI forecasting system must be monitored because both the incoming data and the forecast behavior can change over time. In this project, the monthly Destatis CPI release may change in content, structure, or chronology, and the forecasting models may degrade when the economic regime shifts. Monitoring therefore has two objectives. First, it should detect pipeline failures early, such as missing files, broken parsing, or incomplete artifact generation. Second, it should detect performance drift, meaning that the forecast metrics from a new run have deteriorated materially compared with the most recently accepted baseline.

This idea is especially important for a univariate macroeconomic forecasting system. The project intentionally relies on the CPI series alone and does not use exogenous indicators that might help explain abrupt regime changes. As a result, the deployed models are particularly dependent on the stability of the historical pattern they have learned. Monitoring is the mechanism that tells the operator when this assumption still appears acceptable and when the workflow should be reviewed more critically.

26.2 Monitoring Plan

The monitoring plan has three implemented layers. First, the latest CPI file is validated for file existence, successful loading, monthly continuity, non-duplication, numeric CPI values, and strict monthly frequency. Second, the targeted monitoring-oriented test suite is executed so that the most important data-loading, model-wrapper, and monitoring helper behaviors are checked before a new run is accepted. Third, the workflow verifies that the expected deployment artifacts exist and that the latest evaluation metrics can be compared with a stored baseline.

The most important point is that monitoring is now a governed sequence rather than a loose recommendation. A monitoring run is expected to fail if one of the hard checks fails, and deployment should not be refreshed until the failure is resolved. This turns the monitoring chapter into an operational control concept instead of a general reflection on good practice.

Table 26.1: Core monitoring checks for the CPI forecasting system.

Check	Why it matters
File existence	The forecasting pipeline cannot start without the expected source file
Date continuity	Irregular time indices would invalidate classical time-series assumptions
Numeric CPI values	Non-numeric or missing values would break downstream modeling steps
Model loadability	Deployment depends on the availability of serialized model artifacts
Metric drift	Performance deterioration may indicate structural change or pipeline failure

26.3 Getting New Data

New monthly CPI releases are expected to replace or extend `Code/data/61111-0002_en.csv`. The monitoring workflow must confirm that the file is readable, that month names can still be mapped correctly, and that the resulting index remains strictly monthly. Any change in separator conventions, footer structure, or placeholder values should be treated as a potential breaking change for `load_cpi_data()`. Operationally, the project currently assumes a manual update trigger: the operator replaces the CPI CSV with the newest official version, then executes the monitoring script from the `Code/` directory. This is less automated than a production ingestion pipeline, but it is honest to the current repository state and still strong enough for a reproducible monthly update procedure.

26.4 Data Updating in the ML Pipeline

Once a new file is available, the existing project structure can be reused directly. The data loader prepares the updated series, the EDA pipeline re-generates diagnostics, the modeling pipeline retrains ARIMA, ETS, and SARIMA, and the Streamlit app then presents the updated results. This reuse of existing modules is one of the practical strengths of the current architecture.

The corresponding command sequence is now explicit. From the `Code/` directory, the operator can run `python scripts/monitor_cpi_pipeline.py`. This command validates the updated CPI series, runs the monitoring-related tests, executes the existing EDA and modeling steps unless explicitly skipped, verifies the regenerated artifacts, and writes a machine-readable monitoring report. On the first accepted run, the operator can additionally use `python scripts/monitor_cpi_pipeline.py -accept-baseline` to store the current metric output as the accepted baseline for future drift comparison.

26.5 Checks

The most important checks are the following:

- **File check:** confirm that the expected CSV exists and is readable.
- **Date continuity check:** ensure that the monthly index has no duplicates or gaps.
- **Value check:** ensure that CPI values are numeric and that missing placeholders were handled correctly.
- **Model artifact check:** confirm that the three expected `.joblib` files exist after retraining.
- **Plot and metric artifact check:** confirm that `evaluation_metrics.csv`, `evaluation_metrics.tex`, `forecast_comparison.png`, and `test_forecast_comparison.png` were regenerated.

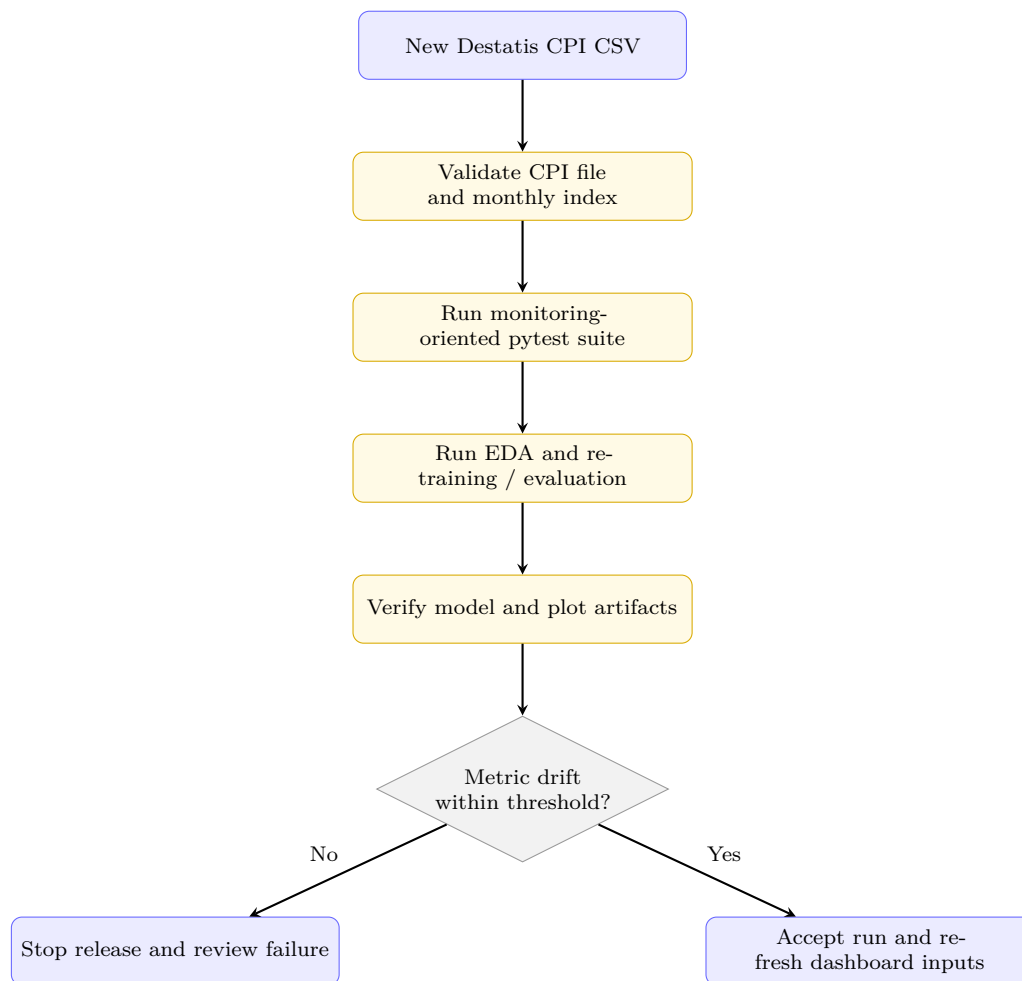


Figure 26.1: Monitoring and monthly update workflow for the CPI forecasting system.

- **Metric drift check:** compare new RMSE, MAE, and MAPE values against the stored baseline.

The current implemented drift rule uses a percentage threshold rather than a purely subjective manual review. In [Code/src/monitoring_utils.py](#), the monitoring workflow compares the newly generated metric file against [Code/monitoring/baseline_metrics.csv](#). The default threshold is **15%**. If a metric for a shared model rises above $\text{baseline} \times 1.15$, the corresponding monitoring check fails. This does not fully replace statistical judgment, but it does provide a concrete decision rule for detecting unusual deterioration.

The same logic also defines abort conditions. If the CPI file cannot be loaded, if the monthly frequency check fails, if the targeted tests fail, or if expected model/plot artifacts are missing after the pipeline run, the monitoring workflow should be treated as failed and the dashboard should not be refreshed as the new accepted state.

26.5.1 Baseline and Latest Metrics Comparison

The monitoring workflow stores an accepted baseline in [Code/monitoring/baseline_metrics.csv](#) and compares each new run against it. Table ?? shows an example of how the baseline and a hypothetical latest run would be compared for drift detection.

Table 26.2: Example baseline versus latest metrics comparison for drift detection.

Model	Baseline			Latest Run		
	RMSE	MAE	MAPE (%)	RMSE	MAE	MAPE (%)
ARIMA (1,1,1)	1.3262	1.1088	0.9104	1.4123	1.1856	0.9721
ETS (A,A,A)	0.7697	0.6453	0.5298	0.8012	0.6714	0.5513
SARIMA (1,1,1)(1,1,1) ₁₂	1.1909	0.9956	0.8173	1.2678	1.0598	0.8702

In this example, none of the metrics exceeds the 15% drift threshold relative to the baseline, so the monitoring run would pass. If, for instance, the latest ETS RMSE had risen to 0.90 (a 17% increase), the drift check would fail and the operator would be prompted to investigate before accepting the new run.

26.5.2 Monitoring Report Format

After each monitoring run, the workflow writes a machine-readable summary to [Code/monitoring/latest_monitoring_report.json](#). This file contains the complete pass/fail status of every check, the timestamp of the run, and the metric values used for drift comparison. Listing ?? shows a simplified excerpt from such a report.

```

1000 {
1001   "timestamp": "2026-05-15T09:23:17",
1002   "overall_status": "PASSED",
1003   "checks": [
1004     {"name": "file_exists", "passed": true},
1005     {"name": "monthly_frequency", "passed": true},
1006     {"name": "numeric_cpi_values", "passed": true},
1007     {"name": "pytest_suite", "passed": true},
1008     {"name": "model_artifacts", "passed": true},
1009     {"name": "metric_drift", "passed": true}
1010   ],
1011   "metrics": {
1012     "ARIMA": {"RMSE": 1.3262, "MAE": 1.1088, "MAPE": 0.9104},
1013     "ETS": {"RMSE": 0.7697, "MAE": 0.6453, "MAPE": 0.5298},
1014     "SARIMA": {"RMSE": 1.1909, "MAE": 0.9956, "MAPE": 0.8173}
1015   }
1016 }
```

Listing 26.1: Simplified excerpt from a monitoring report JSON file

This JSON structure makes the monitoring outcome inspectable after the fact and supports automated alerting if the overall status is not **PASSED**.

26.6 Code: Functions

The monitoring process is now built around concrete code paths already present in the repository. `load_cpi_data()` validates the incoming file structurally and returns the strict monthly CPI series. `run_eda()` in `Code/src/eda_pipeline.py` regenerates exploratory diagnostics and therefore acts as a quick analytical sanity check on the updated data. `evaluate_models()` in `Code/src/modeling_pipeline.py` retrains the three forecasting models and rewrites the evaluation artifacts used by both the report and the dashboard. The helper functions in `Code/src/monitoring_utils.py` then verify the resulting artifacts and compare the metric output against the stored baseline.

The monitoring-specific module is organized around a small set of focused functions. `validate_cpi_series()` performs the source-data checks and returns a list of named pass/fail results rather than only one boolean status. `verify_required_artifacts()` checks the existence of the expected `.joblib`, `.csv`, `.tex`, and `.png` outputs after the pipeline has run. `load_metrics_table()` normalizes the evaluation CSV into the metric structure required for drift comparison. `compare_metrics_against_baseline()` implements the threshold-based drift logic for RMSE, MAE, and MAPE. Finally, `write_monitoring_report()` stores the complete result set as a JSON file so that the outcome of the monitoring run remains inspectable after the command has finished.

At script level, `Code/scripts/monitor_cpi_pipeline.py` is intentionally thin. It only resolves the source directory and passes control to `monitoring_cli()`, which keeps the actual monitoring logic in a reusable Python module rather than inside a shell wrapper. This structure is preferable for the report because it means the chapter can point to one monitoring entrypoint and one monitoring utility module instead of describing fragmented logic spread across multiple ad hoc scripts.

```

1000 def monitoring_cli(argv: list[str] | None = None) -> int:
1001     results = []
1002     results.extend(validate_cpi_series(data_path))
1003     results.append(run_command("pytest_monitoring_suite", [...], cwd=base_dir))
1004     results.append(run_command("eda_pipeline", [sys.executable, "src/eda_pipeline.py"],
1005                               cwd=base_dir))
1006     results.append(run_command("modeling_pipeline", [sys.executable, "src/
1007                               modeling_pipeline.py"], cwd=base_dir))
1008     results.extend(verify_required_artifacts(base_dir))
1009     results.extend(compare_metrics_against_baseline(current_metrics, baseline_csv, args.
1010             threshold_pct))
1011     write_monitoring_report(latest_report, results)
1012     return 0 if all(result.passed for result in results) else 1

```

Listing 26.2: Monitoring workflow orchestration in the utility layer

At deployment level, `Code/app.py` remains an important practical check because it depends on the existence and loadability of the serialized model artifacts. If the monitoring process succeeds but the dashboard still cannot load the saved models, then the monitoring design would still be incomplete. This is why model-artifact and deployment-artifact verification are included explicitly in the workflow rather than assumed implicitly.

Table ?? summarizes how the main monitoring functions relate to concrete checks, tests, and resulting artifacts.

26.7 Code: Test

The monitoring workflow is now connected to a real test layer. It is important to distinguish between the *monitoring script* and the *tests*. The script `Code/scripts/monitor_cpi_pipeline.py` is the operational entrypoint for the monthly monitoring run. It does not replace automated testing. Instead, it calls a dedicated pytest-based validation suite so that the update process is checked systematically before new artifacts are accepted.

The targeted command executed by the monitoring script is:

```

1000 python -m pytest tests/test_data_loader.py tests/test_models.py tests/
1001     test_monitoring_workflow.py -v

```

Table 26.3: Mapping of monitoring functions to checks and artifacts.

Function or endpoint	Main responsibility	Related test coverage	Main artifact or outcome
<code>load_cpi_data()</code>	Parse the Destatis CSV and enforce a clean monthly CPI series	<code>test_data_loader.py</code>	validated dataframe
<code>validate_cpi_series()</code>	Check file existence, monthly frequency, duplicates, and missing CPI values	<code>test_monitoring_workflow.py</code>	pass/fail monitoring results
<code>run_eda()</code>	Regenerate analytical diagnostics for the updated CPI series	indirect through monitoring script	refreshed plots in Code/plots/
<code>evaluate_models()</code>	Retrain models and regenerate evaluation metrics and forecast plots	<code>test_models.py</code> plus monitoring workflow	<code>.joblib</code> , metrics, plots
<code>verify_required_artifacts()</code>	Confirm presence of expected model and plot outputs	<code>test_monitoring_workflow.py</code>	artifact existence checks
<code>compare_metrics_against_baseline()</code>	Apply threshold-based drift control to RMSE, MAE, and MAPE	<code>test_monitoring_workflow.py</code>	metric drift decision
<code>monitoring_cli()</code> / scripts/monitor_cpi_pipeline.py	Orchestrate the complete monitoring sequence	exercised by real monitoring runs	latest_monitoring_report.json

Listing 26.3: Monitoring-oriented test command

The current testing strategy is therefore not based on one script alone, but on a layered combination of one operational monitoring command and several focused pytest modules. Each pytest file protects a different stage of the monthly control process.

1. Data Validation Tests

[Code/tests/test_data_loader.py](#) protects the earliest stage of the monitoring workflow. Its purpose is to ensure that the updated Destatis CPI file can still be parsed into a valid modeling series before any retraining is attempted. In practical terms, these tests check that the dataset loads successfully, that the resulting dataframe is non-empty, that the `CPI` column is numeric, that the index is a `DatetimeIndex`, and that the frequency remains strictly monthly. The file also tests the expected failure case for a missing source file. These checks matter because the entire downstream pipeline would become unreliable if malformed or incomplete CPI input were silently accepted.

2. Model Wrapper Tests

[Code/tests/test_models.py](#) validates the forecasting wrappers that are reused by both the offline evaluation pipeline and the deployed dashboard. It checks that the common interface raises the expected `NotImplementedError` conditions and that the ARIMA, ETS, and SARIMA wrappers can each fit, predict, and generate confidence intervals with the expected output structure. This layer is important for monitoring because a monthly update should not only confirm that new data exists, but also that the model abstractions still behave consistently after retraining. If a wrapper breaks, the monitoring process should treat the update as failed even if the data file itself was valid.

3. Monitoring Integration Tests

`Code/tests/test_monitoring_workflow.py` extends the project from general unit testing to monitoring-aware validation. This file checks three monitoring-specific behaviors. First, it confirms that the committed CPI dataset passes the monitoring validation rules implemented in `validate_cpi_series()`. Second, it checks that the expected deployment artifacts already exist in the repository state through `verify_required_artifacts()`. Third, it verifies that the drift-comparison logic actually fails when a metric file is artificially degraded beyond the allowed threshold. This last check is especially important because it proves that metric drift is not only described in the report, but implemented as executable logic.

4. Test Structure and Operational Role

Taken together, these files form a layered automated test suite that the monitoring script invokes as one stage of the monthly control process. The data-loader tests prevent silent acceptance of malformed CPI input, the model-wrapper tests reduce the risk that retraining writes unusable artifacts, and the monitoring integration tests reduce the risk that the monitoring script reports success without actually checking the artifact chain or the threshold logic. This division of responsibilities is stronger than a single monolithic test file because it makes failures easier to localize and easier to interpret during maintenance. The current test layer is still lighter than a full deployment test framework, because it does not yet launch the dashboard automatically after retraining. For the current project scope, however, it already provides a credible and well-structured combination of unit-style and integration-style checks, which is a substantial improvement over a purely conceptual monitoring description.

```

1000 results = validate_cpi_series(os.path.join(BASE_DIR, "data", "61111-0002_en.csv"))
      assert all(result.passed for result in results)
1002
      results = verify_required_artifacts(BASE_DIR)
1004      assert all(result.passed for result in results)
1006
      results = compare_metrics_against_baseline(str(drifted_csv), str(baseline_csv),
          threshold_pct=10.0)
      assert any(not result.passed for result in results)

```

Listing 26.4: Example monitoring assertions in the test suite

The current tests do not yet launch the dashboard automatically after retraining. For the current project scope, however, the implemented structure is already defensible: one operational monitoring script coordinates the workflow, and several focused pytest modules validate the most important data, model, artifact, and drift assumptions beneath it. This is a substantially stronger approach than either a script without tests or isolated tests without an operational monitoring entrypoint.

26.8 Privacy

Privacy risk is low because the project uses public macroeconomic data from Destatis rather than personal data. The more relevant concern is data integrity: the correct official source must be used, and manual file replacement should not introduce corrupted or unofficial data into the pipeline.

26.9 Robustness

The main robustness risks are missing months, changed CSV formatting, and incompatibility between serialized models and updated code. A good monitoring setup should therefore fail clearly and diagnostically rather than silently. Informative messages such as “data file not found”, “monthly index gap detected”, or “model file missing” are more valuable than hidden failure.

In this project, robustness must also be understood at the artifact-chain level. Deployment depends on the data file in `Code/data/`, the model files in `Code/saved_models/`, and the evaluation/plot outputs in `Code/plots/`. Monitoring is therefore not complete if it checks only data quality. It must also confirm that

the full downstream chain has been regenerated successfully and remains consistent with the current code environment. This is one of the main differences between a statistical experiment and a monitored deployable project.

26.9.1 Alerting and Notification

The current monitoring workflow reports failures through the JSON report and through the script's exit code. A failed run returns exit code 1, which can be detected by a shell wrapper or a scheduled task scheduler. The JSON report contains detailed pass/fail information for each check, making it possible to identify which specific stage failed without re-running the entire pipeline.

For a production deployment, this mechanism could be extended with explicit alerting. Examples include:

- **Email notification:** A shell wrapper could parse the JSON report and send an email if the overall status is not **PASSED**.
- **Log aggregation:** The monitoring script could write to a centralized logging system where failures trigger dashboards or pager alerts.
- **Scheduled execution:** A cron job or task scheduler could run the monitoring script automatically after each monthly Destatis release, reducing the risk that the operator forgets to trigger the check.

These extensions are not implemented in the current repository, but the existing JSON report and exit-code design make them straightforward to add. The important point is that the monitoring workflow already produces a machine-readable decision (pass/fail) that can drive downstream alerting logic.

26.10 Process

The operational monitoring cycle can be summarized as follows:

1. Obtain the latest official Destatis CPI release and replace or extend [Code/data/61111-0002_en.csv](#).
2. Run `python scripts/monitor_cpi_pipeline.py` from the [Code/](#) directory.
3. Validate the source file, monthly continuity, numeric CPI values, and test-suite status.
4. Regenerate EDA outputs and retrain/evaluate the forecasting models.
5. Verify that all required model and plot artifacts exist.
6. Compare the new metric file against the stored baseline using the 15% drift threshold.
7. If all checks pass, accept the run and keep the regenerated dashboard inputs; if needed, update the baseline with `-accept-baseline` after manual review.
8. If any hard check fails, stop deployment refresh and investigate the reported failure before publishing new results.

This procedure keeps the system aligned with official monthly CPI updates while preserving reproducibility. It is still partly manual because data acquisition and deployment publication are triggered by the operator. However, it is now substantially stronger than a purely narrative monitoring plan because it is backed by a real script, a real baseline file, a real JSON monitoring report, and a focused test layer that exercises the central monitoring claims made in this chapter.

Part IX

Application Deployment

27 Application Deployment

27.1 Application Description

The deployed application is a Streamlit dashboard for interactive inspection of Germany's CPI forecasts. Its purpose is not to retrain models online, but to present the outputs of the offline forecasting pipeline in an accessible and reproducible way. Users can select a forecasting model, view actual and predicted CPI values, inspect confidence intervals, and compare evaluation metrics.

This deployment choice matters for the project because it turns the forecasting work from a static script output into a reusable analytical interface. A report table is sufficient to document who won the model comparison, but it is less useful for understanding where forecasts diverge over time, how confidence intervals widen, or how the relative performance of the models appears visually on the same holdout period. The dashboard therefore functions as the user-facing endpoint of the project: it does not replace the underlying modeling pipeline, but it makes the results easier to inspect, demonstrate, and interpret.

27.2 Structure

The application is implemented in `Code/app.py`. Its structure consists of four main parts: page configuration and introductory text, sidebar controls, metric and plot display, and explanatory sections.

27.2.1 Sidebar Controls

The sidebar is the primary interaction panel. It contains two dropdown selectors:

- **Select Model:** A dropdown menu offering four options:
 - **ARIMA (1,1,1)** – displays the ARIMA forecast with confidence intervals
 - **ETS (A,A,A)** – displays the ETS forecast with confidence intervals
 - **SARIMA (1,1,1)×(1,1,1,12)** – displays the SARIMA forecast with confidence intervals
 - **Compare All Models** – overlays all three forecasts on the same chart for direct visual comparison
- **Confidence Interval Level:** A dropdown allowing the user to select 80%, 90%, 95%, 96%, 97%, 98%, or 99% confidence bands. The default is 95%.

When the user selects an individual model, the dashboard loads the corresponding serialized `.joblib` file from `Code/saved_models/`, regenerates the 24-step forecast, and displays the forecast curve together with the selected confidence band. When the user selects **Compare All Models**, the dashboard loads all three models simultaneously, computes their forecasts on the common test period, and renders an interactive Plotly chart with all forecast curves overlaid on the actual test data. This comparison mode is particularly useful because it allows the user to see immediately which model tracks the actual CPI path most closely.

27.2.2 Dashboard Runtime Stages

At a more detailed level, the dashboard separates responsibilities into a small number of understandable runtime stages:

1. **Path resolution:** The application resolves paths relative to its own file location so that the CPI CSV and serialized models can be found reliably.
2. **Data loading:** It loads the preprocessed CPI series and applies the same chronological split that is used in the offline evaluation.
3. **Model mapping:** It maps the user-selected model name to the corresponding saved model artifact and generates forecasts and, where supported, confidence intervals.
4. **Metric computation:** It computes or recomputes the evaluation metrics shown to the user and passes the results to the plotting and layout layer.

This structure is simple enough for a student project, but also modular enough that future deployment features could be added without rewriting the whole application.

Table 27.1: Main deployment elements of the CPI dashboard.

Deployment element	Role in the application
CPI CSV file	Provides the time series used for comparison and plotting
Saved model files	Supply the forecasting backend for ARIMA, ETS, and SARIMA
Sidebar controls	Collect user input for model selection and interval level
Metrics section	Summarizes forecast accuracy numerically
Plotly chart	Visualizes actual and forecast CPI values interactively

27.3 Idea

The main deployment idea is to separate modeling from presentation. Training and evaluation are performed once in the development pipeline, whereas the Streamlit app focuses on communicating the results. This improves usability because end users do not need to understand the internal forecasting scripts in order to inspect the outcomes.

This separation also improves reproducibility. Because the app consumes already-defined data and model artifacts, a user who launches the dashboard sees results that are traceable back to the same underlying repository pipeline described in the report. The deployment layer is therefore intentionally thin: it does not silently retune models, alter the holdout horizon, or introduce a second evaluation method. Its role is to expose the already-established analytical logic through an interactive interface.

27.4 Flow Chart of Each Step in the Deployment

27.5 Dashboard Screenshots

The deployed Streamlit dashboard is shown in Figures ??–??. These screenshots were captured from the running application on `localhost:8502` and illustrate the actual user interface that results from the deployment process described in this chapter.

Figure ?? shows the complete dashboard with the sidebar visible on the left and the main content area on the right. In this view, the user has selected **ARIMA (1,1,1)** from the model dropdown and **95%** from the confidence interval dropdown. The main area displays the corresponding RMSE, MAE, and MAPE metric cards, followed by the interactive Plotly chart showing the ARIMA forecast with its 95% confidence band overlaid on the actual test data.

Figure ?? provides a focused view of the model selector in its expanded state. The available choices are the three individual model views and **Compare All Models**. This confirms that the dashboard exposes both detailed single-model inspection and direct model comparison through one sidebar control.

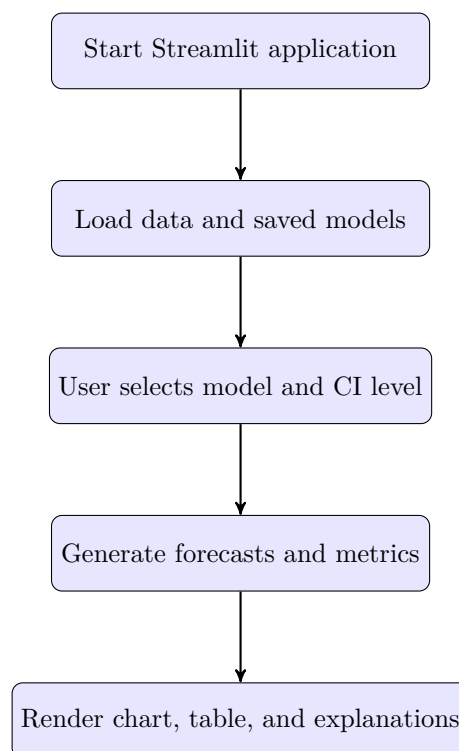


Figure 27.1: Deployment flow of the Streamlit CPI forecasting application.

Figures ?? and ?? document the two additional single-model views. They are important deployment evidence because they show that the same loading, metric, and plotting branch works not only for ARIMA but also for ETS and SARIMA. Figure ?? then shows the comparison mode, where the metric cards are replaced by a table and the chart overlays all three forecast curves.

27.5.1 Model Comparison Mode

When the user selects **Compare All Models** from the dropdown, the dashboard switches to comparison mode. In this mode, the sidebar remains the same, but the main area changes to display a metrics table highlighting the best-performing model and an interactive chart overlaying all three forecast curves on the same axes. This allows direct visual comparison of how ARIMA, ETS, and SARIMA diverge from the actual CPI path. The comparison mode is one of the most useful features of the dashboard because it makes the relative strengths and weaknesses of each model immediately visible without requiring the user to switch between individual model views manually.

27.6 ML Pipeline

The deployment layer consumes the outputs of the machine-learning pipeline rather than re-executing the full training stage. It uses the cleaned CPI series, the serialized model artifacts, and the metric logic already established in the development stage. This ensures consistency between the values shown in the dashboard and the values documented in the report.

In KDD terms, the deployment chapter belongs after data selection, preprocessing, transformation, mining, and interpretation because it depends on the artifacts created there. The app requires a valid CPI data file, successfully trained `.joblib` models, and evaluation logic that is already known to be correct. This dependency chain should be made explicit because it explains why deployment cannot be treated as an isolated front-end exercise. If the saved models are missing or inconsistent with the current data file, the

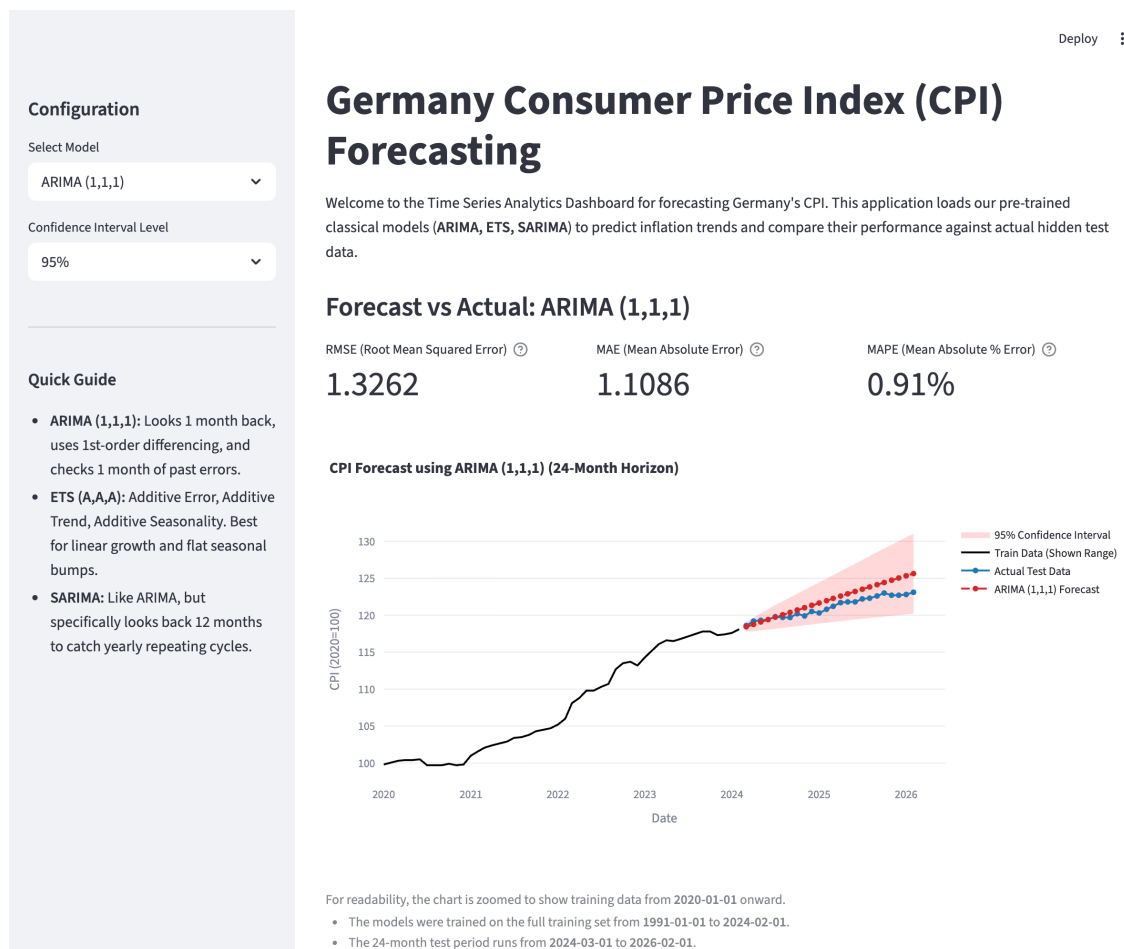


Figure 27.2: Live dashboard view for ARIMA (1,1,1), including metric cards and the forecast chart with a 95% confidence band.

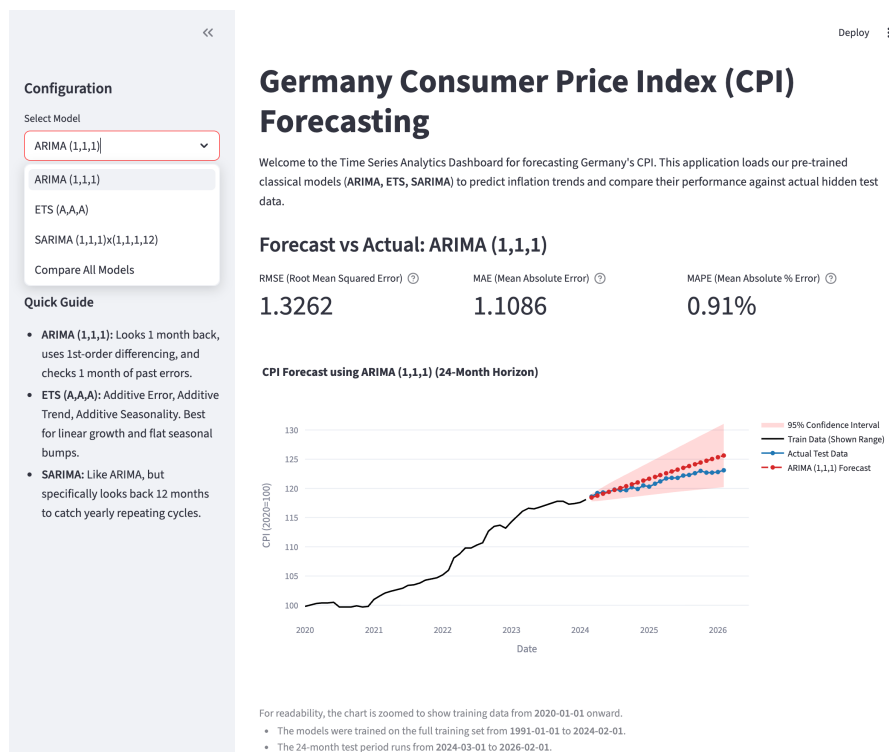


Figure 27.3: Expanded model selector in the live dashboard, showing ARIMA, ETS, SARIMA, and the all-model comparison option.

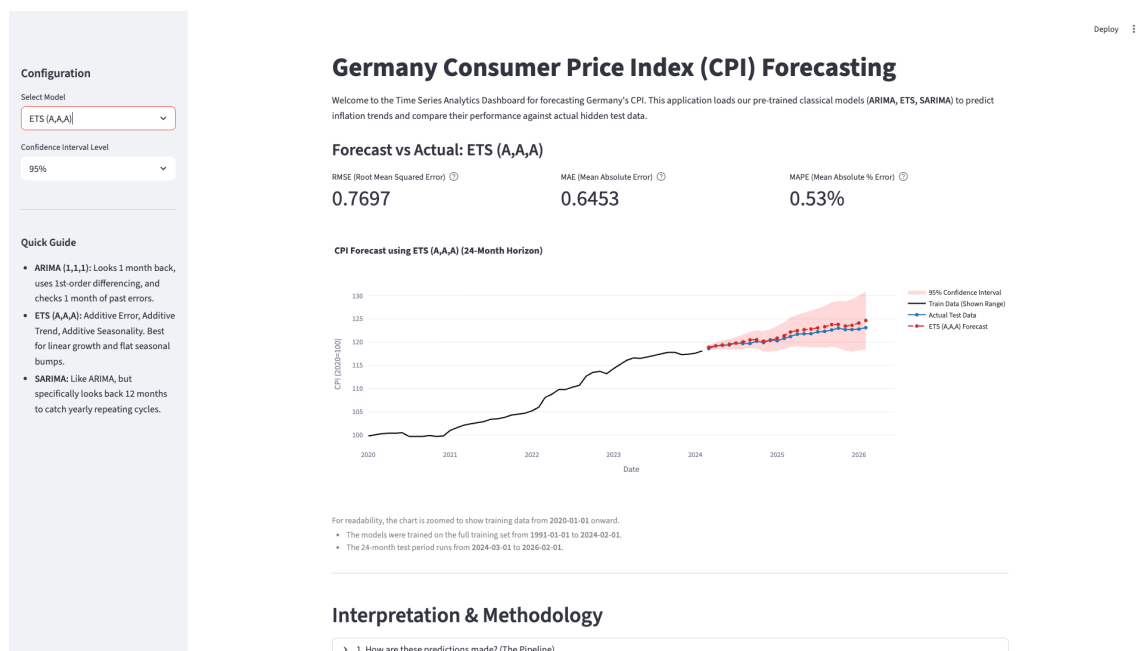


Figure 27.4: Live dashboard view for ETS (A,A,A), showing the lowest error metrics among the implemented models and the corresponding forecast chart.

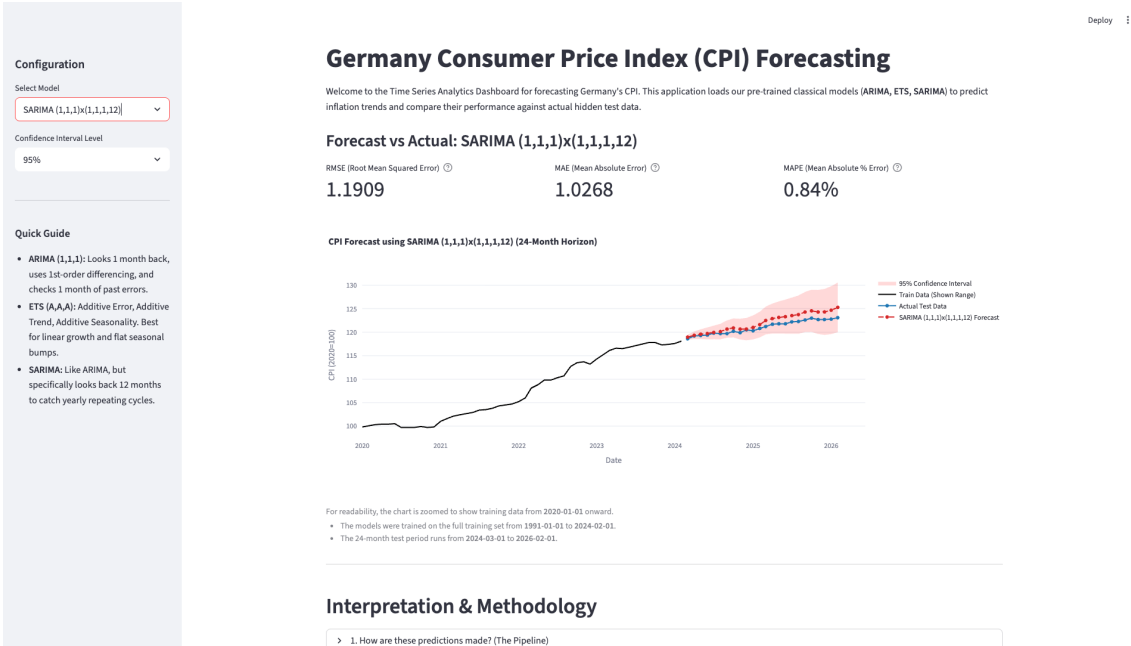


Figure 27.5: Live dashboard view for SARIMA $(1,1,1) \times (1,1,1,12)$, showing the seasonal model's metrics and forecast chart.

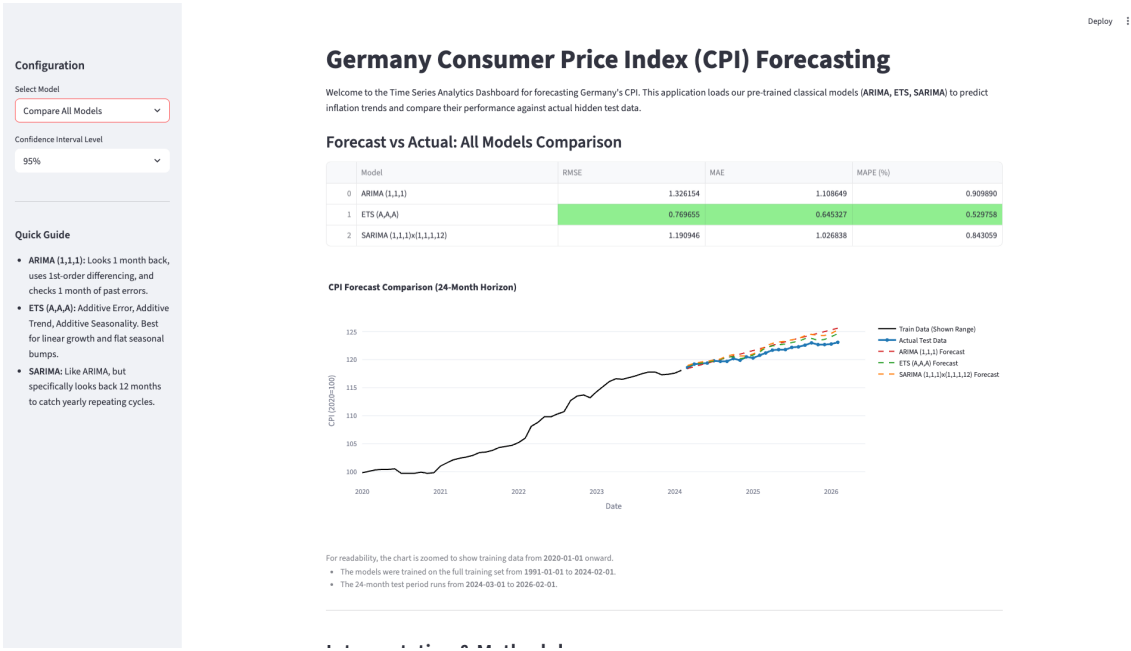


Figure 27.6: Live comparison mode showing the metrics table and an overlaid forecast chart for ARIMA, ETS, and SARIMA.

deployment layer will fail or display misleading results even if the user interface itself is implemented correctly.

27.7 Program Readability and Modules

The application code is reasonably readable because it separates configuration, data loading, model-file mapping, forecast generation, and explanatory text into recognizable blocks. The path handling is more robust than in some of the training scripts because `app.py` resolves its main directories relative to its own file location.

This readability is relevant to deployment quality because a dashboard often becomes the part of a project that other people interact with most frequently. In `Code/app.py`, the visible configuration options, the loading logic, the metric computation, and the plotting code are still located in one file, but they are organized in a sufficiently clear order that future changes remain realistic. The current structure is therefore acceptable for the project scope, although a more production-oriented version would likely separate the user-interface layer from the forecasting-service layer even more explicitly.

27.8 Parameter Handling

The dashboard exposes only deployment-relevant parameters to the user. These are the model choice and the confidence-interval level. Forecast horizon and evaluation horizon remain fixed in the underlying application logic, which preserves comparability with the report results.

This restricted parameter design is deliberate rather than accidental. Allowing the user to alter the model and uncertainty level supports meaningful interaction, while keeping the test horizon fixed protects the comparability of the results. If the app allowed arbitrary retraining windows or different evaluation horizons at runtime, the deployment view would no longer correspond directly to the report's documented evaluation setup. The current parameter handling therefore favors interpretability and reproducibility over unrestricted configurability.

27.9 Error and Message Handling

If model files are missing, the application shows a visible error message and instructs the user to run the training pipeline first. If confidence intervals cannot be calculated, the app raises a warning instead of terminating the session. In addition, the interface uses spinners, metric descriptions, and expandable explanations to guide the user through the interpretation of the results.

The deployment layer is therefore moderately robust, but not yet fully hardened. It handles several expected failure modes in a user-visible way, especially missing artifacts and interval-calculation problems. However, the app still depends on the surrounding file structure remaining stable and on the serialized models being compatible with the installed Python environment. This means that deployment robustness currently relies more on disciplined project maintenance than on a fully isolated packaging strategy. For the scope of this report, that limitation should be stated clearly rather than hidden.

27.10 Testing and Test Automation

The project includes a `pytest` test suite located in `Code/tests/`. The tests cover the data-loading layer, the model wrappers, and the monitoring workflow. Table ?? summarizes the current test coverage.

The tests can be executed from the project root with the following command:

```
1000 | pytest Code/tests/
```

Listing 27.1: Running the `pytest` suite

Table 27.2: Summary of the pytest test suite.

Test file	What it checks
<code>test_data_loader.py</code>	CPI CSV loads correctly, index is monthly MS, no missing values, file-not-found error handling
<code>test_models.py</code>	ARIMA, ETS, and SARIMA wrappers fit, predict, and return confidence intervals on synthetic data
<code>test_monitoring_workflow.py</code>	Monitoring script runs, baseline metrics are readable, drift detection triggers correctly
<code>test_ets_pred.py</code>	ETS model predictions match expected behavior
<code>test_ets_ci.py</code>	ETS confidence intervals are generated correctly
<code>test.py</code>	General sanity checks

At the time of writing, the test suite reports **10 passed** tests across the modules listed above. This indicates that the core data-loading and model-wrapper logic is protected by automated checks, which reduces the risk of silent regressions when the codebase is modified.

27.10.1 CI/CD Integration

While the project does not currently use a continuous-integration platform such as GitHub Actions or GitLab CI, the test suite is structured so that it could be integrated into one with minimal effort. A typical CI/CD workflow for this project would consist of the following steps:

1. **Environment setup:** Install Python dependencies from [Code/requirements.txt](#).
2. **Linting:** Run `flake8` or `ruff` to check code style and catch syntax issues.
3. **Unit tests:** Execute `pytest Code/tests/` to verify data loading, model fitting, and monitoring logic.
4. **Coverage check:** Use `coverage.py` to ensure that critical paths in [Code/src/](#) are exercised by the test suite.
5. **Pipeline execution:** Run [Code/src/modeling_pipeline.py](#) to confirm that the full training and evaluation workflow completes without error.
6. **Artifact verification:** Check that [Code/saved_models/](#) and [Code/plots/](#) contain the expected files after pipeline execution.

Figure ?? shows a conceptual CI/CD pipeline that could be applied to this project. The diagram is not a description of the current setup, but rather a blueprint for how the existing tests and scripts could be orchestrated into an automated deployment workflow.

The absence of a live CI/CD pipeline is acknowledged as a current limitation. The project relies on manual execution of tests and scripts, which is acceptable for a student project but would need to be automated before the system could be described as production-ready. The test suite and the modular script structure are, however, deliberate preparatory steps in that direction.

27.11 Deployment Test Protocol

Formal automated application tests are still limited, but the deployment can be validated through a practical protocol: confirm that the saved model files exist, start Streamlit successfully, verify that each model can be loaded, and check that the dashboard renders charts and metrics without runtime failure. A more explicit deployment test protocol can be stated as follows. First, verify that the CPI CSV file and all expected model artifacts exist in their designated directories. Second, launch the dashboard from

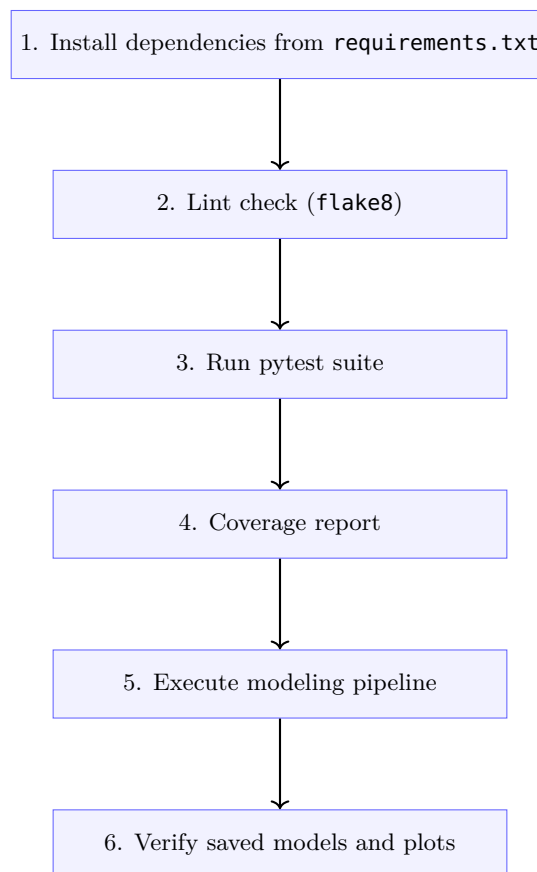


Figure 27.7: Conceptual CI/CD pipeline for the CPI forecasting project.

the intended working directory and confirm that the application starts without import or path errors. Third, test each model option individually and then the comparison mode so that every loading branch in `app.py` is exercised. Fourth, change the confidence-interval selector and verify that the chart updates without breaking the metric display. Fifth, compare at least one visible metric set against the offline evaluation outputs to confirm consistency. This protocol is still lightweight, but it provides a clearer bridge between development validation and practical deployment testing.

Future automated deployment tests should focus on the most failure-prone paths. These include checking whether the saved models can be deserialized, whether the dashboard still renders when confidence intervals are unavailable, and whether key user-interface elements appear after launch. Such tests would strengthen the report's deployment section because they would transform the current manual validation process into a repeatable testable deployment workflow.

Part X

Appendix

28 Bill of Material

28.0.1 Hardware Bill of Materials

The following hardware components are required to execute the CPI forecasting project. The hardware listed below provides a complete development environment for data analysis, model training, and result visualization.

Materials	Description	Quantity	Link	Price
	A standard personal Laptop capable of running data analysis	1	Laptop	500 € to 1000 €
	Keyboard.	1	Keyboard	29.58 €
	Mouse.	1	Mouse	7.49 €

Table 28.1: Hardware Bill of Materials

28.0.2 Laptop

The laptop is the primary hardware component used to execute the CPI forecasting project. It provides the computational resources necessary for running the Python environment, performing statistical modelling, and generating visualizations.

- Brand and Model: Dell XPS / Lenovo ThinkPad / MacBook Pro
- Processor: Intel Core i5/i7 or AMD Ryzen 5/7
- RAM: Minimum 8GB (16GB recommended)
- Storage: 512GB SSD minimum
- Operating System: Windows 10/11, macOS, or Linux
- Display: 15" Full HD (1920x1080) or higher

28.0.3 Keyboard

An external keyboard improves typing efficiency and ergonomics during coding and report writing.

- Brand and Model: Keychron K2

- Type: Wireless mechanical keyboard
- Connectivity: Bluetooth / USB-C
- Layout: Tenkeyless
- Battery: Rechargeable battery

28.0.4 Mouse

A mouse enhances navigation efficiency when working with development tools and visualizations.

- Brand and Model: Logitech MX Master Series
- Type: Wireless optical mouse
- Connectivity: Bluetooth / USB receiver
- Battery: Rechargeable battery
- Ergonomics: Designed for long working sessions

28.0.5 Hardware Note

The CPI forecasting project is a software-first application. The computational requirements are modest: the dataset contains only 422 monthly observations, and the models (ARIMA, ETS, SARIMA) are classical statistical methods that do not require GPU acceleration or large amounts of memory. Any standard laptop or desktop computer from the last five years is sufficient for development, training, and deployment. The hardware BOM is therefore included for completeness, but it is not a critical project constraint.

29 SBOM

29.0.1 Software Bill of Materials

The software environment used in this project supports data preprocessing, time-series modelling, visualization, and reproducible experimentation. The tools listed below form the complete development environment used for implementing the CPI forecasting models.

Software	Description	Quantity	Price
	Lightweight and extensible source code editor used for Python development, debugging, and project implementation.	1	Free
	Primary programming language used for data preprocessing, statistical modelling, and CPI forecasting implementation.	1	Free
	Distributed version control system used to track code changes and manage project versions.	1	Free
	Online platform used to host the project repository and maintain project documentation and version history.	1	Free
	Libraries including Pandas, NumPy, Matplotlib, Statsmodels, and Scikit-learn used for data processing, visualization, and forecasting models.	1	Free

Table 29.1: Software Bill of Materials

29.0.2 Python Environment

Python is the primary programming language used in this project. It provides a large ecosystem of libraries for statistical modelling, machine learning, and data analysis.

- Python Version: Python 3.11 or later
- Package Manager: pip
- Libraries Used:
 - Pandas – Data manipulation and time-series handling
 - NumPy – Numerical computation
 - Matplotlib – Data visualization
 - Statsmodels – Implementation of ARIMA, SARIMA, and ETS forecasting models
 - Scikit-learn – Evaluation metrics and model validation

29.0.3 Project Dependencies

The complete set of Python dependencies is recorded in [Code/requirements.txt](#). This file ensures that the software environment can be reproduced on any machine with a compatible Python installation. Listing ?? reproduces the current dependency list.

```

1000 pandas
      matplotlib
1002 statsmodels
      scikit-learn
1004 jinja2
      joblib
1006 streamlit
      plotly

```

Listing 29.1: Project dependencies from `Code/requirements.txt`

Table ?? provides a brief description of each dependency and its role in the CPI forecasting pipeline.

Table 29.2: Description of Python dependencies used in the project.

Package	Role in the project
pandas	Data loading, cleaning, time-index construction, and CSV export
matplotlib	Static plot generation for EDA and forecast comparison
statsmodels	ARIMA, ETS, and SARIMA model estimation and forecasting
scikit-learn	RMSE, MAE, and MAPE evaluation metrics
jinja2	Templating engine (transitive dependency via pandas and plotly)
joblib	Serialization and deserialization of trained model objects
streamlit	Interactive dashboard deployment
plotly	Interactive chart rendering inside the dashboard

Version pinning note: The current `requirements.txt` does not specify exact version numbers. For maximum reproducibility, a production deployment would pin each package to a tested version (e.g., `pandas==2.1.0`). The current unpinned format is acceptable for a student project because it allows compatibility with the latest stable releases, but it means that future installations may use slightly different package versions than those originally tested.

29.0.4 Visual Studio Code (VS Code)

Visual Studio Code (VS Code) is used as the primary development environment for implementing the CPI forecasting models.

1. **User Interface:** VS Code provides a customizable interface that supports themes, layouts, and workspace configuration.
2. **Extensions and Marketplace:** Extensions such as the Python extension and Git integration provide additional development tools and debugging support.
3. **Integrated Terminal:** Developers can execute Python scripts and command-line operations directly within the editor.
4. **IntelliSense:** Provides intelligent code completion, parameter hints, and syntax suggestions.
5. **Debugging Support:** Enables breakpoints, variable inspection, and step-by-step execution of Python programs.
6. **Version Control Integration:** Built-in Git integration allows efficient version control and code management.

29.0.5 Version Control and Documentation

Version control is implemented using Git and GitHub. All project code, scripts, and configuration files are stored in a structured repository.

The repository includes:

- Source code for data preprocessing and forecasting models
- Documentation describing the project setup and execution steps
- Configuration files specifying software dependencies

This documentation ensures that the project environment can be reproduced and that the experiments and results can be verified by other researchers.

Literature

- [Agg17] C. C. Aggarwal. *Outlier Analysis*. Springer, 2017.
- [Arm01] J. S. Armstrong. *Principles of Forecasting: A Handbook for Researchers and Practitioners*. Springer Science & Business Media, 2001.
- [Box+15] G. E. P. Box et al. *Time Series Analysis: Forecasting and Control*. Wiley, 2015.
- [Bro17] J. Brownlee. *Introduction to Time Series Forecasting with Python: How to Prepare Data and Develop Models to Predict the Future*. Machine Learning Mastery, 2017.
- [CBK09] V. Chandola, A. Banerjee, and V. Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys (CSUR)* 41.3 (2009), pp. 1–58.
- [Cha13] C. W. Chase. *Demand-Driven Forecasting: A Structured Approach to Forecasting*. John Wiley & Sons, 2013.
- [FPSS96] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. “The KDD Process for Extracting Useful Knowledge from Volumes of Data”. In: *Communications of the ACM* (1996). Ed. by ACM. URL: <https://doi.org/10.1145/240455.240464> (accessed on Mar. 25, 2026).
- [HA18] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 2018. URL: <https://otexts.com/fpp2/> (accessed on May 2, 2026).
- [HA21] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2021.
- [HTF09] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.
- [Ham94] J. D. Hamilton. *Time Series Analysis*. Princeton University Press, 1994.
- [Har+20] C. R. Harris et al. “Array Programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. URL: <https://www.nature.com/articles/s41586-020-2649-2> (accessed on Apr. 20, 2026).
- [Hu25] Y. Hu. “Consumer Price Index Prediction by ARIMA and ETS”. In: *Highlights in Business, Economics and Management* 65 (2025), pp. 223–227.
- [Kar21] A. Karasan. *Machine Learning for Financial Risk Management with Python: Algorithms for Modeling Risk*. O’Reilly Media, 2021.
- [MB18] S. Makridakis and G. E. P. Box. *Statistical Methods for Forecasting*. Wiley, 2018.
- [MMM25] E. W. Mchukwa, S. K. Mbwambo, and Z. B. Mchomvu. “Forecasting Consumer Price Index (CPI): Empirical Analysis”. In: *Economic Insights – Trends and Challenges* 14.1 (2025), pp. 73–83.
- [NSU25] M. E. Nalici, I. Soylemez, and R. Unlu. “Forecasting the Consumer Price Index in Turkey Using Machine Learning Models: A Comparative Analysis”. In: *Gazi University Journal of Science* (2025).
- [SS17] R. H. Shumway and D. S. Stoffer. *Time Series Analysis and Its Applications: With R Examples*. Springer, 2017.
- [SW99] J. H. Stock and M. W. Watson. “Forecasting Inflation”. In: *Journal of Monetary Economics* 44.2 (1999), pp. 293–335.
- [Sta22] Statistisches Bundesamt (Destatis). *Impact of the Corona Crisis on the CPI/HICP Price Collection*. Federal Statistical Office of Germany, 2022.

-
- [Sta23a] Statistisches Bundesamt (Destatis). *Consumer Price Index for Germany: Weighting Pattern for Base Year 2020*. Federal Statistical Office of Germany, 2023.
- [Sta23b] Statistisches Bundesamt (Destatis). *Data on Energy Price Trends: Long-Time Series from January 2005 to December 2022*. Federal Statistical Office of Germany, 2023. URL: <https://www.destatis.de/EN/Themes/Economy/Prices/Publications/Downloads-Energy-Price-Trends/energy-price-trends-pdf-5619002.pdf> (accessed on May 16, 2026).
- [Sta23c] Statistisches Bundesamt (Destatis). *Quality Report: Consumer Price Index for Germany*. Federal Statistical Office of Germany, Dec. 13, 2023. URL: https://www.destatis.de/EN/Methods/Quality/QualityReports/Prices/consumer-price-index.pdf?__blob=publicationFile.
- [Str26] Streamlit developers. *Streamlit Documentation*. 2026. URL: <https://docs.streamlit.io/> (accessed on May 5, 2026).
- [Wil07] P. Wilmott. *Paul Wilmott Introduces Quantitative Finance*. Wiley, 2007.
- [sci26] scikit-learn developers. *scikit-learn: Machine Learning in Python – Documentation*. 2026. URL: <https://scikit-learn.org/stable/> (accessed on May 2, 2026).
- [sta26] statsmodels developers. *statsmodels: Statistics in Python – Documentation*. 2026. URL: <https://www.statsmodels.org/stable/> (accessed on May 5, 2026).

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.
If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.